Deep Learning 25/26

Master's Degree in Artificial Intelligence

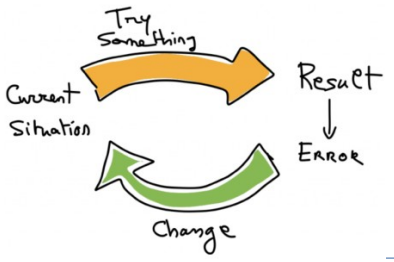
Topic U6: Reinforcement Learning

Lecture 1:
Intro to Reinforcement Learning

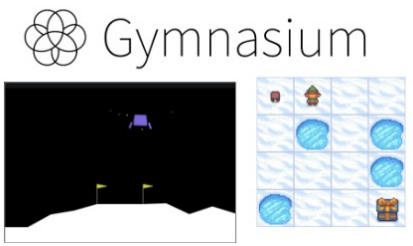
2025/2026

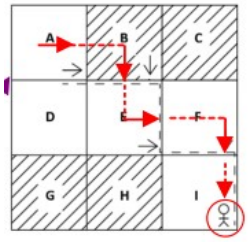
David Olivieri
Leandro Rodriguez Liñares
Uvigo, E.S. Informatica

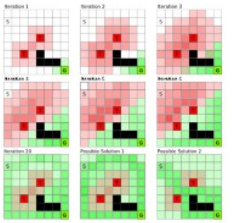
1. Introductio to RL and motivations



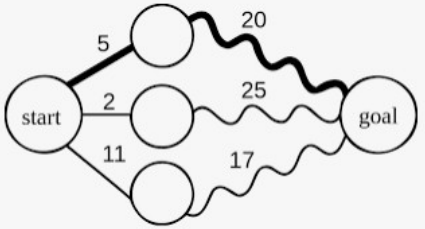
2. Practical building blocks for RL





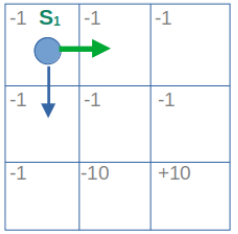


3. Some Definitions and theory



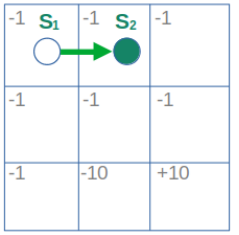
$$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

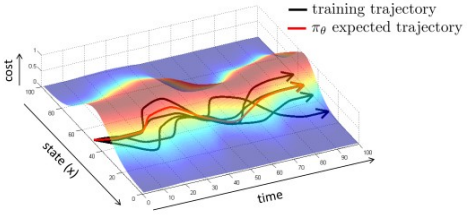
4. Solving Q-Learning With a DNN



Agent takes a decision based upon a behavior policy; in this case random

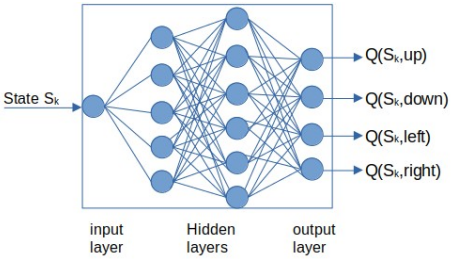
Step 1.

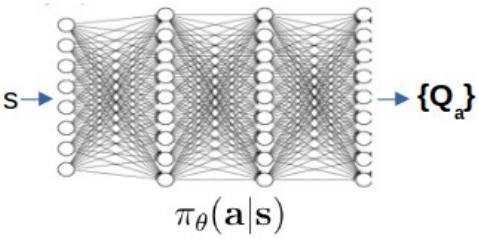




5. Implementation of a DQN

Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	0.815	2.1	1.3
S_2	0.5	1.267	-0.5	1.5
S_6	-1.2	1.2	0.7	2.54

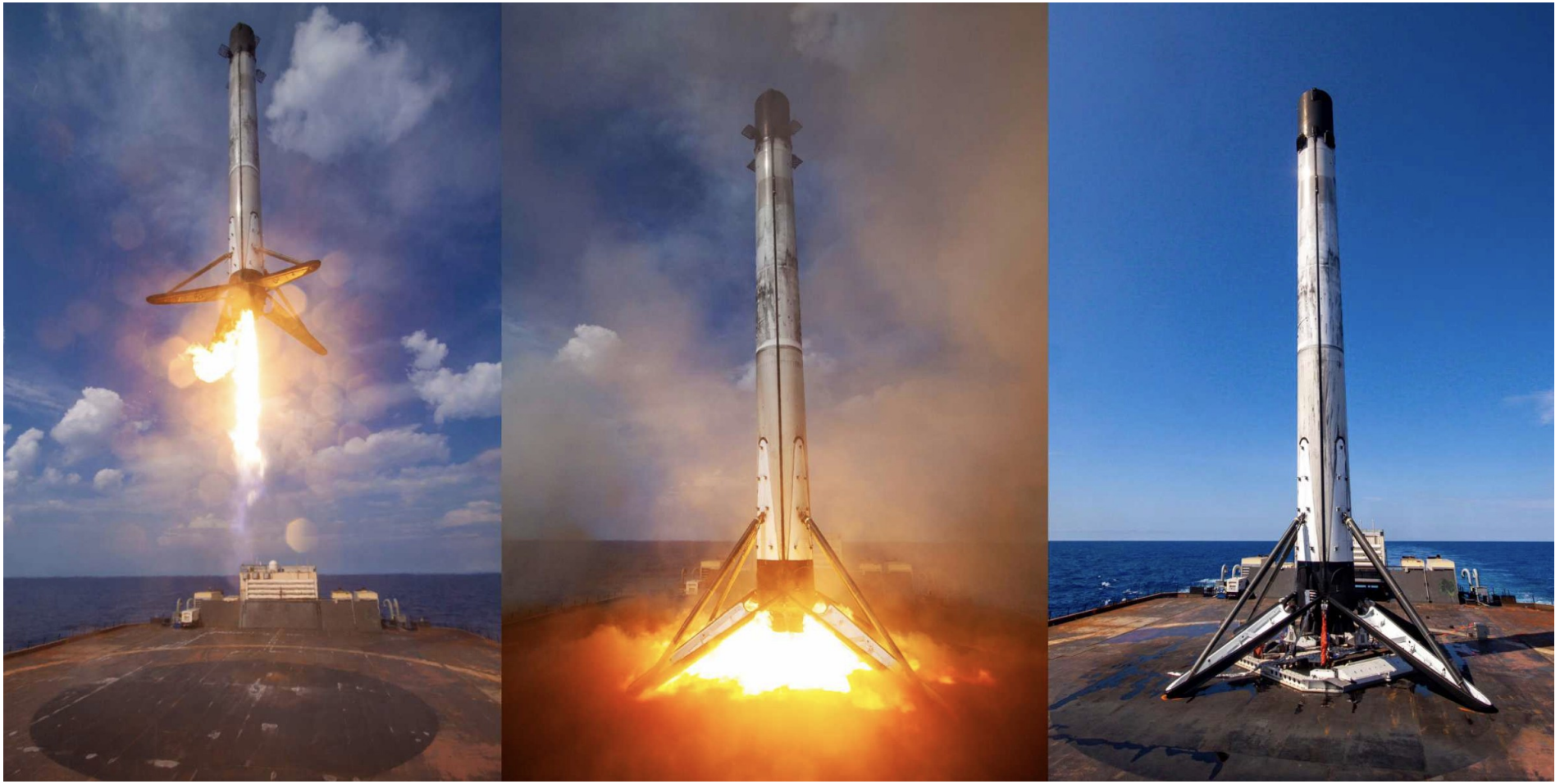




1

Introduction to RL

Capture of SpaceX booster: Oct. 13, 2024

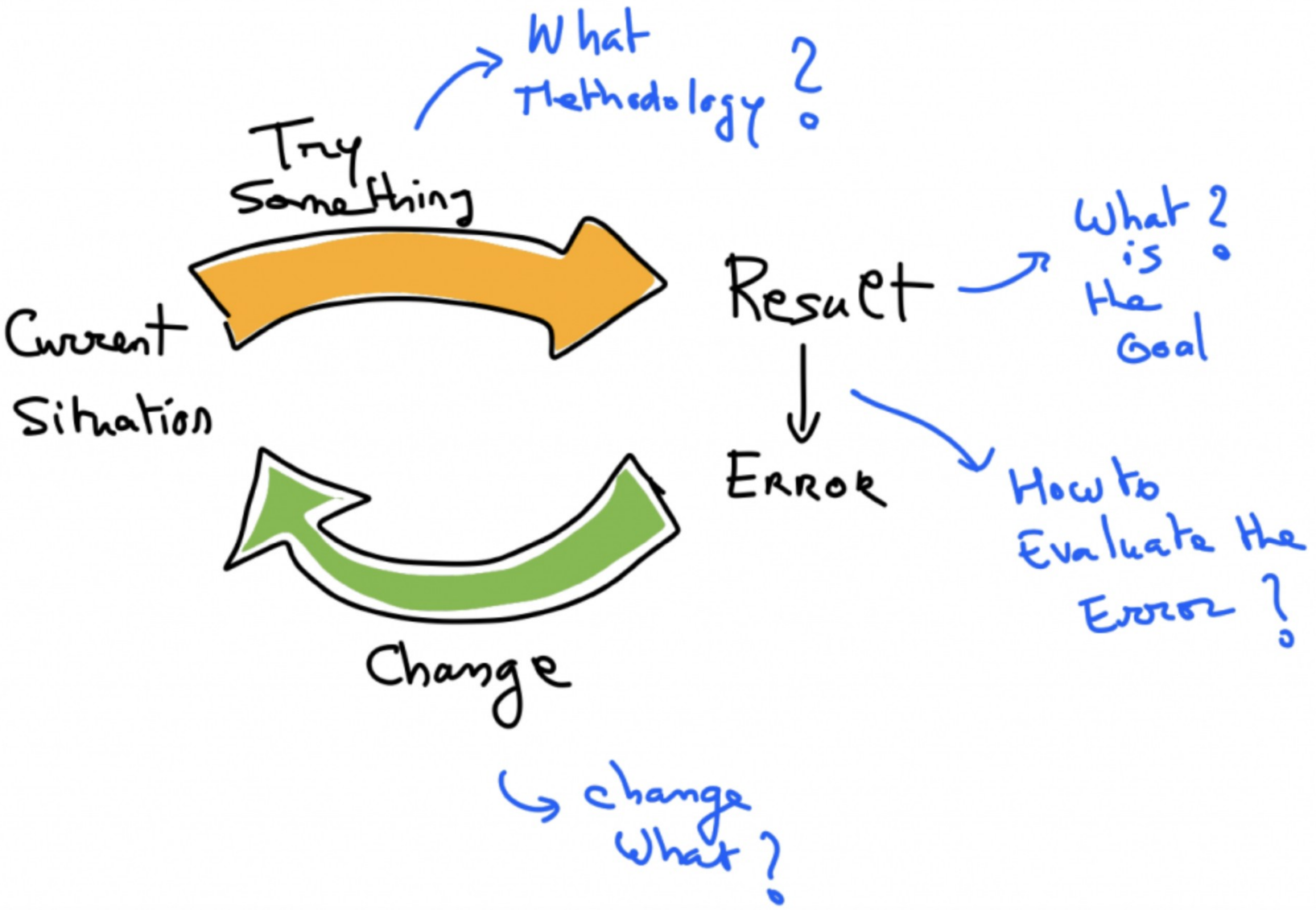


autonomous spaceport drone ship (ASDS)

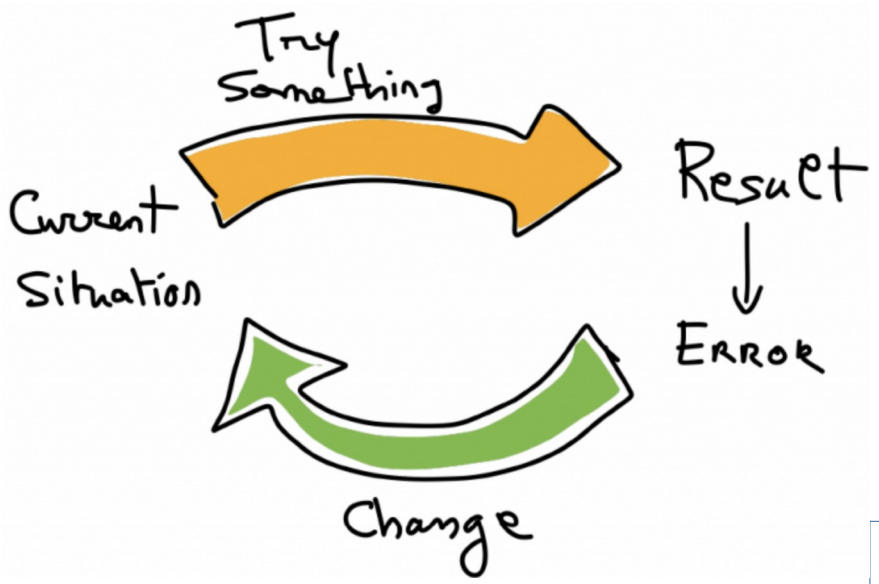
Vessel	Home Port	Status
Just Read The Instructions (I)	—	Scrapped
Of Course I Still Love You	Long Beach	Active
Just Read The Instructions (II)	Port Canaveral	Active
A Shortfall of Gravitas	Port Canaveral	Active

https://en.wikipedia.org/wiki/Autonomous_spaceport_drone_ship

How to learn


What Methodology ?
Try Something
Current Situation
Result
What ? is the Goal
Error
How to Evaluate the Error ?
Change
change What ?

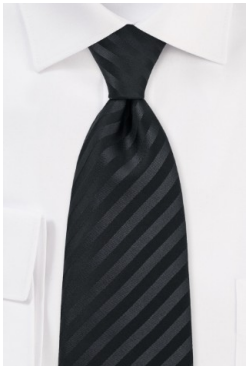
What is RL?



Very simple idea:

Learn by trial and error

More formal



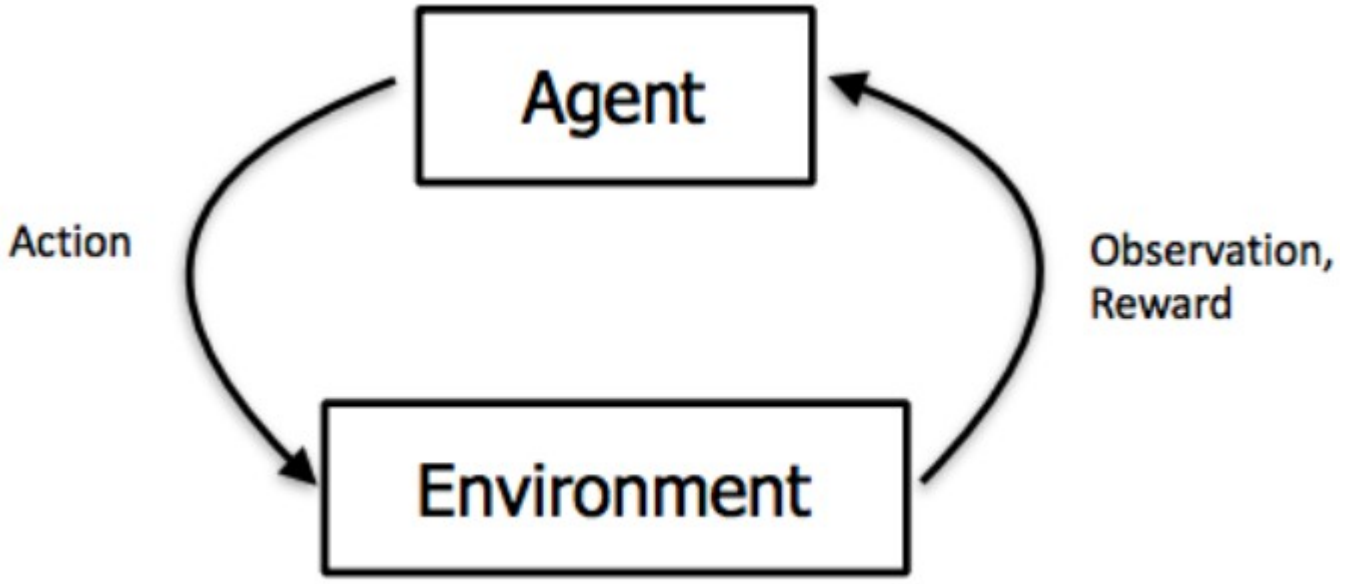
Goal-Oriented Learning: RL focuses on training an agent to make sequences of decisions to maximize cumulative rewards.

Agent-Environment Interaction: An agent interacts with an environment, taking actions, and receiving feedback in the form of rewards.

Trial and Error: RL learns optimal strategies through trial and error, adjusting actions based on past successes and failures.

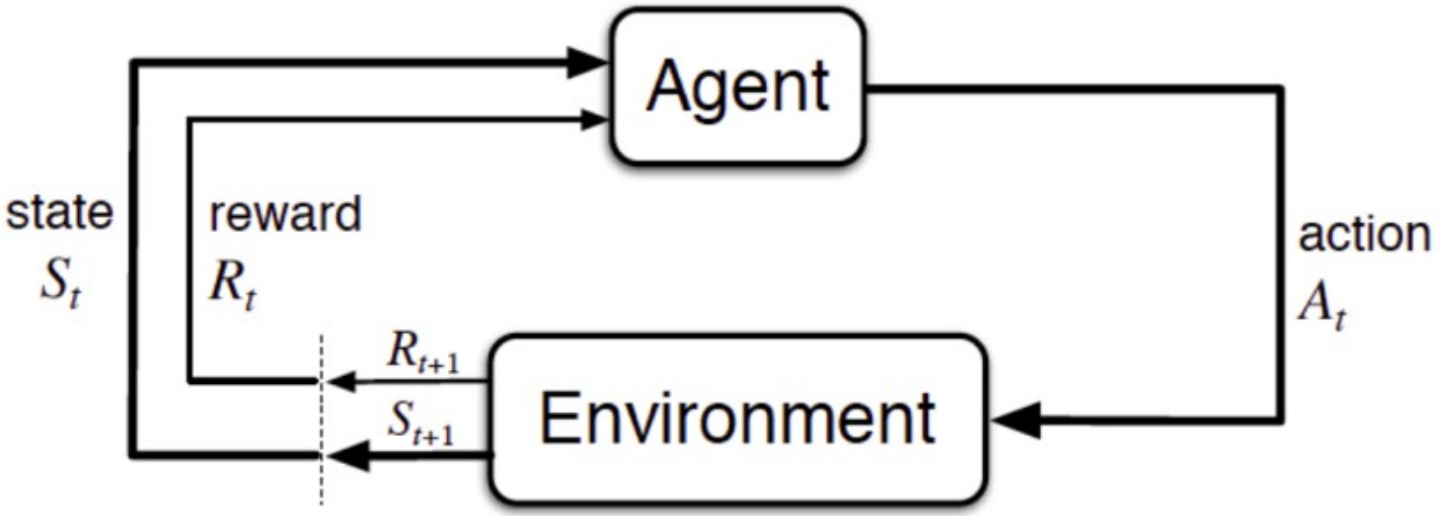
Exploration vs. Exploitation: Balances exploring new actions to discover potentially higher rewards and exploiting known actions to maximize immediate reward

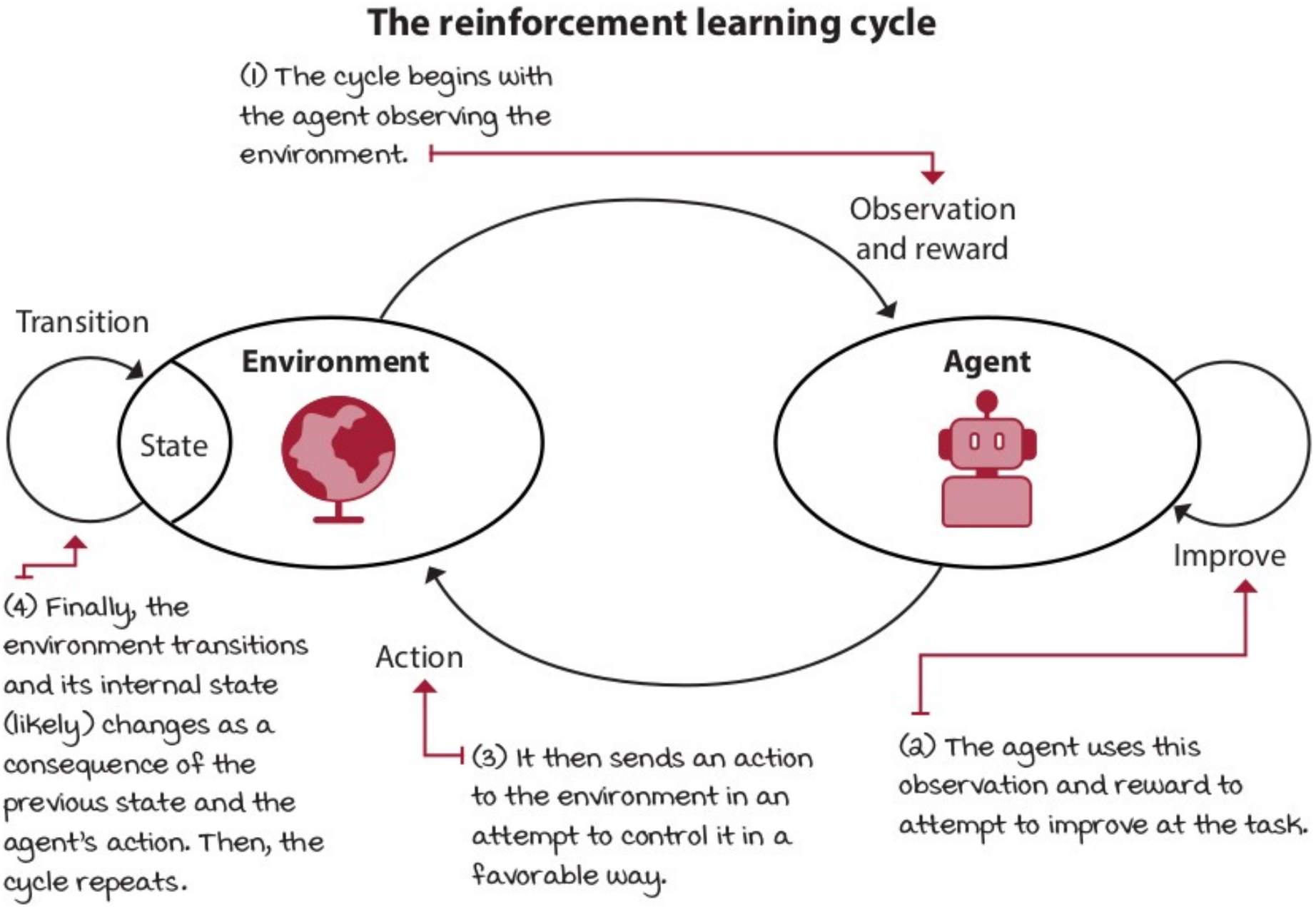
Language and model of reinforcement learning



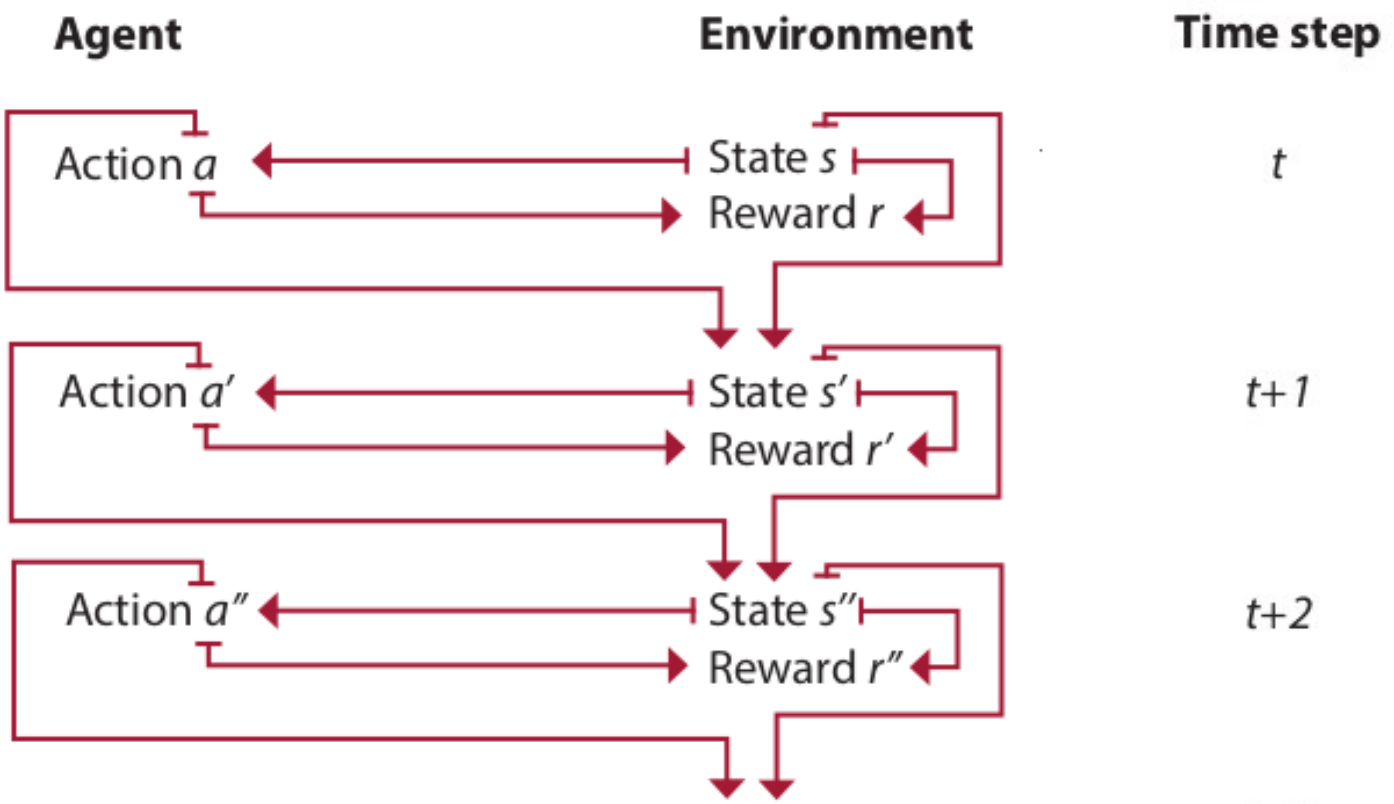
Language and model of reinforcement learning

- Learn by interacting
- (state, action) → reward value




The reinforcement learning cycle
(1) The cycle begins with the agent observing the environment.
Observation and reward
Transition
Environment
State
Agent
Improve
(4) Finally, the environment transitions and its internal state (likely) changes as a consequence of the previous state and the agent's action. Then, the cycle repeats.
Action
(3) It then sends an action to the environment in an attempt to control it in a favorable way.
(2) The agent uses this observation and reward to attempt to improve at the task.

Learning is an iterative process



Experiences:

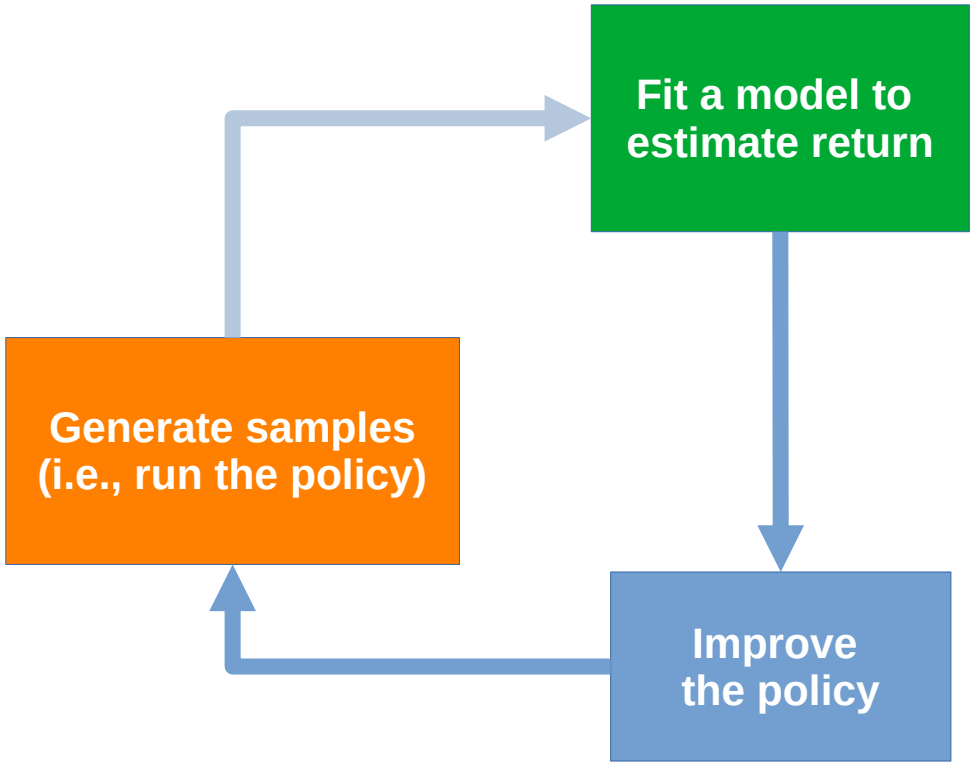
t, (s, a, r′, s′)

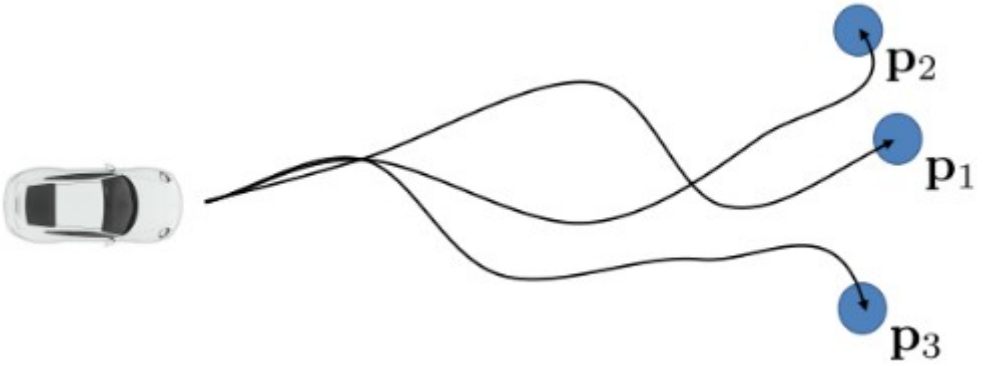
t+1, (s′, a′, r″, s″)

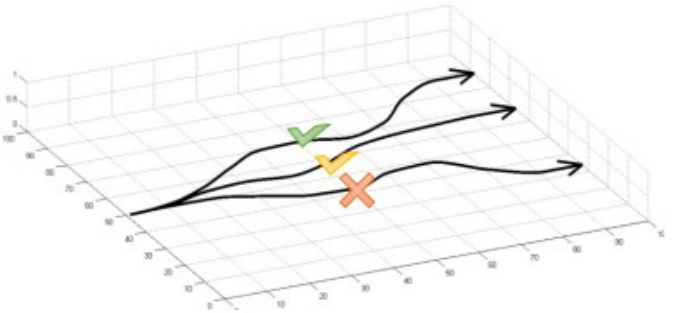
t+2, (s″, a″, r‴, s‴)

...

...


Fit a model to estimate return
Generate samples (i.e., run the policy)
Improve the policy


$\mathbf{p}_2$
$\mathbf{p}_1$
$\mathbf{p}_3$



Dynamic decisions-making in Aerospace

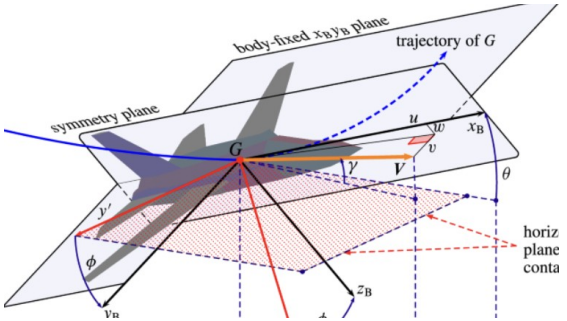
Some Application Areas

- **Real-Time Adjustments**: RL enables quick decision-making in changing flight conditions (e.g., turbulence, weather shifts).
- **Fault Detection and Mitigation:** Automatically responds to system failures to maintain safe operation.
- **Fuel Efficiency Optimization:** Learns to adjust flight patterns for optimal fuel usage over time.

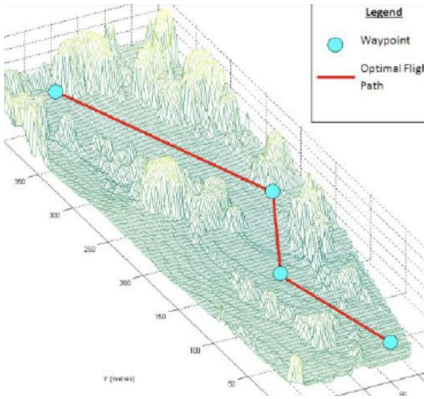
Navigation & Control

Application Areas

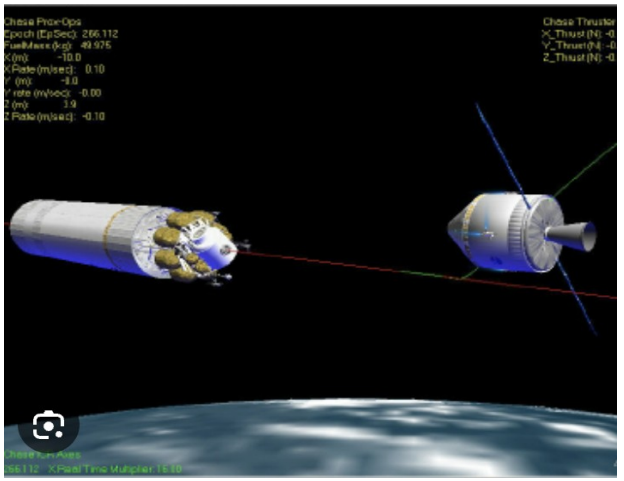
- **Autonomous Navigation:** RL guides aircraft or spacecraft to destinations, avoiding obstacles.



- **Trajectory Planning:** Learns efficient, safe flight paths under various constraints (e.g., air traffic).



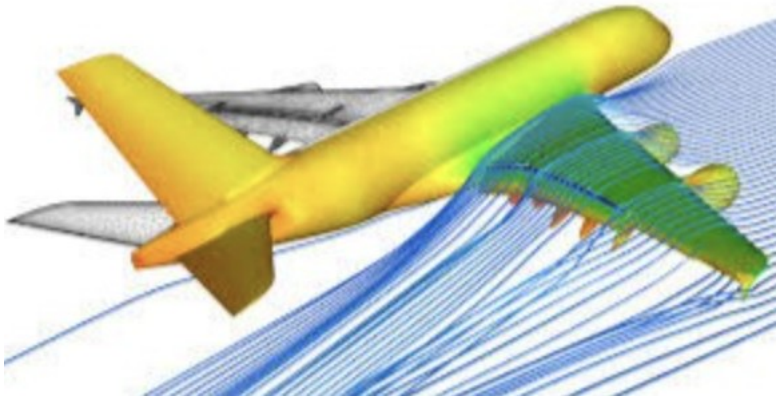
- **Landing and Docking:** Handles precise maneuvers, like satellite docking or autonomous landings.



Application Areas

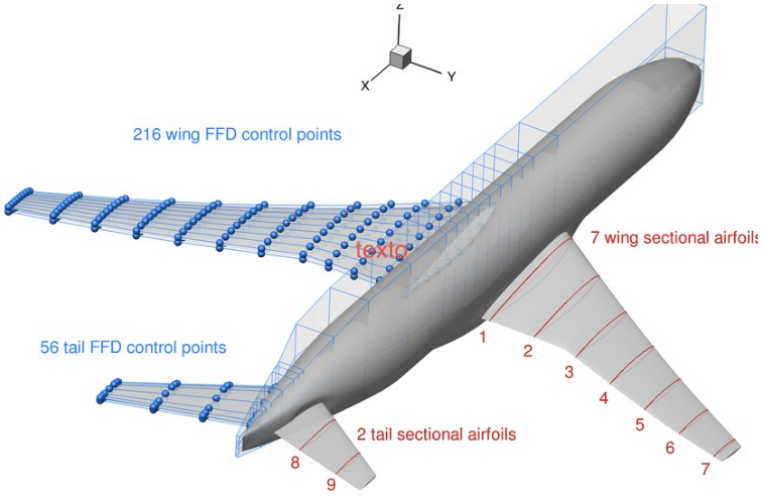
Mechanical Design and aeronautical optimization

- **Aerodynamic Optimization:** RL can iteratively improve wing or fuselage shapes for better lift and reduced drag.



- **Control Surface Design:** Learns to optimize control surfaces (e.g., ailerons, rudders) for stability and maneuverability.

- **Failure-Resistant Structures:** Designs resilient structures that can adapt to unexpected stressors.



Digital Twins

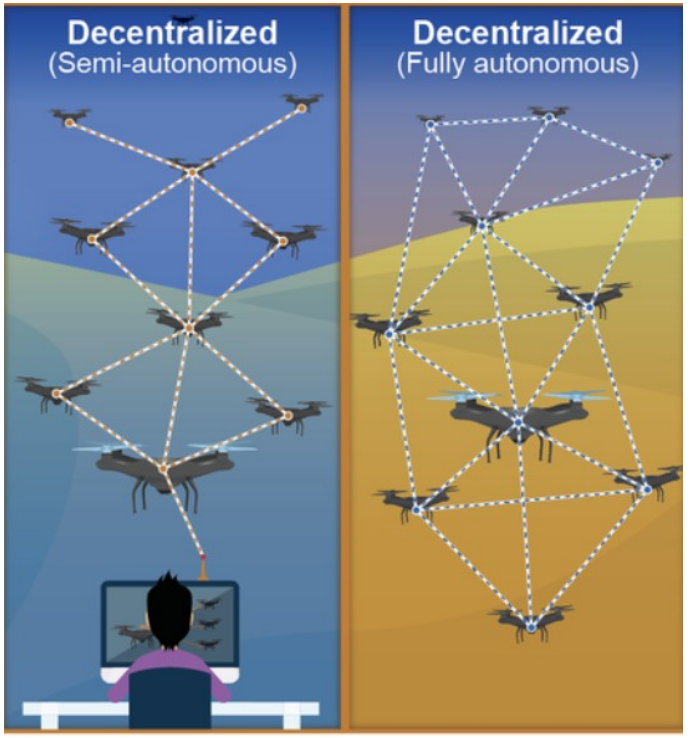
Application Areas

- **Real-Time Simulation:** Digital twins create real-time, virtual models of aircraft or spacecraft for continuous monitoring.
- **Predictive Maintenance:** Uses real-time data to predict and prevent failures, reducing downtime and improving safety.
- **Optimized Control:** RL allows digital twins to test control adjustments virtually before applying them to physical systems.
- **Data-Driven Enhancements:** Enables continuous learning and adaptation, improving system performance based on simulated feedback.

Multi-Agent Systems Synchronization

Application Areas

- **Swarm Coordination:** RL helps coordinate multiple agents (e.g., drones) to work together, maintain formation, and achieve objectives.
- **Collision Avoidance:** Multi-agent RL teaches drones or aircraft to avoid each other dynamically in shared airspace.
- **Task Allocation:** Assigns roles to agents within a swarm, optimizing for mission goals (e.g., area coverage or search-and-rescue).
- **Decentralized Decision-Making:** Each agent learns to make local decisions that benefit the group, achieving robust, scalable performance.



2

Practical elements for building an RL

The Overall Parts

1. The Environment

2. The Rewards Graph Structure

3. Learning and Policy algorithm

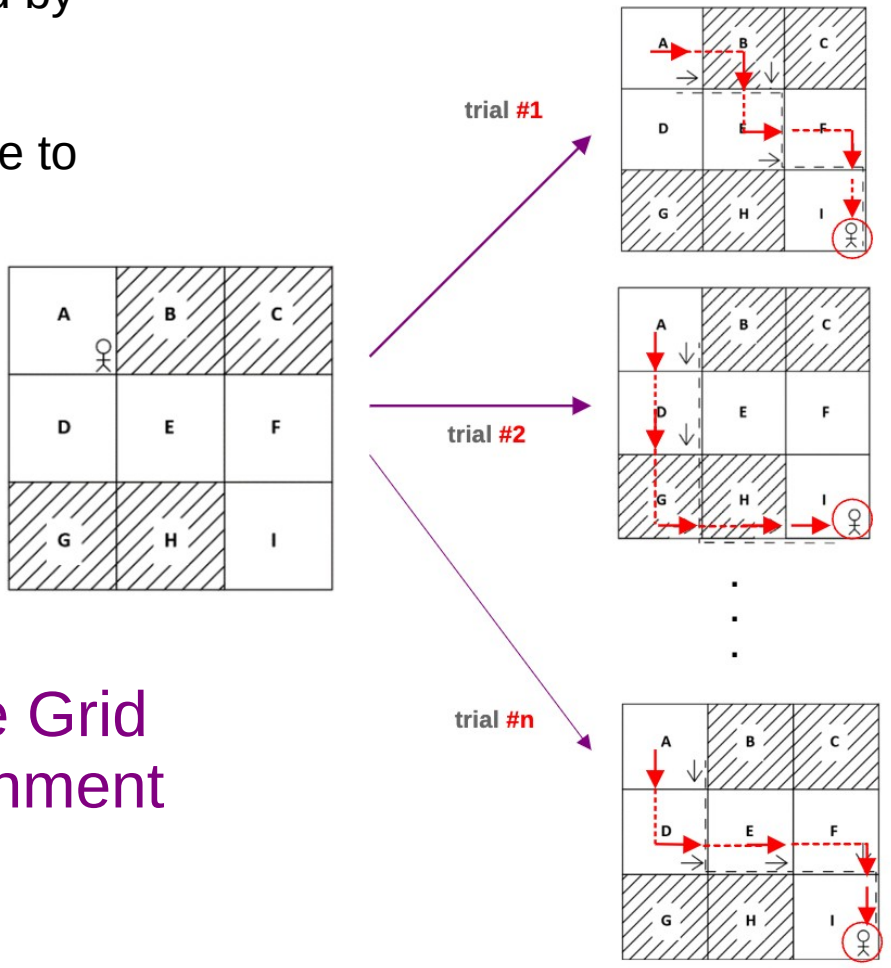
1. The Environment

Description:

- **States:** Complete representation of the environment at a given time.
- **Observations:** Partial or noisy information derived from the state.
- **Actions:** Decisions or controls executed by the agent to influence the environment.
- **Environment Dynamics:** How actions transition the environment from one state to another.

Purpose:

- Defines the setting in which the agent operates.
- Establishes the rules and constraints of interaction.



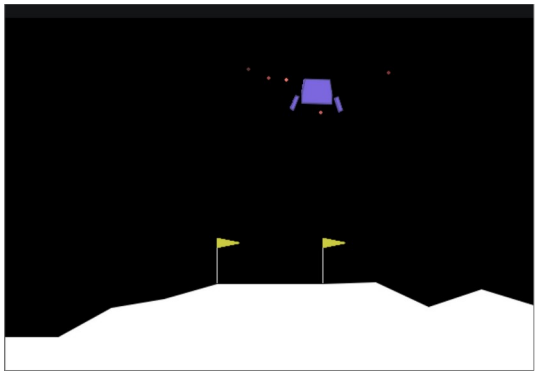
A very simple Grid World environment

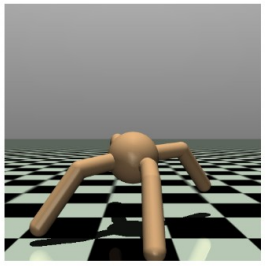
Example Environment & Agent

Toy models for reinforcement

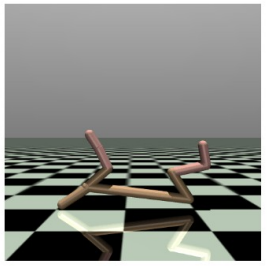
Gymnasium

An API standard for reinforcement learning with a diverse collection of reference environments

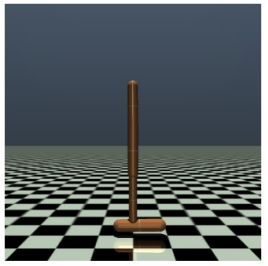




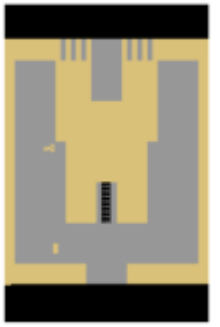
Ant



Half Cheetah

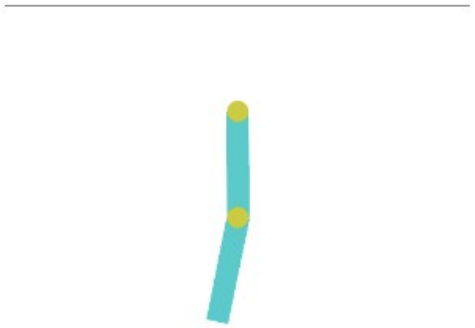


Hopper

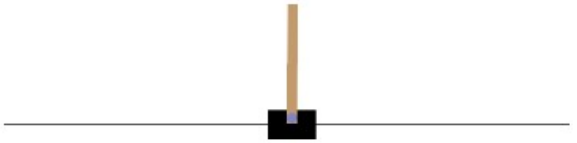


Atari games

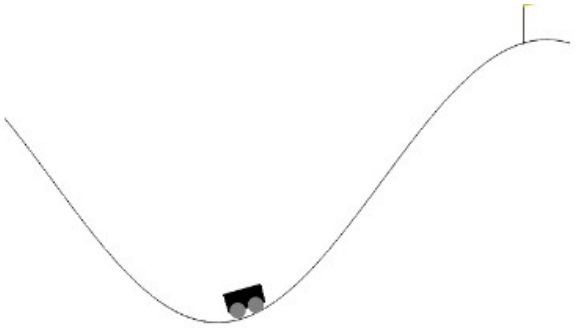
Classic control



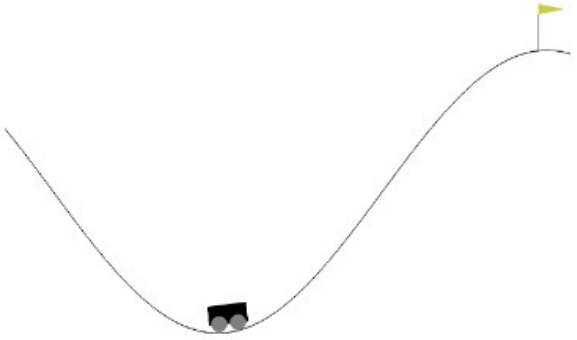
Acrobot



Cart Pole



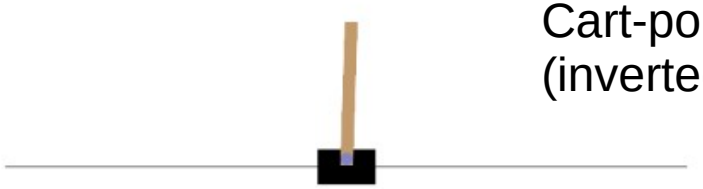
Mountain Car Continuous



Mountain Car

Pendulum

Classic control



Cart-pole
(inverted pendulum)

Actions:

0: Push cart to the left

1: Push cart to the right

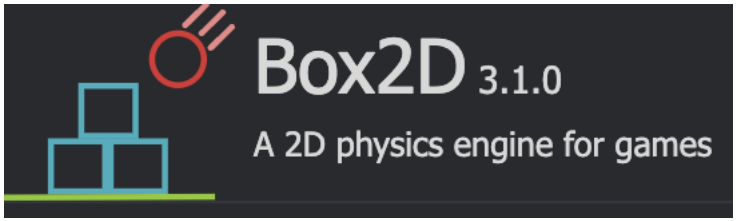
Action Space	`Discrete(2)`
Observation Space	`Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)`
import	`gymnasium.make("CartPole-v1")`

Observation state

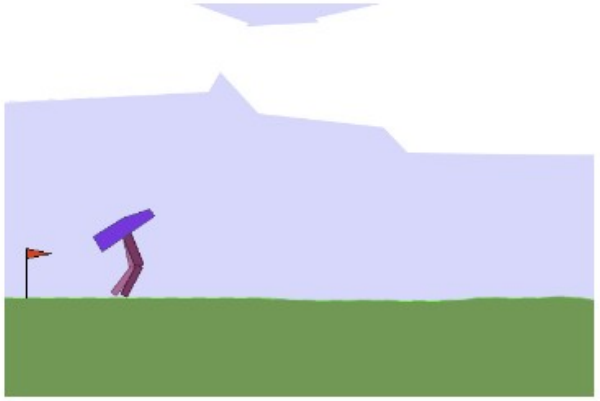
Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~ -0.418 rad (-24°)	~ 0.418 rad (24°)
3	Pole Angular Velocity	-Inf	Inf

Rewards, Start state, End State?

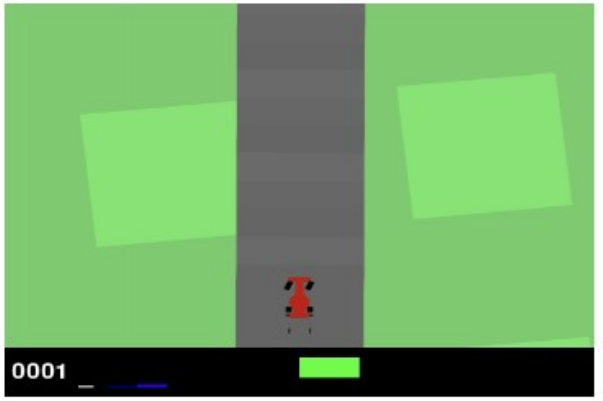
Box2D



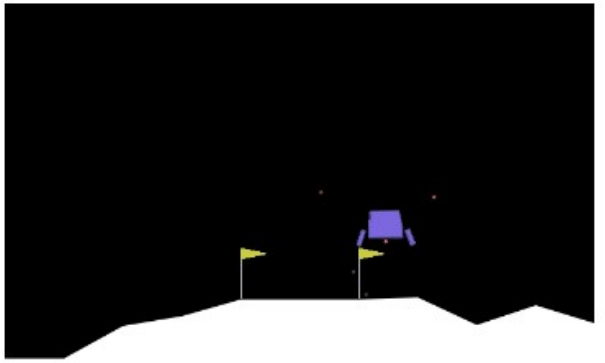
These environments all involve toy games based around physics control, using box2d based physics and PyGame-based rendering.



Bipedal Walker

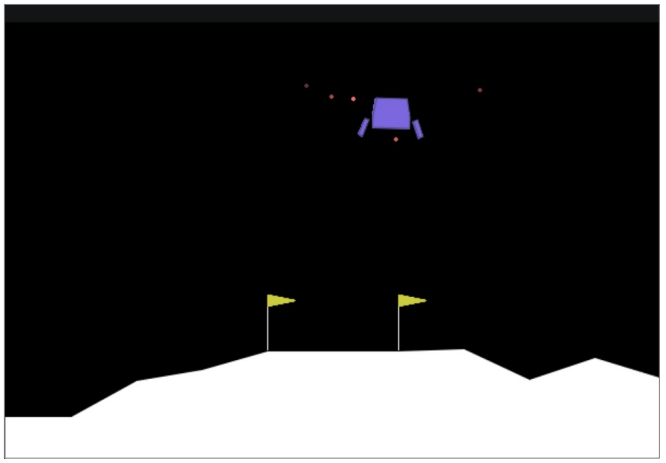


Car Racing



Lunar Lander

Box2D



Lunar Lander

This environment is a classic rocket trajectory optimization problem

action

- 0: do nothing
- 1: fire left orientation engine
- 2: fire main engine
- 3: fire right orientation engine

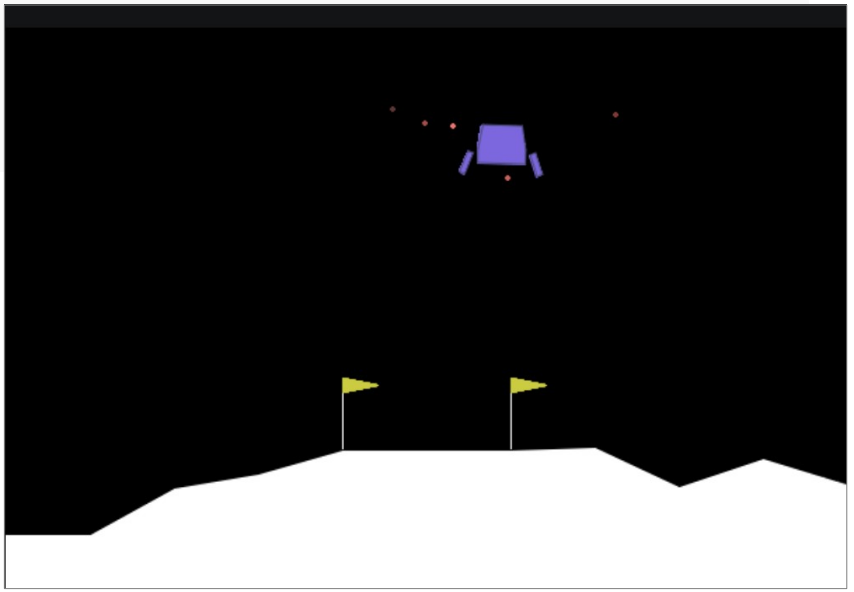
Action Space	`Discrete(4)`
Observation Space	`Box([ -2.5 -2.5 -10. -10. -6.2831855 -10. -0. -0. ], [ 2.5 2.5 10. 10. 6.2831855 10. 1. 1. ], (8,), float32)`
import	`gymnasium.make("LunarLander-v3")`

Observation state

8-dimensional vector: x, y, vx, vy, theta, theta-dot, two booleans (each leg is in contact with the ground or not)

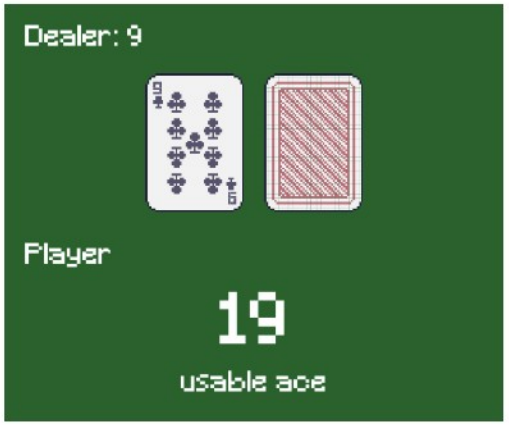
Rewards, Start state, End State?

```python
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```



Toy Text

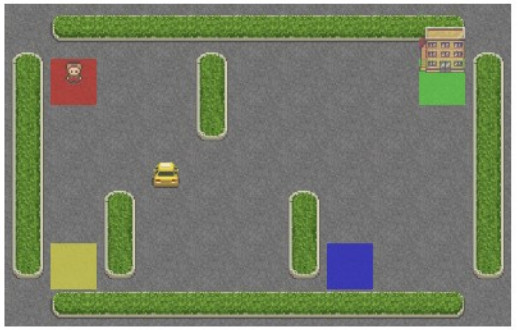
- Toy text environments are designed to be extremely simple, with small discrete state and action spaces, and hence easy to learn.
- suitable for debugging implementations of RL algorithms.



Blackjack

Cliff Walking

Frozen Lake



Taxi

Toy Text

The Grid World:

Frozen-Lake

action

- 0: Move left
- 1: Move down
- 2: Move right
- 3: Move up

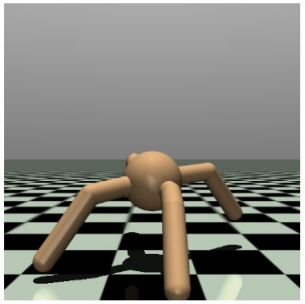
Action Space	`Discrete(4)`
Observation Space	`Discrete(16)`
import	`gymnasium.make("FrozenLake-v1")`

Observation state

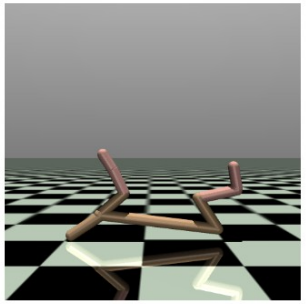
player's current position as current_row * ncols + current_col

Rewards, Start state, End State?

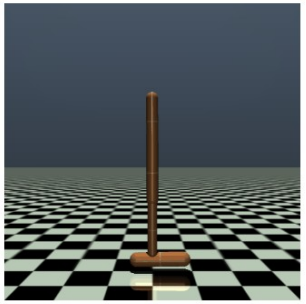
MuJoCo



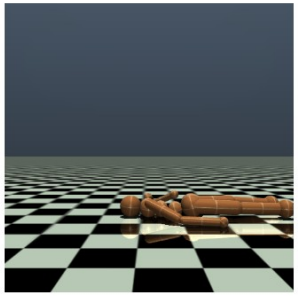
Ant



Half Cheetah

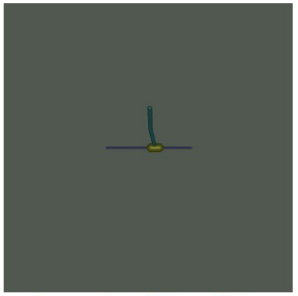


Hopper

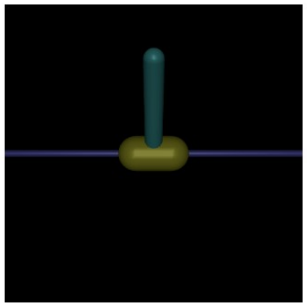


Humanoid Standup

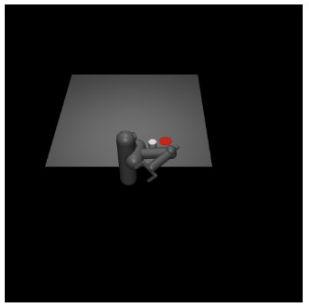
Humanoid



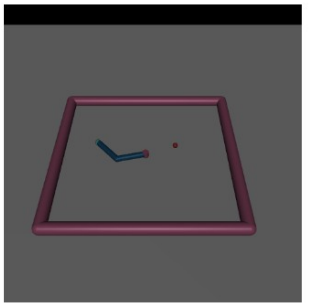
Inverted Double Pendulum



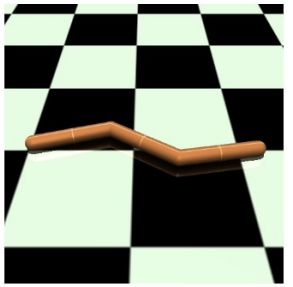
Inverted Pendulum



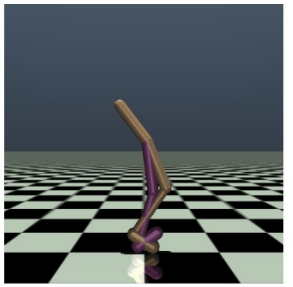
Pusher



Reacher



Swimmer



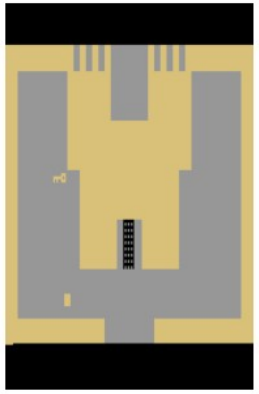
Walker2D

MuJoCo stands for Multi-Joint dynamics with Contact. It is a physics engine for facilitating research and development in robotics, biomechanics, graphics and animation, and other areas where fast and accurate simulation is needed.

ALE: Atari Games

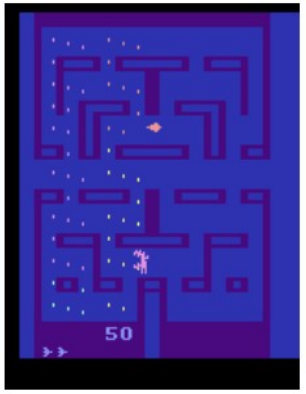
Atari Game ROMs.

- State in the form of pixels
- Locations and velocities found by images



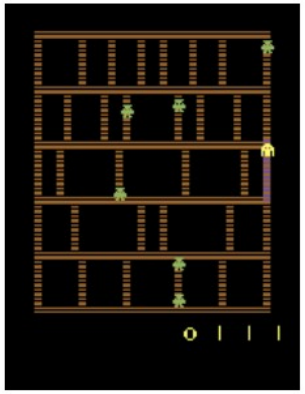
Adventure

Air Raid



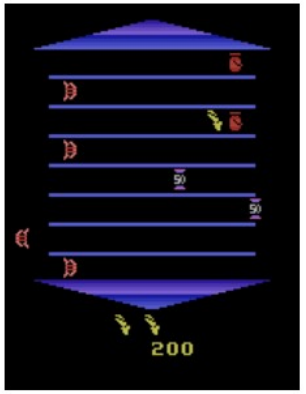
Alien

Atlantis

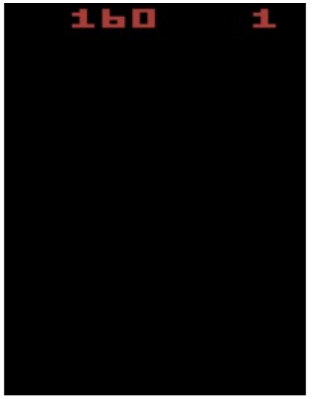


Amidar

Assault



Asterix



Asteroids

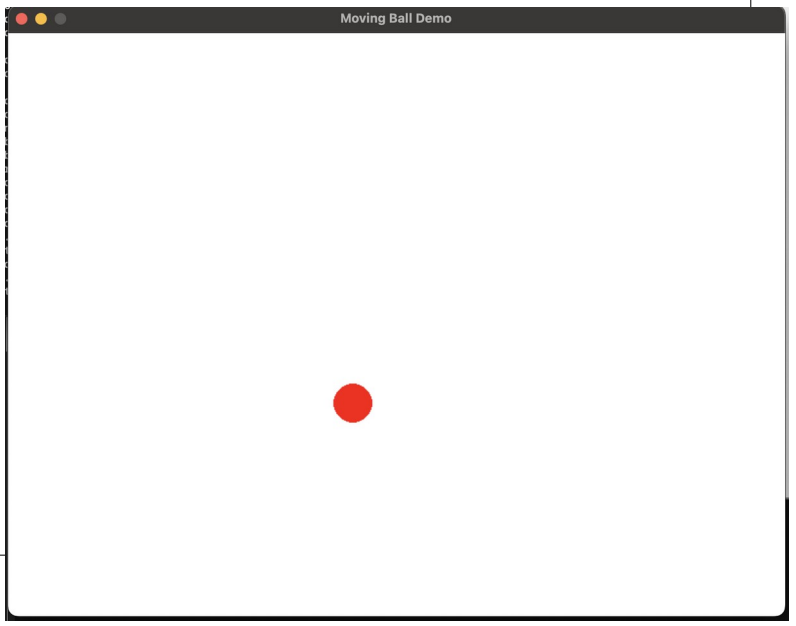
Custom Environments

Easiest way is to construct a custom class with:

- Drawing agent and environment
- Use a pygame environment
- Provide interface similar to gymnasium

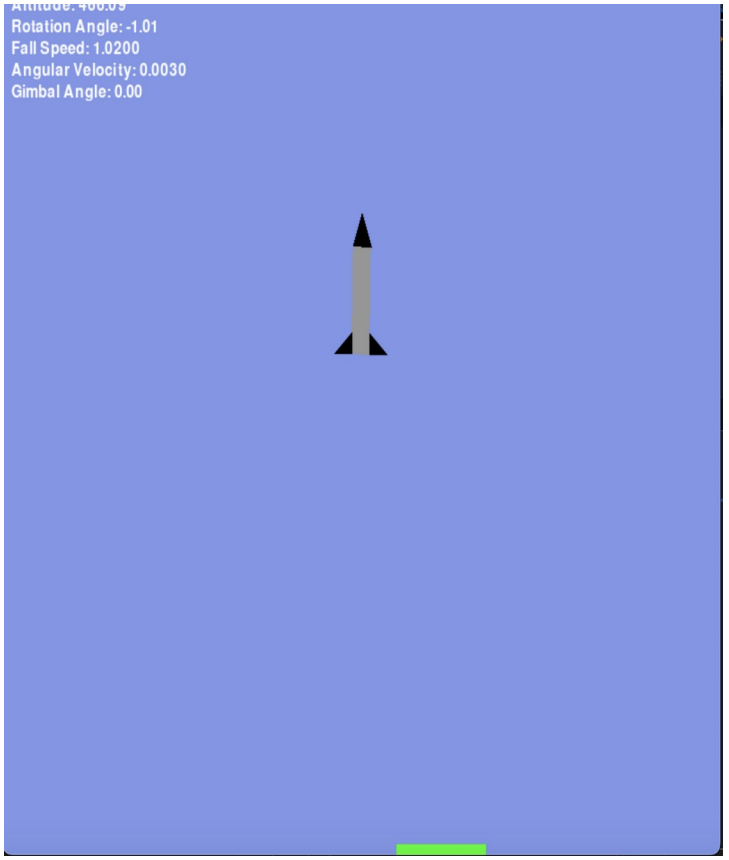
```
import pygame
import sys
# Initialize Pygame
pygame.init()
# Screen dimensions and colors
WIDTH, HEIGHT = 800, 600
WHITE = (255, 255, 255)
RED = (255, 0, 0)
# Ball class
class Ball:
    def __init__(self, x, y, radius, x_speed, y_speed):
        self.x = x
        self.y = y
        self.radius = radius
        self.x_speed = x_speed
        self.y_speed = y_speed
    def move(self):
        # Move the ball
        self.x += self.x_speed
        self.y += self.y_speed
        # Bounce off the walls
        if self.x - self.radius < 0 or self.x + self.radius > WIDTH:
            self.x_speed = -self.x_speed
        if self.y - self.radius < 0 or self.y + self.radius > HEIGHT:
            self.y_speed = -self.y_speed
    def draw(self, screen):
        pygame.draw.circle(screen, RED, (int(self.x), int(self.y)), self.radius)
```

```
# BallDemo class
class BallDemo:
    def __init__(self):
        # Set up display
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Moving Ball Demo")
        self.clock = pygame.time.Clock()
        # Create a Ball instance
        self.ball = Ball(x=WIDTH // 2, y=HEIGHT // 2, radius=20, x_speed=5, y_speed=3)
    def run(self):
        # Game loop
        while True:
            # Handle events
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()
            # Update ball position
            self.ball.move()
            # Draw everything
            self.screen.fill(WHITE)
            self.ball.draw(self.screen)
            pygame.display.flip()
            # Cap the frame rate
            self.clock.tick(60)
# Run the game
if __name__ == "__main__":
    game = BallDemo()
    game.run()
```



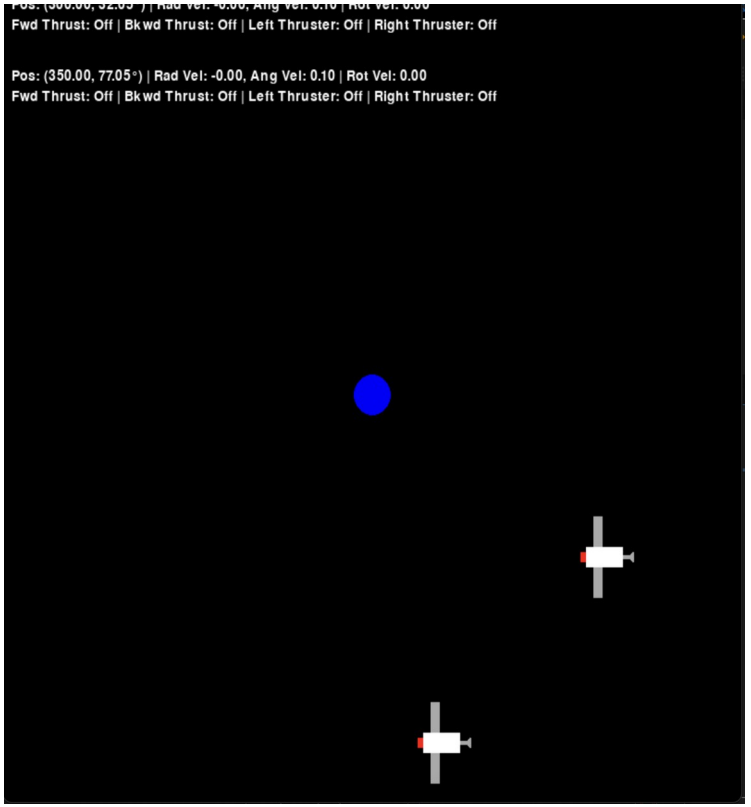
Custom Environments

More sophisticated and have the “interface” similar to Gym



Rocket lander:

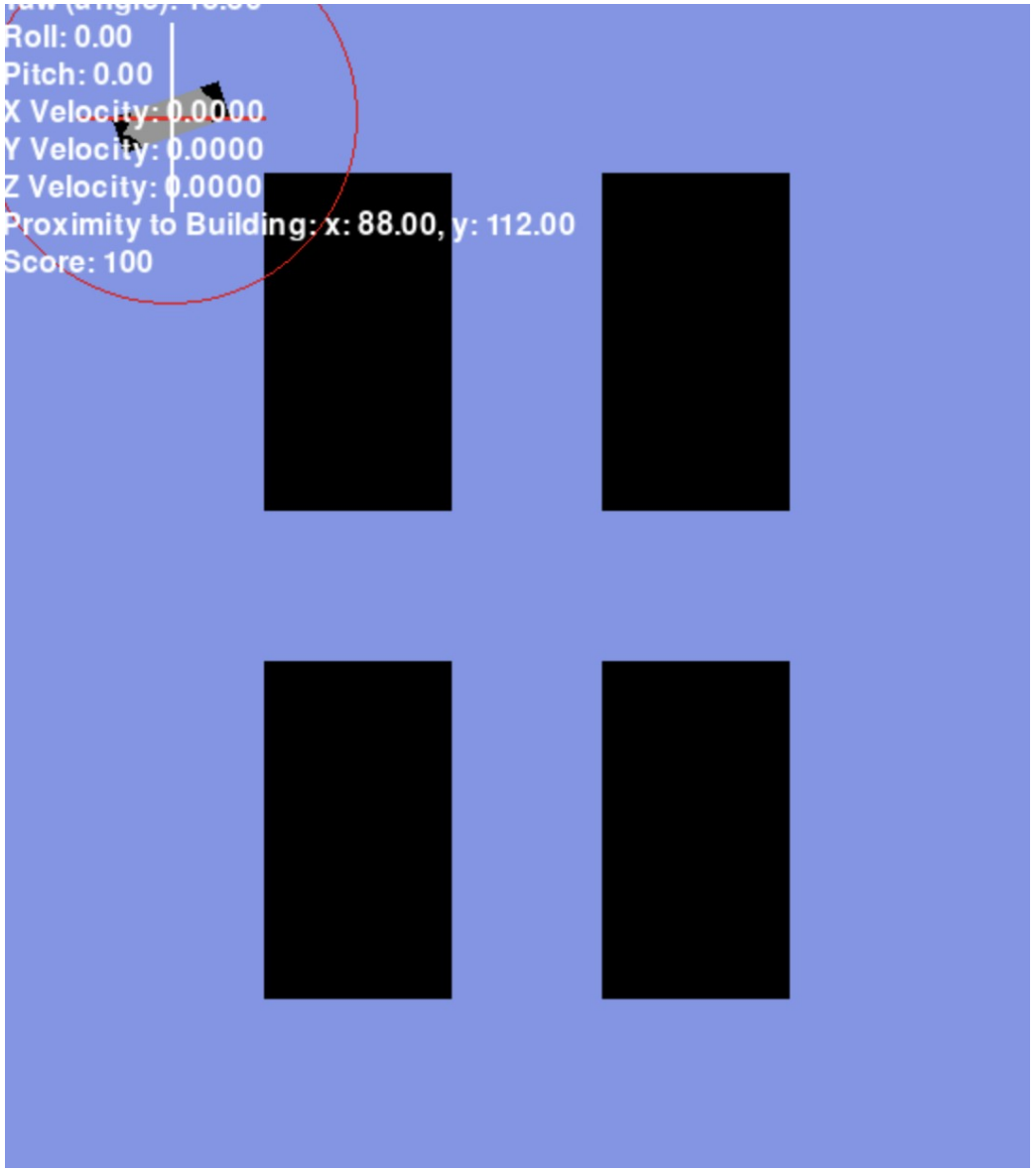
- Using gimble and side thrusters
- Must land on green pad



Multi-agent Satellite Docking

- Each agent obeys law of gravity; revolving around planet (orbital dynamics)
- Each agent has 4 thrusters
- They must dock on red end.

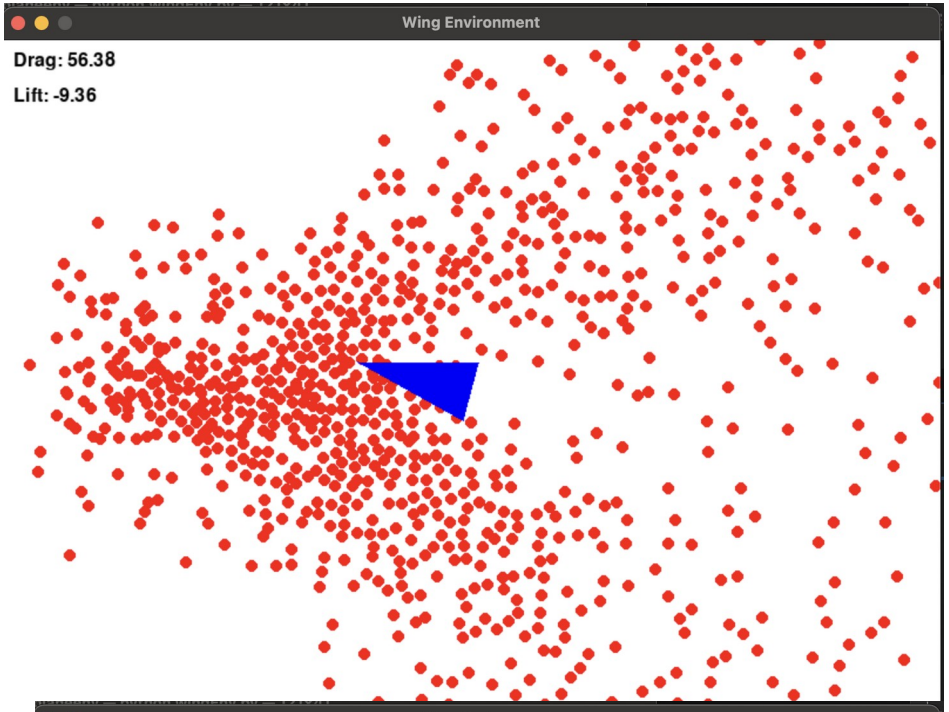
Custom Environments

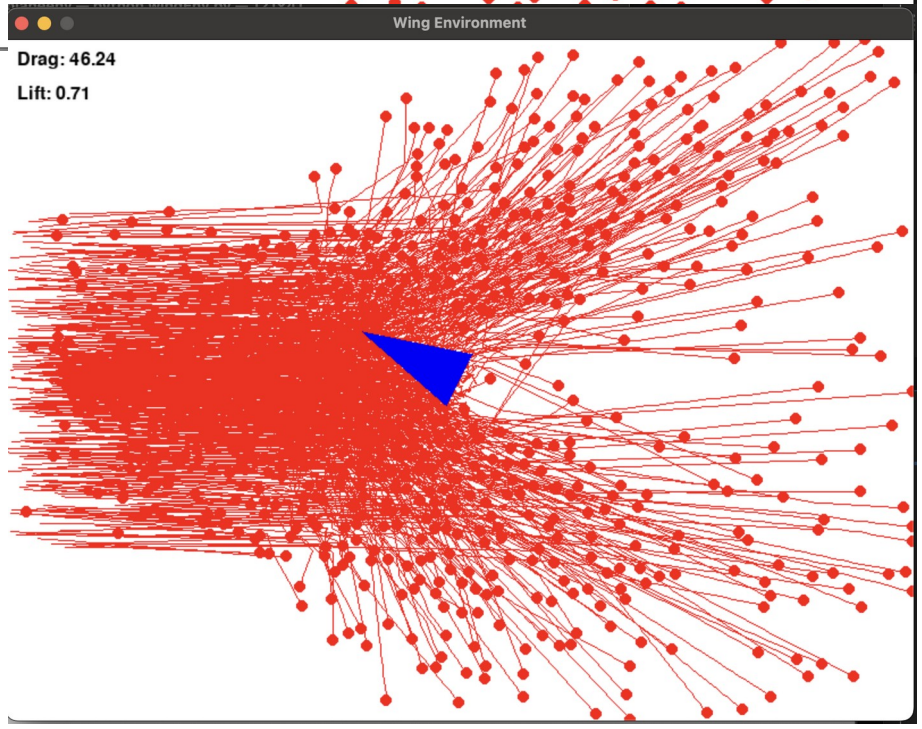


Drone navegation:

- Drone has lift thrust, tilt angle, and side control
- Must navegate through buildings without colliding
- Proximity sensors

Custom Wing design



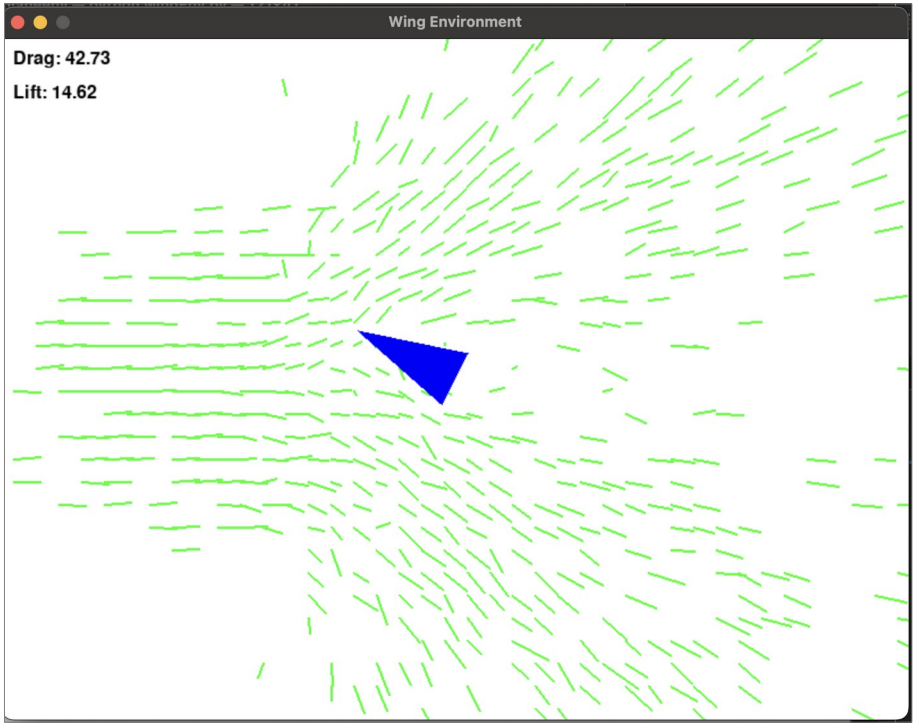


Environment that has:

- Particle based wind dynamics
- A simple parameterized polygon

Objective:

- show how RL can perform design tasks



The Overall Parts

1. The Environment

2. The Rewards Graph Structure

3. Learning and Policy algorithm

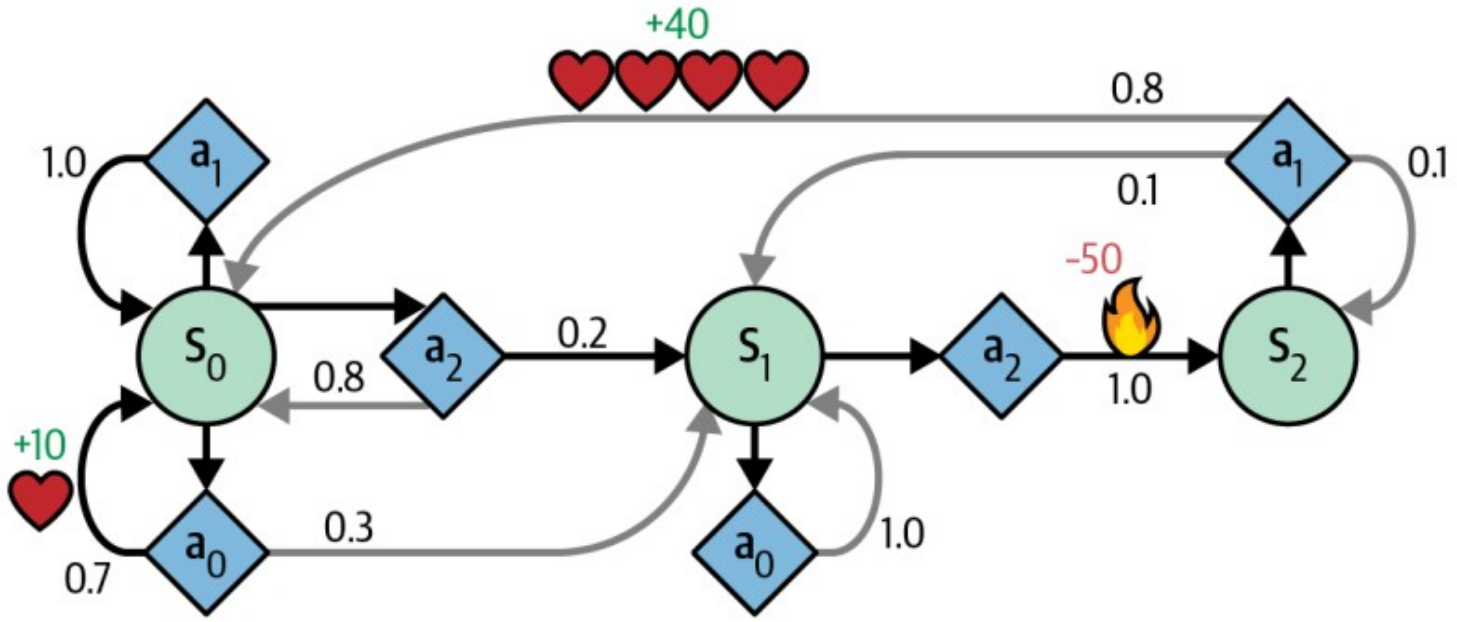
2. The Rewards Graph Structure

Description:

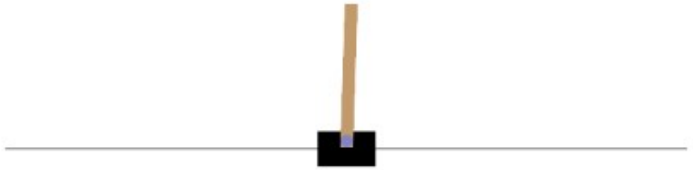
- **Reward Signals:** Immediate feedback received after taking an action.
- **Reward Structure**: Design of the reward function to align with overall objectives.
- **Reward Shaping:** Techniques to guide the agent towards desired behaviors effectively.

Purpose:

- Guides the agent's learning by providing feedback on actions.
- Aligns the agent's objectives with the desired outcomes in aerospace applications.



Cart-pole problem



Actions:

0: Push cart to the left

1: Push cart to the right

Observation state

Num	Observation	Min	Max
0	Cart Position	-4.8	4.8
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~ -0.418 rad (-24°)	~ 0.418 rad (24°)
3	Pole Angular Velocity	-Inf	Inf

Rewards:

Default Reward (standard setting):

- +1 reward for every step (including termination step).
- Reward threshold: 500 (v1) and 200 (v0), based on environment time limits.

Sutton-Barto Reward (if sutton_barto_reward=True):

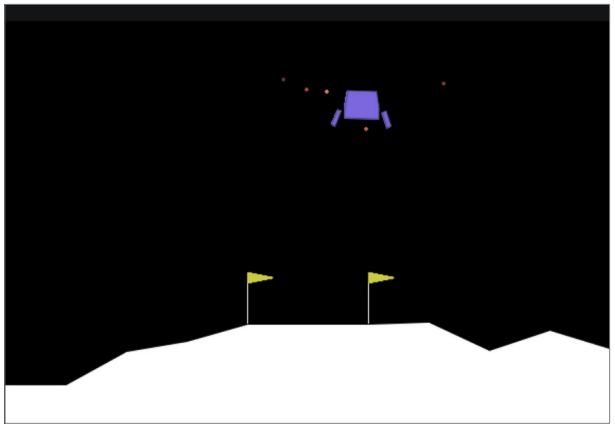
- 0 reward for every non-terminating step.
- -1 reward for the terminating step.
- Reward threshold: 0 for both v0 and v1.

End Episode:

The episode ends if any one of the following occurs:

- Termination: Pole Angle is greater than ±12°
- Termination: Cart Position is greater than ±2.4 (center of the cart reaches the edge of the display)
- Truncation: Episode length is greater than 500 (200 for v0)

Lunar Lander



action

0: do nothing

1: fire left orientation engine

2: fire main engine

3: fire right orientation engine

Observation state

8-dimensional vector: x, y, vx, vy, theta, theta-dot, two booleans (each leg is in contact with the ground or not)

Rewards:

After every step a reward is granted. The total reward of an episode is the sum of the rewards for all the steps within that episode.

For each step, the reward:

- is increased/decreased the closer/further the lander is to the landing pad.
- is increased/decreased the slower/faster the lander is moving.
- is decreased the more the lander is tilted (angle not horizontal).
- is increased by 10 points for each leg that is in contact with the ground.
- is decreased by 0.03 points each frame a side engine is firing.
- is decreased by 0.3 points each frame the main engine is firing.

The episode receive an additional reward of -100 or +100 points for crashing or landing safely respectively.

An episode is considered a solution if it scores at least 200 points.

End (episode termination)

The episode finishes if:

- the lander crashes (the lander body gets in contact with the moon);
- the lander gets outside of the viewport (x coordinate is greater than 1);
- the lander is not awake; a body which is not awake is a body which doesn't move and doesn't collide with any other body:

The Overall Parts

1. The Environment

2. The Rewards Graph Structure

3. Learning and Policy algorithm

3. Learning and Policy algorithm

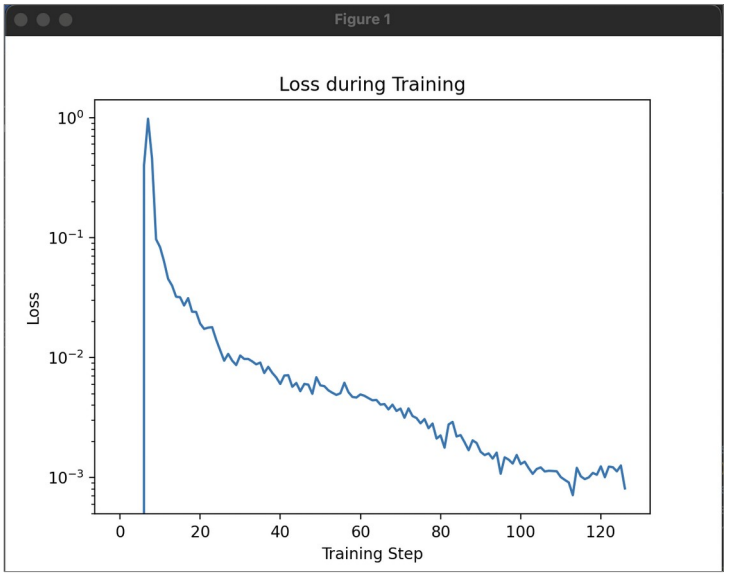
Description: This block encompasses several intertwined components that collectively enable the agent to learn and make decisions:

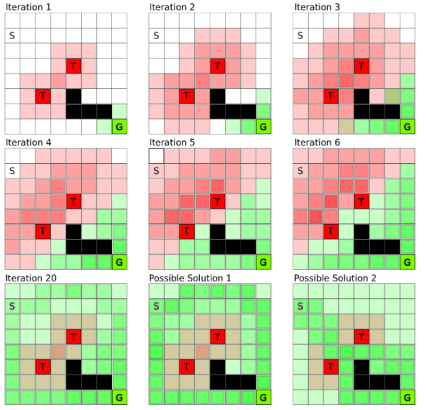
- **Training Algorithms:**
  - Types: Value-based (e.g., DQN), Policy-based (e.g., PPO), Actor-Critic (e.g., A2C, SAC).
  - Mechanisms: Experience replay, target networks, loss functions, and optimization techniques.

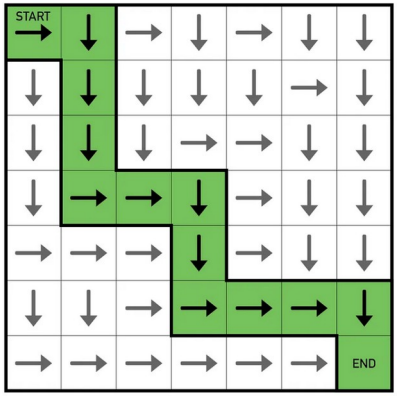
- **Policy and Value Functions:**
  - Policy (π(a ȿ)): Defines the agent's behavior by mapping states to actions.
  - Value Functions (V(s) or Q(s,a)): Estimates the expected return from a state or state-action pair.

- **Exploration Strategies:**
  - Techniques: ε-greedy, Boltzmann exploration, entropy regularization.
  - Purpose: Balances exploration of new actions and exploitation of known rewarding actions.

- **Neural Network Integration:**
  - Role: Approximates policies or value functions, enabling the handling of high-dimensional and continuous spaces.
  - Architecture Considerations: Choice of layers, activation functions, and network structure tailored to aerospace tasks.







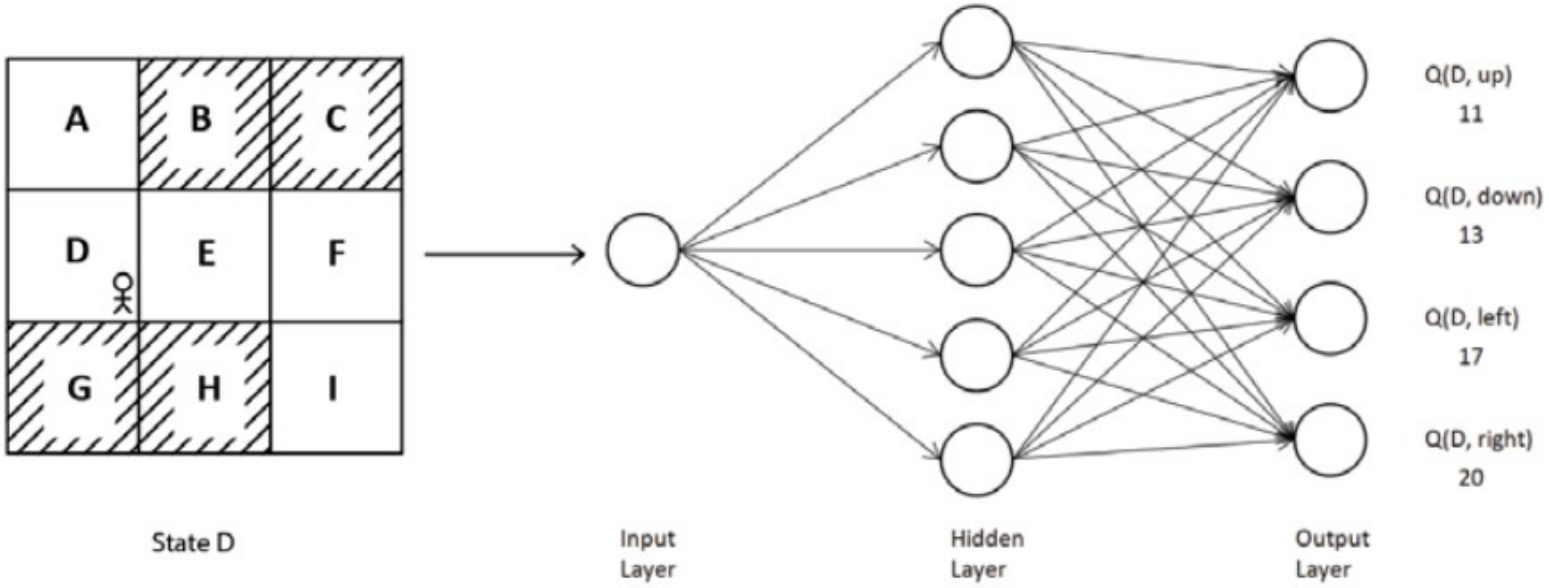
Example: value based

$$V^*(s) = \max_a \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma V^*(s')]$$

The Learning Problem

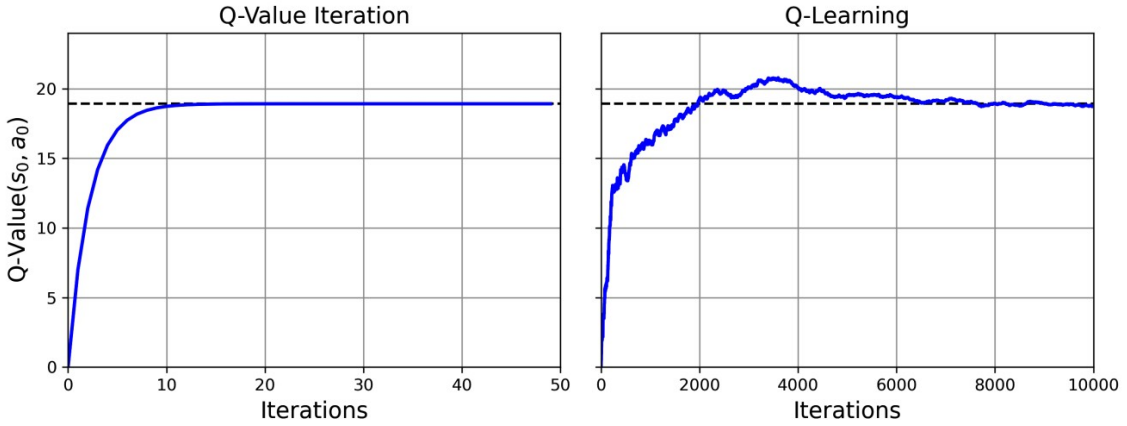
We will see how this is the way that we incorporate the DNN.
As a universal approximator.

$$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$



Aproximate Q-learning

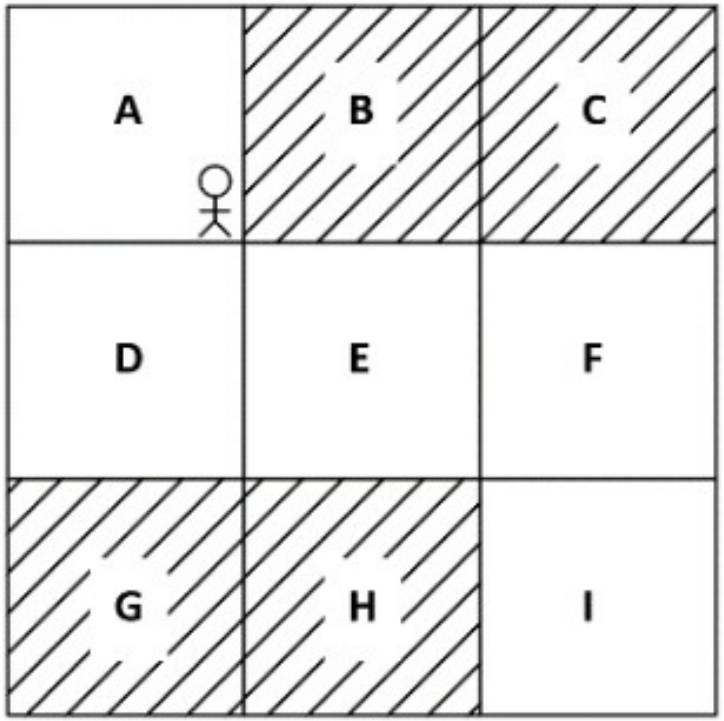
- Use the DNN to approximate a large Q table.



3

Some definitions and some theory

Heavy math the next time….
now just some intuition & definitions

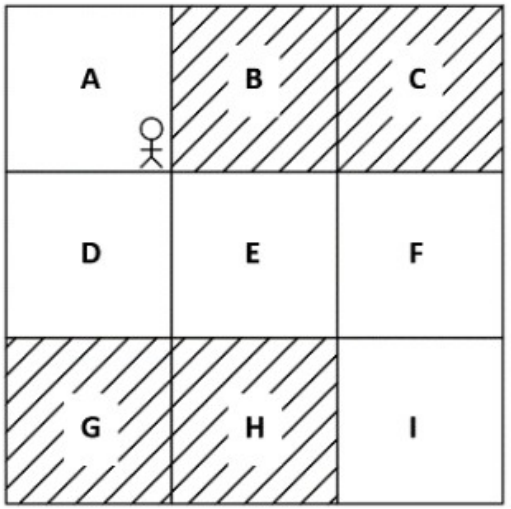


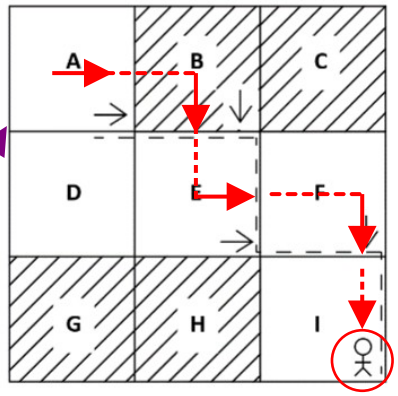
Just let's think about some definitions;

then we will see why
We need Neural networks.

Consider the simple "discrete" problem of The Grid World

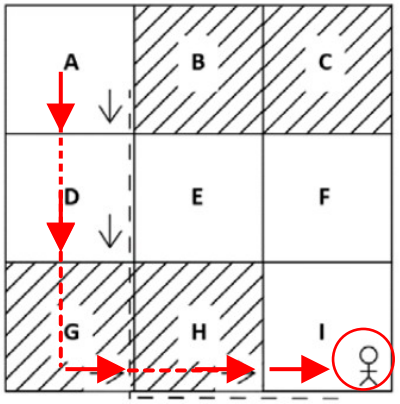
Episodes
in Grid world





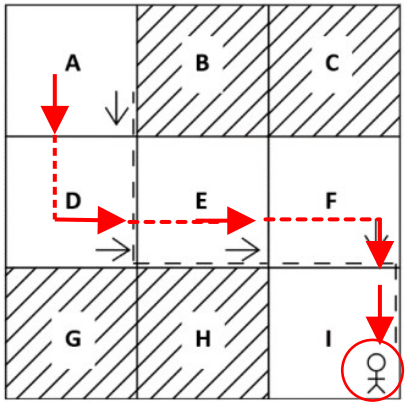
episode #1

$\tau_1 =$ A —Right, -1→ B —Right, -1→ C —Down, +1→ F —Down, +1→ I



episode #2

$\tau_2 =$ A —Down, +1→ D —Down, -1→ G —Right, -1→ H —Right, +1→ I

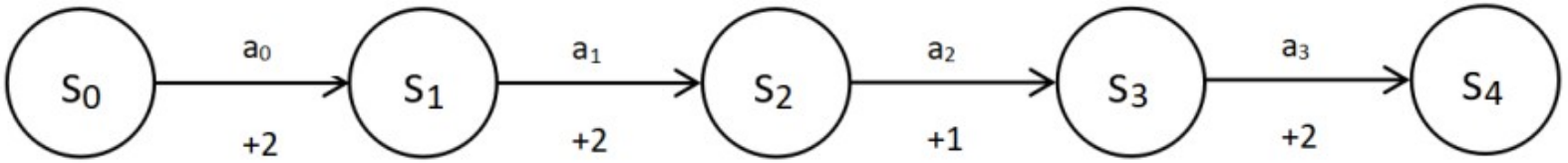


episode #n

$\tau_n =$ A —Down, +1→ D —Right, +1→ E —Right, +1→ F —Down, +1→ I

Return & discount factor

A particular “Episode”:



$$R(\tau) = r_0 + r_1 + r_2 + \ldots + r_T$$

- Return R(τ): Cumulative reward over time
- Goal: Maximize total reward
  - Discount factor (γ):
  - Value: $0 \leq \gamma \leq 1$
- Represents preference for immediate rewards
  - Balances short-term and long-term rewards

$$R(\tau) = \sum_{t=0}^{T} r_t$$

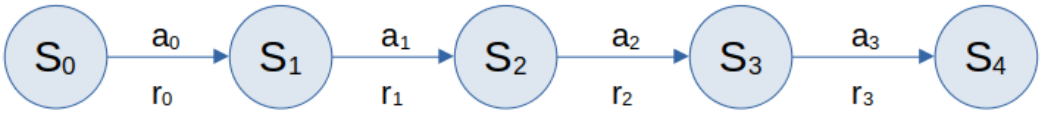
introduce discount factor

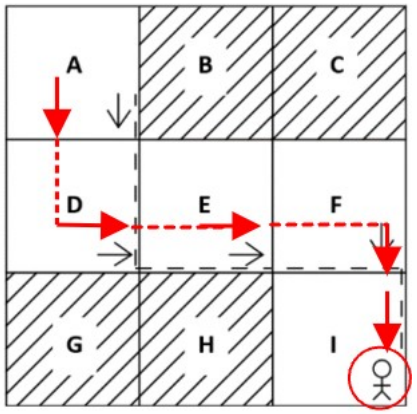
$$R(\tau) = \gamma^0 r_0 + \gamma^1 r_1 + \gamma^2 r_2 + \ldots + \gamma^n r_\infty$$

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$$

- Discount factor (γ) importance:
- Prevents return from reaching infinity
- Balances future and immediate rewards

Value equation/episodes:
understand discount value





How to find an optimal Policy??

Value function (state value function)
- Denotes value of a state
- Represents return when following policy π
- Typically denoted as V(s)

$$V^{\pi}(s) = [R(\tau)|s_0 = s]$$

policy

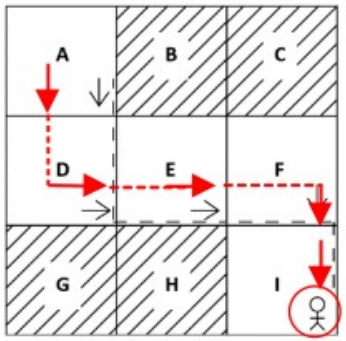
Return for particular trajectory τ

$s_0 = s$ starting state

We are looking for this:

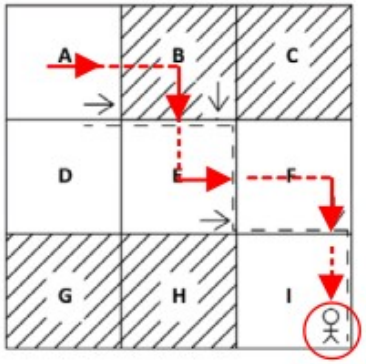
$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

Expected Value function



$\tau_1 =$ A —Down, +1→ D —Right, +1→ E —Right, +1→ F —Down, +1→ I

$$V(A) = R(\tau_1) = 1 + 1 + 1 + 1 = 4$$



$\tau_2 =$ A —Right, -1→ B —Down, +1→ E —Right, +1→ F —Down, +1→ I

$$V(A) = R(\tau_2) = -1 + 1 + 1 + 1 = 2$$

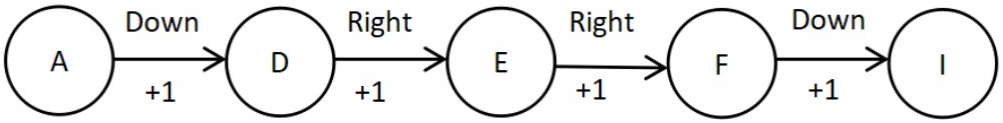
$$V^{\pi}(A) = \mathop{\mathbb{E}}_{\tau \sim \pi}[R(\tau)|s_0 = A] = \sum_i R(\tau_i)\pi(a_i|A)$$

$$= R(\tau_1)\pi(\text{down}|A) + R(\tau_2)\pi(\text{right}|A)$$
$$= 4(0.8) + 2(0.2)$$
$$= 3.6$$

State-action: Q - function

$$Q^{\pi}(s,a) = [R(\tau)|s_0 = s, a_0 = a]$$

- Q function (state-action value function)
- Denotes value of state-action pair
- Represents return when starting at state s, performing action a, and following policy π
- Usually denoted as Q(s,a)
- Difference from value function:
  - Value function: value of state
  - Q function: value of state-action pair



$$Q^{\pi}(A, \text{down}) = [R(\tau)|s_0 = A, a_0 = \text{down}]$$

$$Q(A, \text{down}) = 1 + 1 + 1 + 1 = 4$$

Similarly, we can compute the Q value for all the state-action pairs

represents the expected future rewards for taking a particular action a in a given state s, and thereafter following a certain policy.

Simple model for Q(s,a)

- Definition of State and Action:
  - State (s): Each cell in the 4x4 grid can be considered a state.
  - Action (a): From any cell (state), the action can be to move to one of the neighboring cells (North, South, East, or West).
- Q-value Calculation:
  - The Q(s,a) value is calculated based on the reward received for taking action a from state s and the discounted maximum Q-value of the next state s' that the agent moves to as a result of action a.
  - Essentially, it evaluates the "quality" of each state-action pair.

Blocked	U: -1.00 D: 62.47 L: -1.00 R: -1.00	Blocked	U: 2.00 D: 58.63 L: 2.00 R: 2.00
U: 6.00 D: 63.17 L: 6.00 R: 69.47	U: 64.23 D: 67.83 L: 70.53 R: 66.93	U: 2.00 D: 2.00 L: 65.47 R: 58.63	U: 56.77 D: 59.47 L: 62.93 R: 4.00
U: 63.53 D: 58.76 L: 1.00 R: 60.83	U: 66.47 D: 3.00 L: 60.17 R: 3.00	Blocked	U: 61.63 D: 5.00 L: 5.00 R: 5.00
U: 64.17 D: 7.00 L: 7.00 R: 7.00	Blocked	U: 9.00 D: 9.00 L: 9.00 R: 9.00	Blocked

Initialization:
- Initialize Q-values arbitrarily for all state-action pairs.
- Commonly set to zero for all pairs at the start.

State and Action:
- State (s): Each position or scenario within the environment.
- Action (a): Possible decisions or movements from each state.

Reward and Transition:
- For each action taken in a state, a reward is received based on the environment's response.
- The agent then transitions to a new state based on the action.

Q-value Update (Bellman Equation):
- Update rule: Based upon a Markov chain
- We will derive this!!

We will show code soon….

Q - function

With random return; must sum over all action-state pair probabilities:

Expectation value of Return, R

$$Q^{\pi}(s,a) = \mathop{\mathbb{E}}_{\tau \sim \pi} [R(\tau) | s_0 = s, a_0 = a]$$

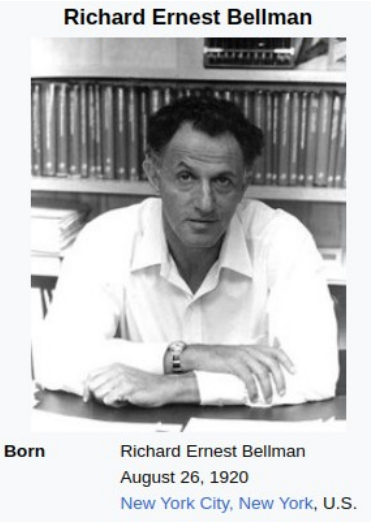
maximum Q value

$$Q^{*}(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

The optimal policy π^* is the policy that gives the maximum **Q** value.

Optimality and the Bellman equation

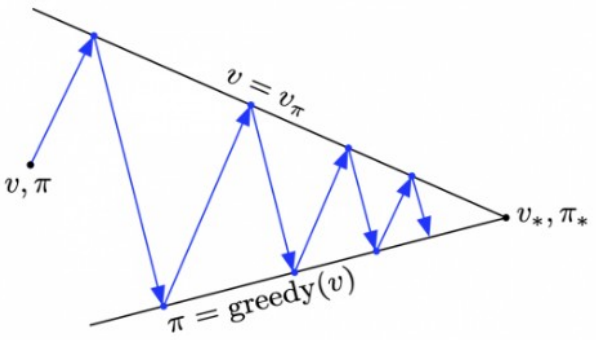
The **Bellman equation**, named after Richard Bellman, helps us solve the Markov decision process (MDP). When we say solve the MDP, we mean finding the optimal policy.



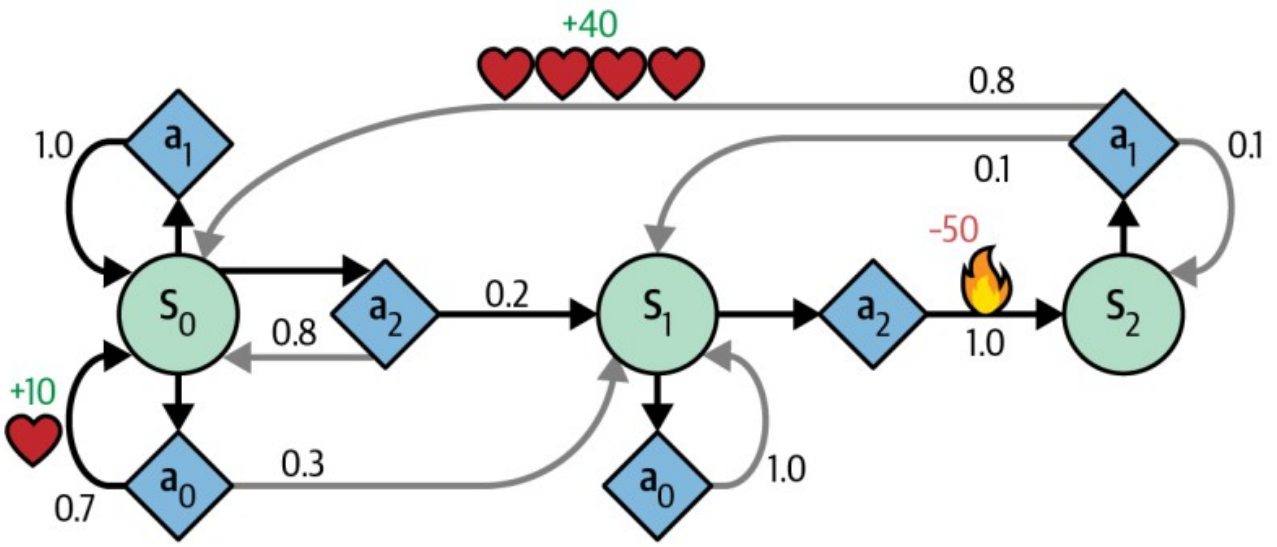
To solve the Markov Decision Process (MDP) by identifying the optimal policy.

Objective:

- Goal: Maximize the expected cumulative rewards over time, identifying strategies that offer the best long-term benefits.



Markov Decision Processes in RL



Optimal Policy Search

In reinforcement learning our goal is to find the optimal policy.

The optimal policy is the policy that selects the correct action in each state so that the agent can get the maximum return and achieve its goal.

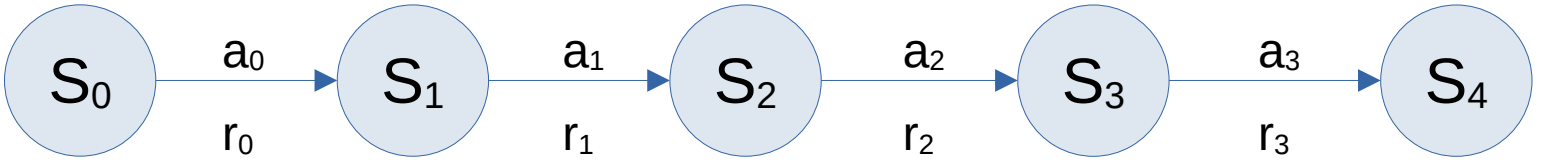
classic reinforcement learning algorithms called the value and policy iteration methods, which we can use to find the optimal policy.

Done with Bellaman equation:

(dynamic programming)

Bellman "backup" equation of a Deterministic Environment

Trajectory: τ



$$V(s) = R(s, a, s') + \gamma V(s')$$

Determines the "value" of a state
Depends on a reward between the States and the "discounted" value

$R(s, a, s')$ Immediate reward performing action a with s → s'

γ Discount factor: (the extent to which future rewards are considered when choosing an action; can Vary how agents value short- long-term rewards)

$V(s')$ Value of next state

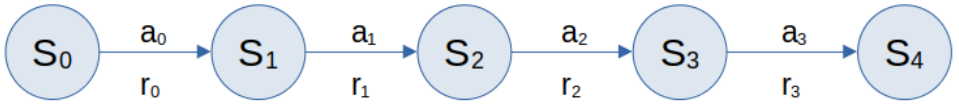
With respect to the policy π

$$V^{\pi}(s) = R(s, a, s') + \gamma V^{\pi}(s')$$

Called the Bellman backup

Bellman "backup" equation of a Stochasic environment

We take weighted average uncertainty:



a sum of the Bellman backup multiplied by the corresponding transition probability of the next state

$$V^{\pi}(s) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$$

$P(s'|s,a)$ transition probability

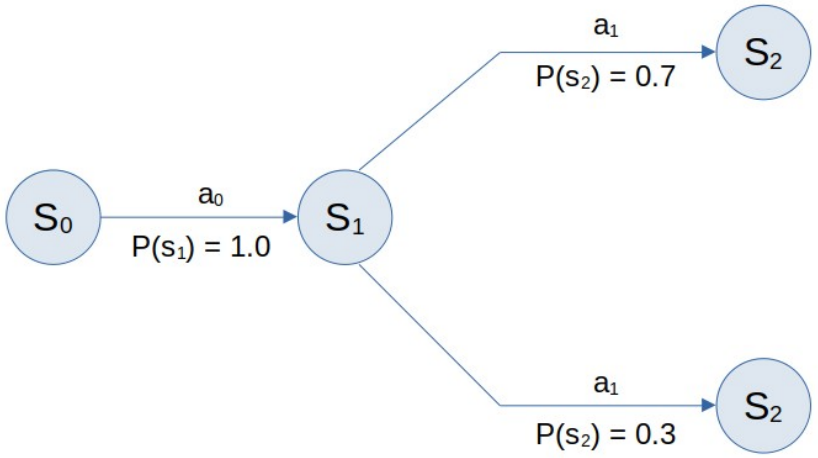
$[R(s,a,s') + \gamma V^{\pi}(s')]$ Bellman backup

Include stochasticity:

- Expectation (the weighted average),
- a sum of the Bellman backup multiplied by the corresponding transition probability of the next state.

$$\mathbb{E}_{x\sim p(x)}[f(X)] = \sum_{i=1}^{N} f(x_i)p(x_i)$$

Bellman "backup" equation of a Stochasic environment and policy



Policy is stochastic: We take weighted average uncertainty:

- With stochastic policy: Bellman equation takes expectation
  - (the weighted average), that is, a sum of the Bellman backup multiplied by the corresponding probability of action.

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^{\pi}(s')]$$

$$V^{\pi}(s) = \mathop{\mathbb{E}}_{\substack{a\sim\pi \\ s'\sim P}} [R(s,a,s') + \gamma V^{\pi}(s')]$$

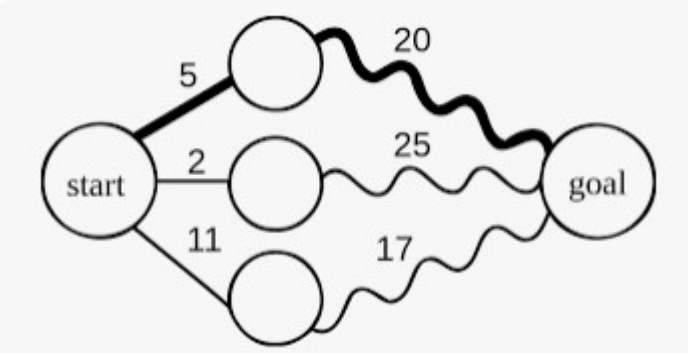
Recall: expectation value

$$\mathbb{E}_{x\sim p(x)}[f(X)] = \sum_{x} p(x) f(x)$$

Optimality: Bellman equation

$$V^*(s) = \max_a \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma V^*(s')]$$

- Optimality condition for value equation
  - Value function depends on policy
  - State value varies based on chosen policy
  - Multiple value functions for different policies
- Optimal value function $V^*(s)$
  - Yields maximum value compared to other functions
  - Optimal Bellman value function has maximum value



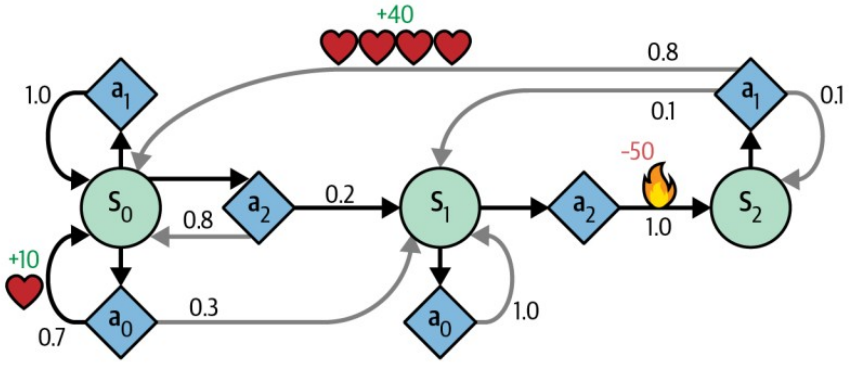
- Computing optimal Bellman value function
  - Select action with maximum value
  - Calculate state value using all possible actions
  - Choose maximum value as the state value

Bellman Optimality Equation

$$V^*(s) = \max_a \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma \cdot V^*(s')] \quad \text{for all } s$$

$T(s, a, s')$	is the **transition** probability from state s to state s′, given that the agent chose action a.
$R(s, a, s')$	is the **reward** that the agent gets when it goes from state s to state s′, given that the agent chose action a.
γ	is the discount factor



Bellman Optimality
Value iteration

example of ***dynamic programming***, which breaks down a complex problem into tractable subproblems that can be tackled iteratively.

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma \cdot V_k(s')\right]$$

$$\text{for all } s$$

In this equation, $V_k(s)$ is the estimated value of state s at the k-th iteration of the algorithm.

```
1  def value_iteration():
2      initialize(V)
3      while not convergence(V):
4          for s in range(S):
5              for a in range(A):
6                  Q[s,a] = ∑_{s'∈S} T_a(s,s')(R_a(s,s') + γV[s'])
7              V[s] = max_a(Q[s,a])
8      return V
```

Taxi world Example

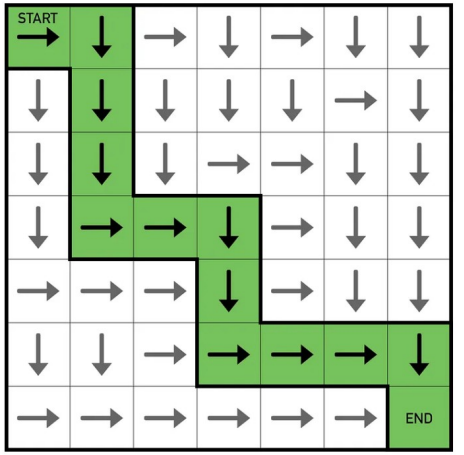
```python
def iterate_value_function(v_inp, gamma, env):
    ret = np.zeros(env.observation_space.n)
    for sid in range(env.observation_space.n):
        temp_v = np.zeros(env.action_space.n)
        for action in range(env.action_space.n):
            for (prob, dst_state, reward, is_final) in env.P[sid][action]:
                temp_v[action] += prob * (reward + gamma * v_inp[dst_state] * (not is_final))
        ret[sid] = max(temp_v)
    return ret

def build_greedy_policy(v_inp, gamma, env):
    new_policy = np.zeros(env.observation_space.n)
    for state_id in range(env.observation_space.n):
        profits = np.zeros(env.action_space.n)
        for action in range(env.action_space.n):
            for (prob, dst_state, reward, is_final) in env.P[state_id][action]:
                profits[action] += prob * (reward + gamma * v_inp[dst_state])
        new_policy[state_id] = np.argmax(profits)
    return new_policy
```

```python
env = gym.make('Taxi-v3')
gamma = 0.9
cum_reward = 0
n_rounds = 500
env.reset()

for t_rounds in range(n_rounds):
    observation = env.reset()
    v = np.zeros(env.observation_space
    # solve MDP
    for _ in range(100):
        v_old = v.copy()
        v = iterate_value_function(v, gamma, env)
        if np.all(v == v_old):
            break
    policy = build_greedy_policy(v, gamma, env).astype(np.int)
    # apply policy
    for t in range(1000):
        #print("Observation:", observation, "Type:", type(observation))
        if isinstance(observation, tuple):
            observation = observation[0]
        action = policy[observation]
        step_result = env.step(action)
        if len(step_result) == 4:
            observation, reward, done, info = step_result
        elif len(step_result) == 5:
            observation, reward, done, _, info = step_result
        else:
            print("Unexpected step_result:", step_result)
            break
        cum_reward += reward
        if done:
            break

    if t_rounds % 50 == 0 and t_rounds > 0:
        print(cum_reward * 1.0 / (t_rounds + 1))

env.close()
```

More complicated model

```
class GridWorldQLearner:
    def __init__(self, size=4, gamma=0.9, alpha=0.5, initial_epsilon=0.9, iterations=100):
        self.size = size
        self.gamma = gamma  # Discount factor
        self.alpha = alpha  # Learning rate
        self.epsilon = initial_epsilon  # Exploration rate
        self.iterations = iterations

        np.random.seed(0)
        self.rewards = np.random.randint(-1, 10, size=(size, size))
        blocked_indices = np.random.choice(range(size*size), size=5, replace=False)
        self.rewards.flat[blocked_indices] = -10  # Mark blocked cells

        self.Q_values = np.random.rand(size, size, 4) * 0.01
        self.actions = {'up': (-1, 0), 'right': (0, 1), 'down': (1, 0), 'left': (0, -1)}
        self.action_list = ['up', 'right', 'down', 'left']
        self.last_action = None
```

```
Updated Q-values for state (1,1) and action down: 66.33535966666902
Updated Q-values for state (1,1) and action left: 69.03350833044044
Updated Q-values for state (1,2) and action right: 57.160872497846164
Updated Q-values for state (1,2) and action left: 63.99721475443535
Updated Q-values for state (1,3) and action up: 55.31445666297325
Updated Q-values for state (1,3) and action down: 58.01623947640244
Updated Q-values for state (1,3) and action left: 61.466822574000986
Updated Q-values for state (2,0) and action up: 62.033508330440455
Updated Q-values for state (2,0) and action right: 59.33535966666901
Updated Q-values for state (2,0) and action down: 57.26535966666908
Updated Q-values for state (2,1) and action up: 64.99721475443535
Updated Q-values for state (2,1) and action left: 58.697214754435365
Updated Q-values for state (2,3) and action up: 60.19177823692091
Updated Q-values for state (3,0) and action up: 62.697214754435365
```



Bellman of Q function

The Bellman equation of the Q function is very similar to the Bellman equation of the value function

1

$$Q(s,a) = R(s,a,s') + \gamma Q(s',a')$$

$R(s,a,s')$ Immediate reward doing action a from s to s'

γ Discount factor

$Q(s',a')$ Q value of next state-action pair

2

Bellman backup for Q function

$$Q^{\pi}(s,a) = R(s,a,s') + \gamma Q^{\pi}(s',a')$$

3

Expected value

$$Q^{\pi}(s,a) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma Q^{\pi}(s',a')]$$

Just rearrange terms

$$Q^{\pi}(s,a) = \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^{\pi}(s',a') \right]$$

$$Q^{\pi}(s,a) = \mathbb{E}_{s' \sim P}[R(s,a,s') + \gamma \mathbb{E}_{a' \sim \pi} Q^{\pi}(s',a')]$$

Bellman Optimal

Policy: state-action Q-values

1

$$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

2 Q-value iteration algorithm

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma \cdot \max_{a'} Q_k(s',a')\right]$$

$$\text{for all } (s,a)$$

3 the optimal policy $\pi^*(s)$

$$\pi^*(s) = \operatorname*{argmax}_a Q^*(s,a)$$

4

Solving with DNN:
the essential Q-learning algorithm

Jumpstart- A deep dive into DQN

Q-Learning

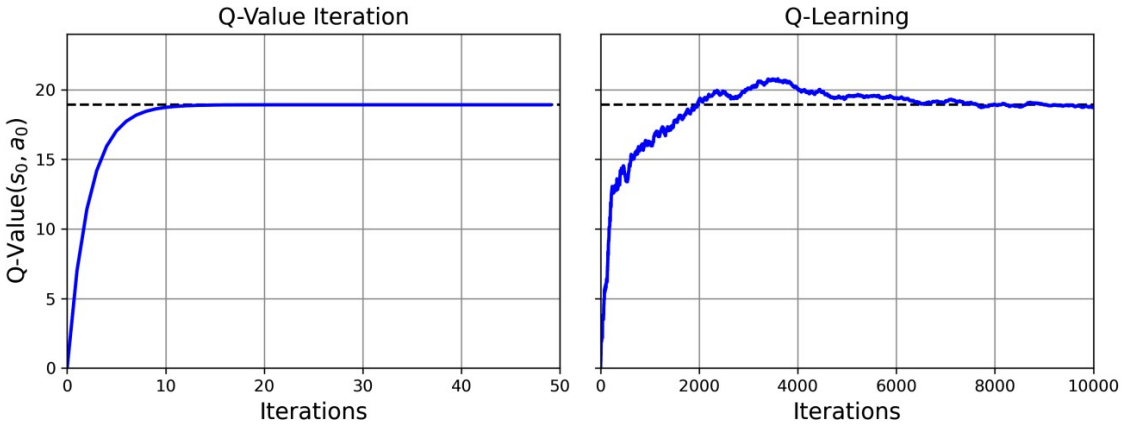
Q-learning algorithm is the Q-value iteration when transition probabilities and the rewards are initially unknown

Q-learning: watch an agent play; improve estimates of Q-values.

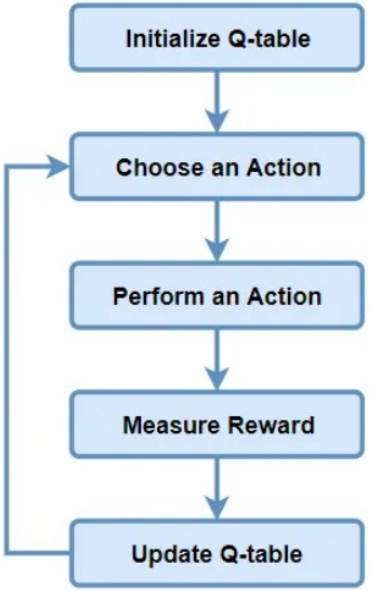
With accurate Q-value estimates; the optimal policy is action with highest Q-value (i.e., the greedy policy).

- Algorithm tracks state-action pairs (s, a)
- Maintains running average of rewards (r)
- Considers rewards upon leaving state s with action a
- Includes sum of discounted future rewards
- Estimates sum using maximum Q-value for next state s′
- Assumes target policy acts optimally from then on

$$Q(s,a) \underset{\alpha}{\leftarrow} r + \gamma \cdot \max_{a'} Q(s',a')$$

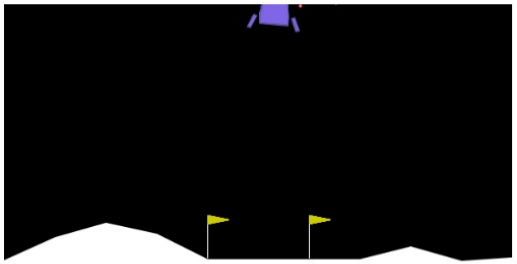


A number of Iterations
-result a good Q-table



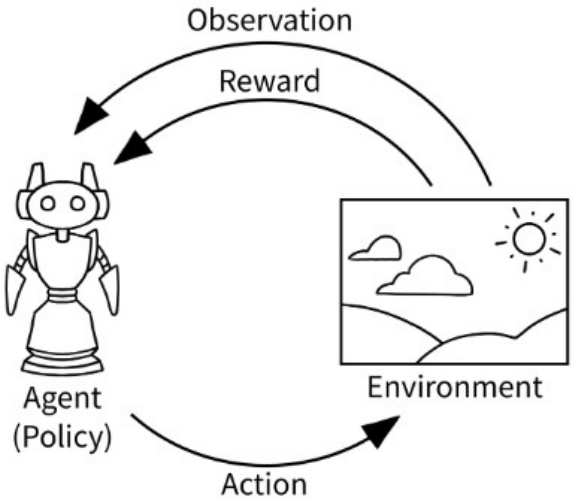
Gymnasium

An API standard for reinforcement learning with a diverse collection of reference environments



Interacting with the Environment

The classic “agent-environment loop” pictured below is simplified representation of reinforcement learning that Gymnasium implements.



This loop is implemented using the following gymnasium code

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset()

for _ in range(1000):
    action = env.action_space.sample()  # agent policy that uses the observation and info
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

https://gymnasium.farama.org/content/basic_usage/

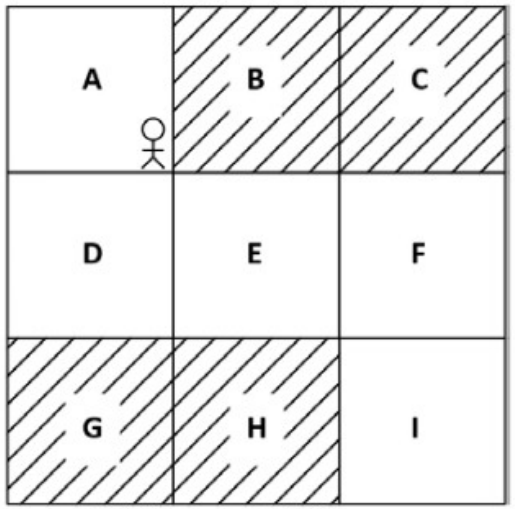
Taxi world
Q-learning Example


```python
# Q learning for OpenAI Gym Taxi environment
import gym
import numpy as np
import random

#Environment Setup
env = gym.make("Taxi-v3")
env.reset()
# Q[state,action] table implementation
Q = np.zeros([env.observation_space.n, env.action_space.n])
# discount factor
gamma = 0.7
alpha = 0.2
# learning rate
epsilon = 0.1 # epsilon greedy
```

```python
for episode in range(1000):
    done = False
    total_reward = 0
    observation = env.reset()
    state = observation[0]  # Extract the integer state from the tuple
    while not done:
        if random.uniform(0, 1) < epsilon:
            action = env.action_space.sample() # Explore state space
        else:
            action = np.argmax(Q[state]) # Exploit learned values
        step_result = env.step(action) #invoke Gym
        next_state = step_result[0]
        reward = step_result[1]
        done = step_result[2]
        info = step_result[3:]

        next_max = np.max(Q[next_state])
        old_value = Q[state,action]
        new_value = old_value + alpha * (reward + gamma *next_max - old_value)
        Q[state,action] = new_value
        total_reward += reward
        state = next_state

    if episode % 100 == 0:
        print("Episode {} Total Reward: {}".format(episode,total_reward))
```

```
Episode 0 Total Reward: -1533
Episode 100 Total Reward: -281
Episode 200 Total Reward: -9
Episode 300 Total Reward: -65
Episode 400 Total Reward: -32
Episode 500 Total Reward: -19
Episode 600 Total Reward: 12
Episode 700 Total Reward: 4
Episode 800 Total Reward: 10
Episode 900 Total Reward: 12
```

Size of space:



State	Action	Value
A	up	17
A	down	10
B	up	11
B	down	20

Given Q value of all state-action pairs, then we select the action in each state that has the maximum Q value.

- So, we select the action up in state A and down in state B as they have the maximum Q value.

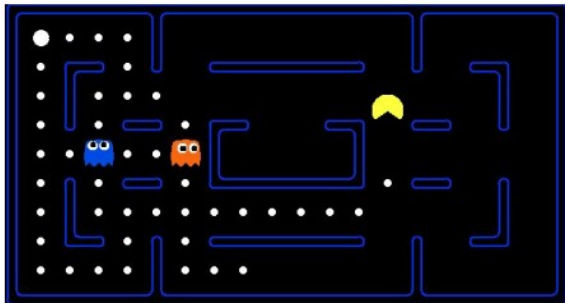
- We improve the Q function on every iteration and once we have the optimal Q function, then we can extract the optimal policy from it.

- Compute Q values for all state-action pairs
  - Example: 9 states (A-I) and 4 actions: Q table = 9 x 4 = 36 rows
- Extract policy by selecting action with maximum Q value
- Exhaustive computation not ideal for large environments
  - Example: 1,000 states, 50 actions; Q table = 1,000 x 50 = 50,000 rows

- Is there a better way?
  - Approximate Q values using function approximators, e.g., neural networks
  - Parameterize Q function with parameter $\theta\,\theta$ (neural network parameter)
  - Feed state to neural network, obtain Q values for all actions
  - Select best action with maximum Q value

Approximate Q-Learning

Imagine a real game and the amount of states!



- Motivation for Approximate Q-learning
  - Q-learning does not scale well for large/medium MDPs
  - Many states and actions make it impractical (e.g., Ms. Pac-Man)
  - Number of possible states can be enormous (e.g., > number of atoms on Earth)

- Solution: Approximate Q-learning
- Use function $Q_\theta(s, a)$ to approximate Q-values
- Manageable number of parameters (parameter vector θ)
- Previously used linear combinations of handcrafted features

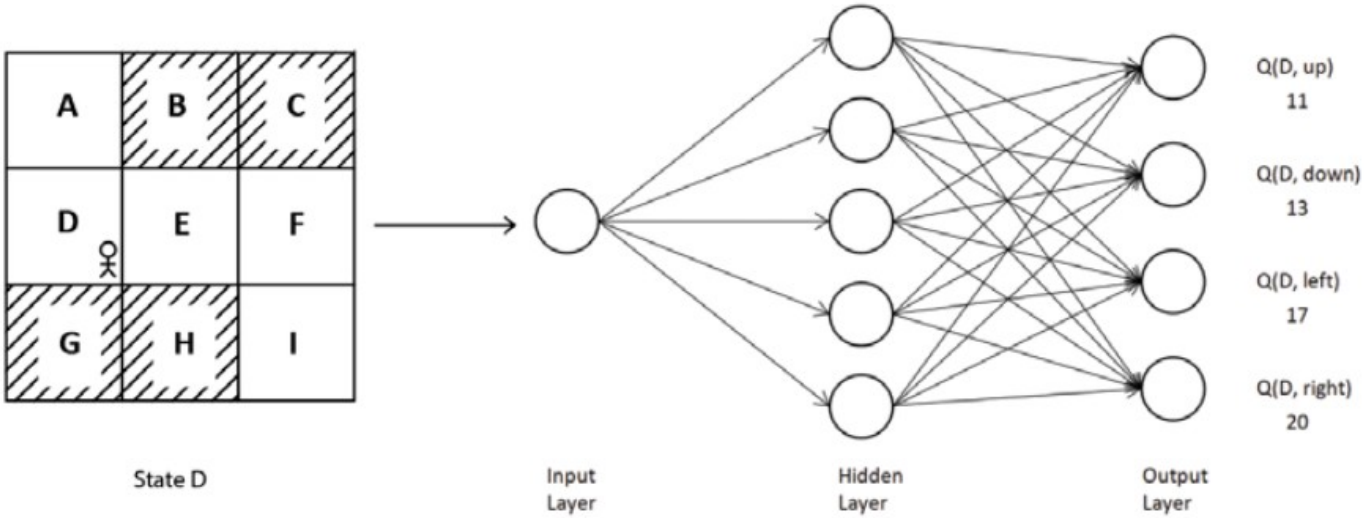
PROS

- Dramatically reduces the size of the Q-table.
- States will share many features.
- Allows generalization to unvisited states.
- Makes behavior more robust: making similar decisions in similar states.
- Handles continuous state spaces!

CONS

- Requires feature selection (often must be done by hand).
- Restricts the accuracy of the learned rewards.
- The true reward function may not be linear in the features

Approximate with DNN

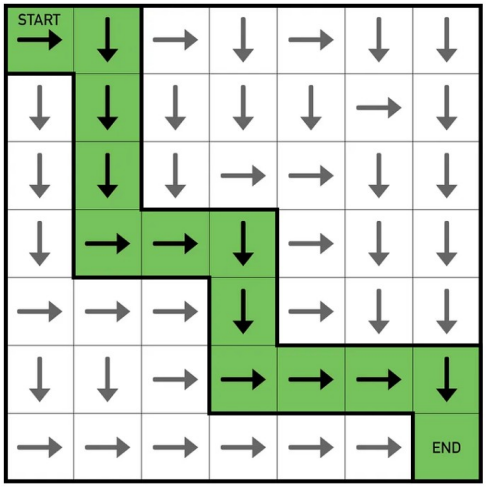


- Approximate Q values using function approximators, like neural networks
- Parameterize Q function with θ, the neural network's parameter; θ representing neural network parameter
- Feed environment state to the neural network, get Q values for all actions
- Select best action with maximum Q value

- Q function as a DNN:
  - Initialize θ with random values, approximate Q function
  - Train network for several iterations to find optimal θ
  - Obtain optimal Q function with optimal θ , extract optimal policy

Q-Learning
And the Deep Q-Network

Goal: Learn how to act optimally in an environment by estimating the quality of state-action pairs (the Q-values).



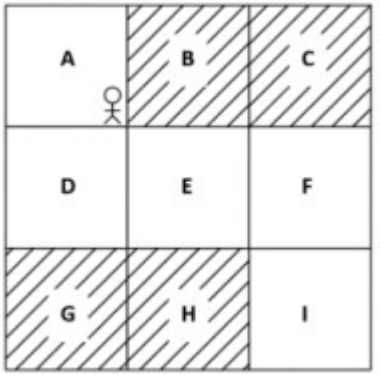
Key Idea:

- We maintain a Q-table: each cell holds Q(s, a), an estimate of how good it is to take action a in state s.
- Over time, we update the Q-values based on the rewards received and our current estimates of future rewards.

Q	a_1	a_2	a_3	a_4
S_1	$Q(s_1,a_1)$	$Q(s_1,a_2)$	$Q(s_1,a_3)$	$Q(s_1,a_4)$
S_2	$Q(s_2,a_1)$	$Q(s_2,a_2)$	$Q(s_2,a_3)$	$Q(s_2,a_4)$
S_3	$Q(s_3,a_1)$	$Q(s_3,a_2)$	$Q(s_3,a_3)$	$Q(s_3,a_4)$
S_4	$Q(s_4,a_1)$	$Q(s_4,a_2)$	$Q(s_4,a_3)$	$Q(s_4,a_4)$

Grid World Example:

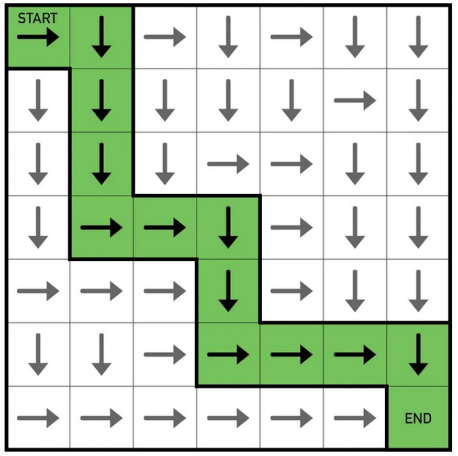
- States: labeled S_1, S_2, ..., S_n (each cell in the grid).
- Actions: move left, right, up, down (or whichever moves are permitted).



Q-learning and Policy

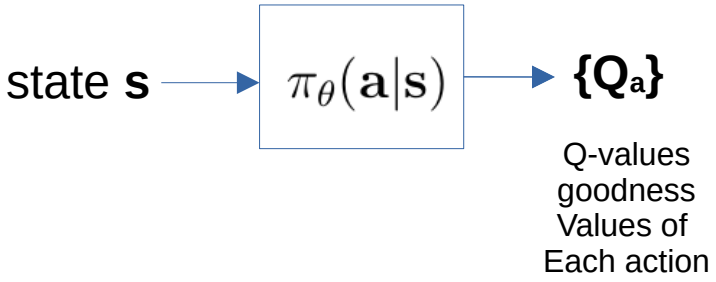
1. We have an agent moving in an environment:

- (left, right, up, down)
- rewards/penalties



2. We imagine that we can quantify the "goodness" of each action

$\{Q_a\}$ Q-values goodness values of each action



Q	a_1	a_2	a_3	a_4
S_1	$Q(s_1,a_1)$	$Q(s_1,a_2)$	$Q(s_1,a_3)$	$Q(s_1,a_4)$
S_2	$Q(s_2,a_1)$	$Q(s_2,a_2)$	$Q(s_2,a_3)$	$Q(s_2,a_4)$
S_3	$Q(s_3,a_1)$	$Q(s_3,a_2)$	$Q(s_3,a_3)$	$Q(s_3,a_4)$
S_4	$Q(s_4,a_1)$	$Q(s_4,a_2)$	$Q(s_4,a_3)$	$Q(s_4,a_4)$

Details of the Q-table

Q	a_1	a_2	a_3	a_4
S_1	$Q(s_1,a_1)$	$Q(s_1,a_2)$	$Q(s_1,a_3)$	$Q(s_1,a_4)$
S_2	$Q(s_2,a_1)$	$Q(s_2,a_2)$	$Q(s_2,a_3)$	$Q(s_2,a_4)$
S_3	$Q(s_3,a_1)$	$Q(s_3,a_2)$	$Q(s_3,a_3)$	$Q(s_3,a_4)$
S_4	$Q(s_4,a_1)$	$Q(s_4,a_2)$	$Q(s_4,a_3)$	$Q(s_4,a_4)$

Goal: Learn these Q values such that the total reward is maximized

state **s** → $\pi_\theta(a|s)$ → **{Q_a}**

Q-values goodness values of each action **a**

A universal mapping from the state to a "goodness value" q_a for each action

How??

Iterate through episodes in order to find these

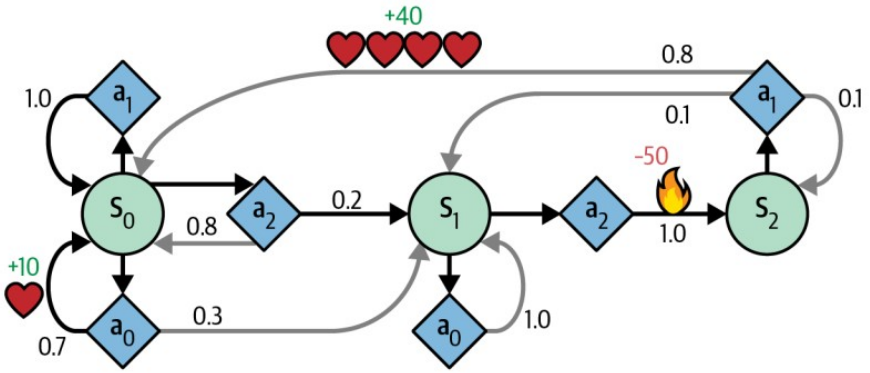
Blocked	U: -1.00 D: 62.47 L: -1.00 R: -1.00	Blocked	U: 2.00 D: 58.63 L: 2.00 R: 2.00
U: 6.00 D: 63.17 L: 6.00 R: 69.47	U: 64.23 D: 67.83 L: 70.53 R: 66.93	U: 2.00 D: 2.00 L: 65.47 R: 58.63	U: 56.77 D: 59.47 L: 62.93 R: 4.00
U: 63.53 D: 58.76 L: 1.00 R: 60.83	U: 66.47 D: 3.00 L: 60.17 R: 3.00	Blocked	U: 61.63 D: 5.00 L: 5.00 R: 5.00
U: 64.17 D: 7.00 L: 7.00 R: 7.00	Blocked	U: 9.00 D: 9.00 L: 9.00 R: 9.00	Blocked

The Bellman Equation (Intuition)

Bellman Equation for Q-Learning (one-step look-ahead):

$$Q(s,a) \leftarrow R(s') + \gamma \max_{a'} Q(s',a')$$

$S_1, S_2, \ldots, S_n$



s' next state after taking action a in state s

$R(s')$ immediate reward for arriving in state s'

γ discount factor (values future rewards vs. immediate ones).

Intuition:

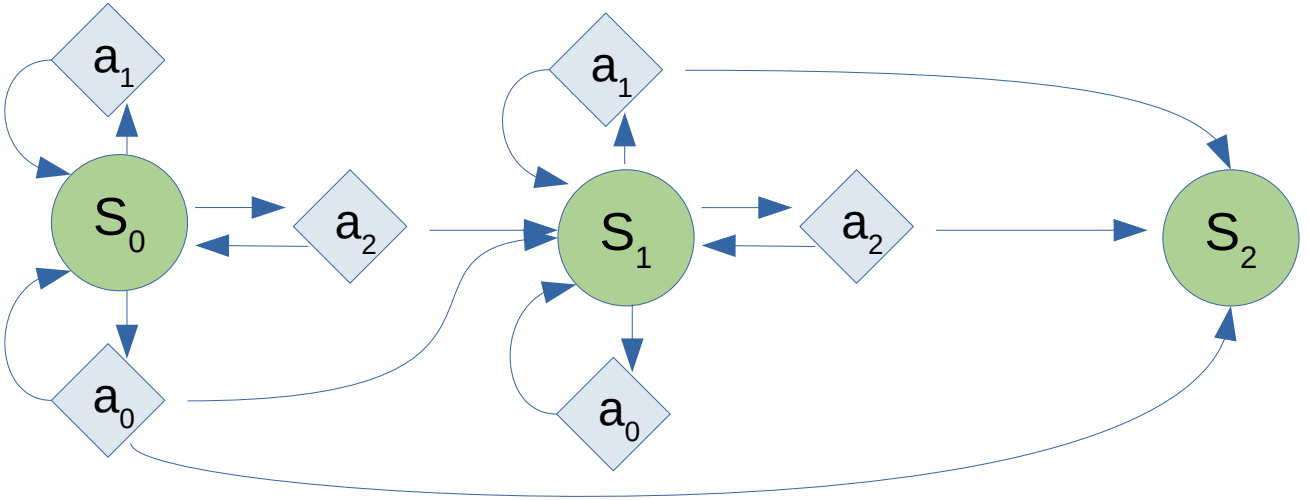
- The value of taking action **a** in state **s** includes both the reward now and the best possible future rewards
- Helps the agent back up future success to earlier decisions.

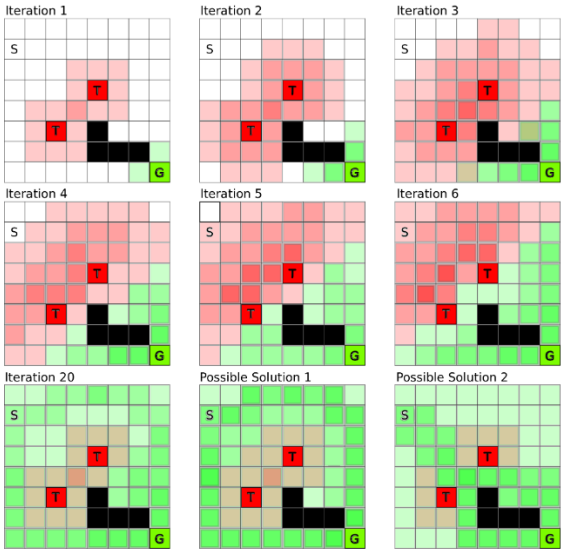
as captured by:

$\max_{a'} Q(s',a')$

Blocked	U: -1.00 D: 62.47 L: -1.00 R: -1.00	Blocked	U: 2.00 D: 58.63 L: 2.00 R: 2.00
U: 6.00 D: 63.17 L: 6.00 R: 69.47	U: 64.23 D: 67.83 L: 70.53 R: 66.93	U: 2.00 D: 2.00 L: 65.47 R: 58.63	U: 56.77 D: 59.47 L: 62.93 R: 4.00
U: 63.53 D: 58.76 L: 1.00 R: 60.83	U: 66.47 D: 3.00 L: 60.17 R: 3.00	Blocked	U: 61.63 D: 5.00 L: 5.00 R: 5.00
U: 64.17 D: 7.00 L: 7.00 R: 7.00	Blocked	U: 9.00 D: 9.00 L: 9.00 R: 9.00	Blocked

We can form the Graph, if we know the “Rewards” for each action

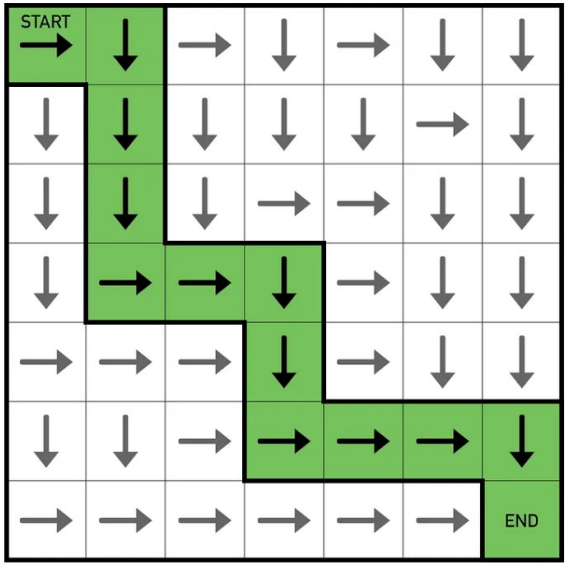




We iteratively Update to refine the Q table

Q	a_1	a_2	a_3	a_4
S_1	$Q(s_1,a_1)$	$Q(s_1,a_2)$	$Q(s_1,a_3)$	$Q(s_1,a_4)$
S_2	$Q(s_2,a_1)$	$Q(s_2,a_2)$	$Q(s_2,a_3)$	$Q(s_2,a_4)$
S_3	$Q(s_3,a_1)$	$Q(s_3,a_2)$	$Q(s_3,a_3)$	$Q(s_3,a_4)$
S_4	$Q(s_4,a_1)$	$Q(s_4,a_2)$	$Q(s_4,a_3)$	$Q(s_4,a_4)$

Result will be an optimal Path (or policy)



Temporal Difference (TD) and Q-update

Observation vs. Expectation:

Observed: $Q(s,a)_{\text{obs}} = R(s') + \gamma \max_{a'} Q(s',a')$

Expected: $Q(s,a)_{\text{exp}}$ our current estimate (what's in the Q-table before updating).

TD Error:

$$\text{TD}_{\text{error}} = Q(s,a)_{\text{obs}} - Q(s,a)_{\text{exp}}$$

Why "Temporal Difference"?

- We use a difference in value estimates across a single step in time, rather than waiting for the entire outcome of an episode.
- Allows learning to occur incrementally after each action

Q-update

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\text{TD}_{\text{error}}\right]$$

α learning rate (how aggressively we adjust based on new information)

The Q-Update Step

Algorithm

1. Take an action a in state s.
2. Observe next state s' and reward R(s')
3. Compute the target:

$$Q(s,a)_{\text{obs}} = R(s') + \gamma \max_{a'} Q(s',a')$$

4. Calculate TD error:

$$\text{TD}_{\text{error}} = Q(s,a)_{\text{obs}} - Q(s,a)_{\text{exp}}$$

5. Update Q(s, a):

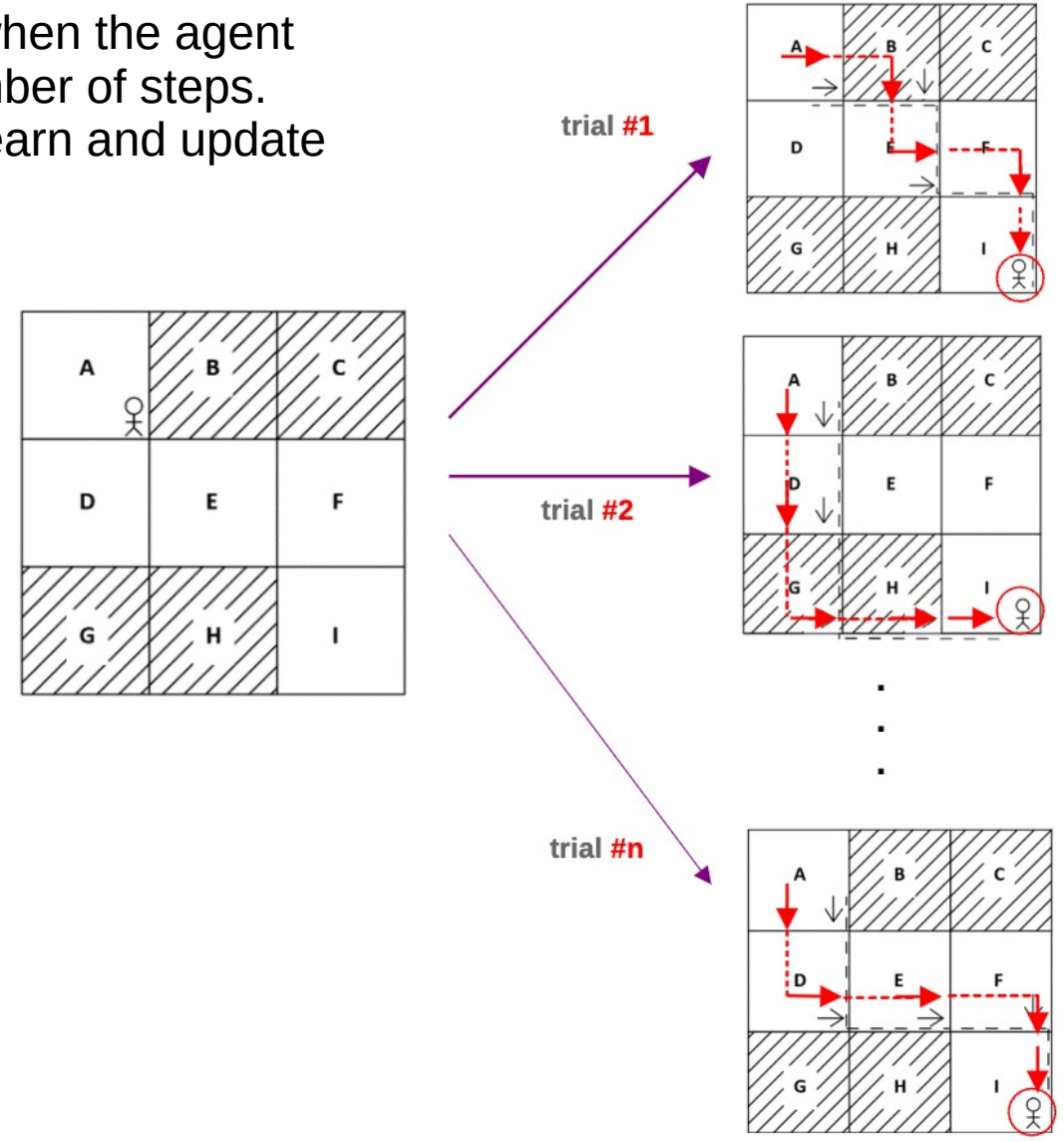
$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\text{TD}_{\text{error}}\right]$$

Result: Q-values gradually converge to the true action-value function for the optimal policy.

Episode

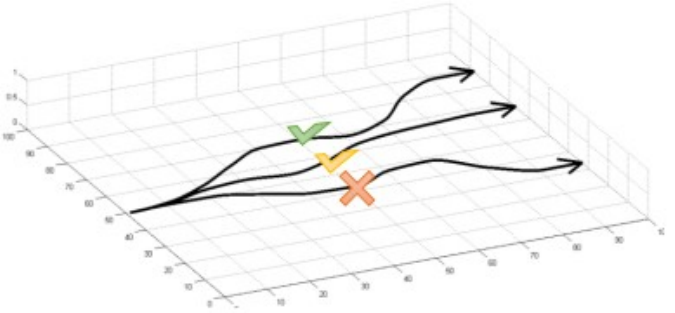
Episode:

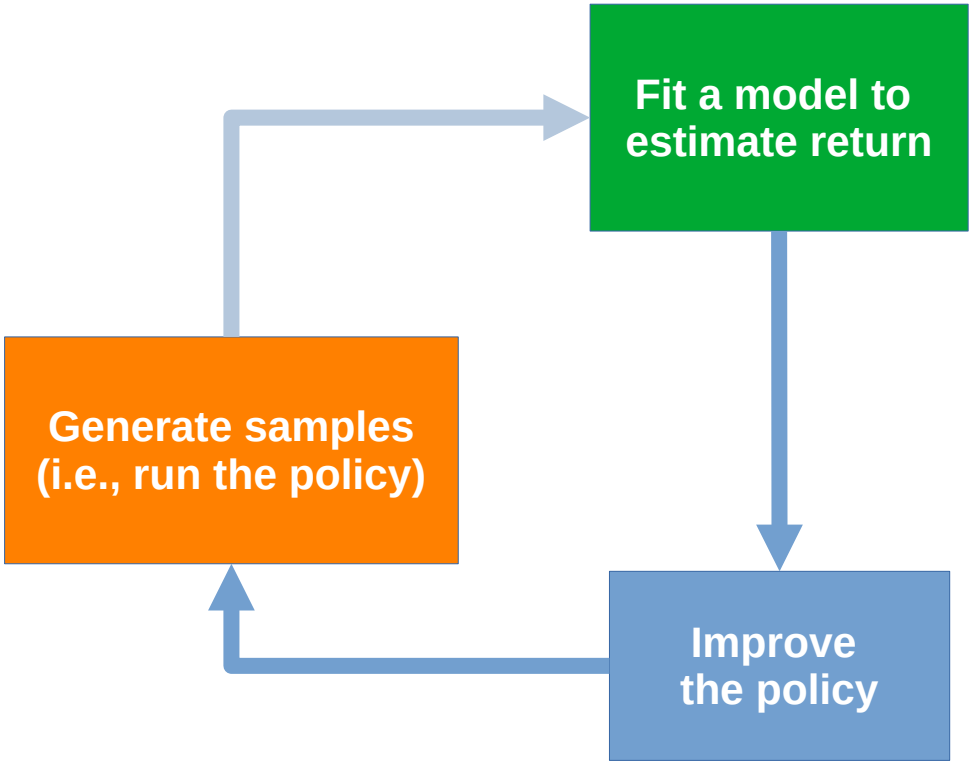
- A sequence of states, actions, and rewards from the start state to a terminal state (or until a set time horizon).
- In grid world, an episode might end when the agent reaches a goal, or after a certain number of steps.
- Each episode provides a chance to learn and update Q-values.



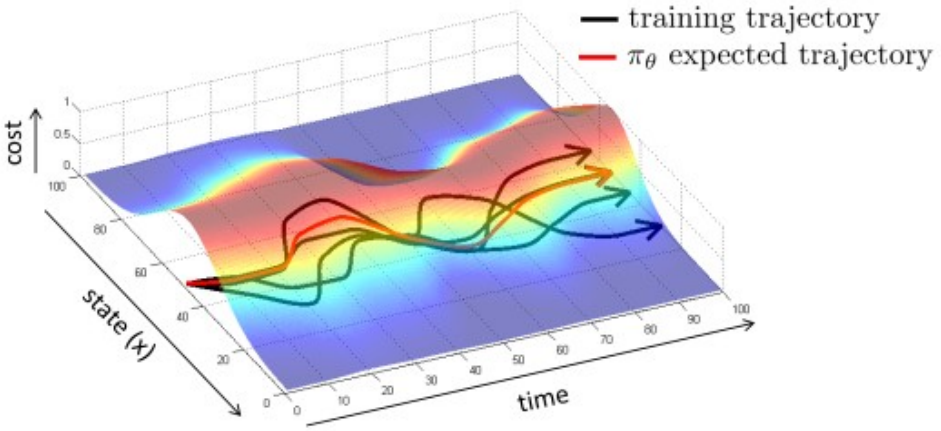
Policy:

- A policy \(\pi \) specifies which action \(a \) to take in state \(s \).
- In Q-learning, we often use an \(\epsilon\)-greedy policy during training:
  - With probability \(\epsilon \), choose a random action to explore.
  - With probability \(1 - \epsilon \), choose the action that maximizes the current \(Q(s, a) \).



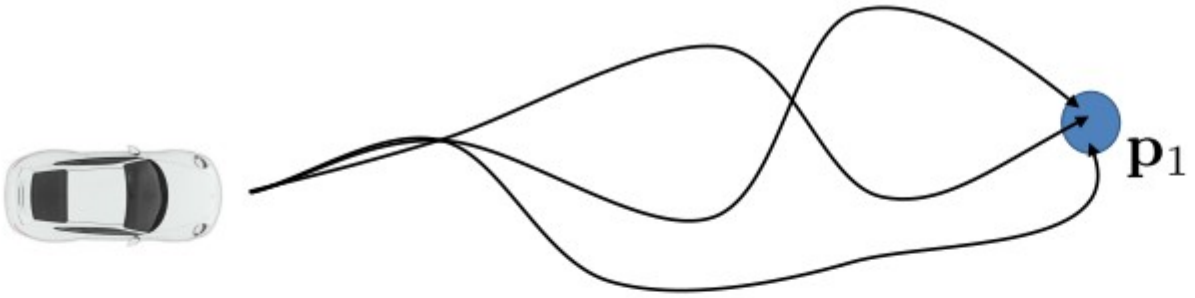


Over time, as Q-values improve, the policy becomes better at choosing actions that lead to higher rewards.



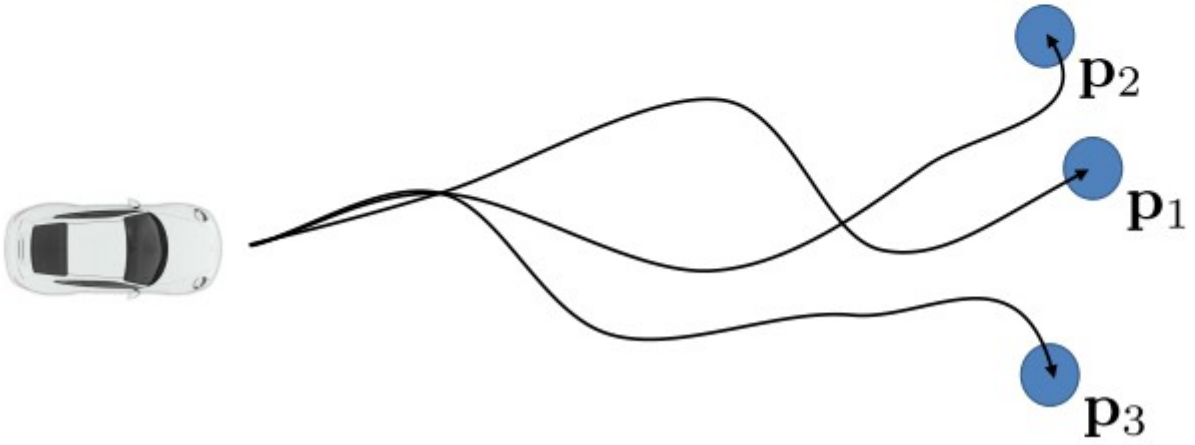
Intuition about a “Policy”

What is behind the math?



$\pi_\theta(\mathbf{a}|\mathbf{s})$

policy for reaching $\mathbf{p}_1$



$\pi_\theta(\mathbf{a}|\mathbf{s},\mathbf{p})$

policy for reaching *any* $\mathbf{p}$

Q-Learning

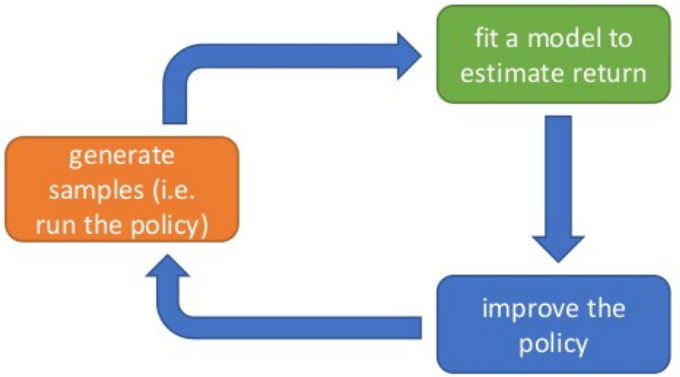
full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy

$K\times$

2. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

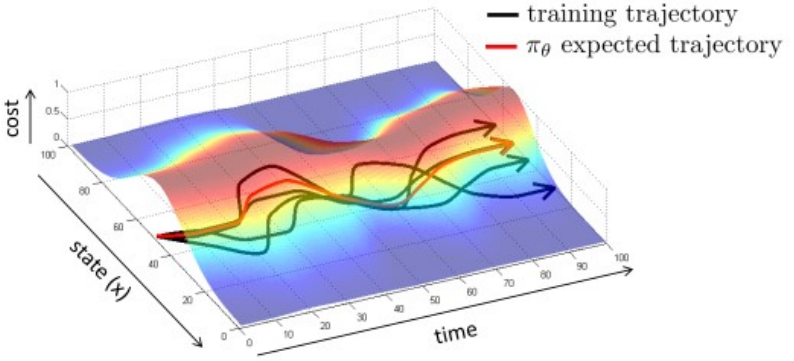
$Q_\phi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$



$\mathbf{a} = \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})$

online Q iteration algorithm:

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$



state **s** → $\pi_\theta(\mathbf{a}|\mathbf{s})$ → **{Q_a}**

Q-values
goodness
Values of
Each action

Q-Learning Step-by-Step

Example with Grid World

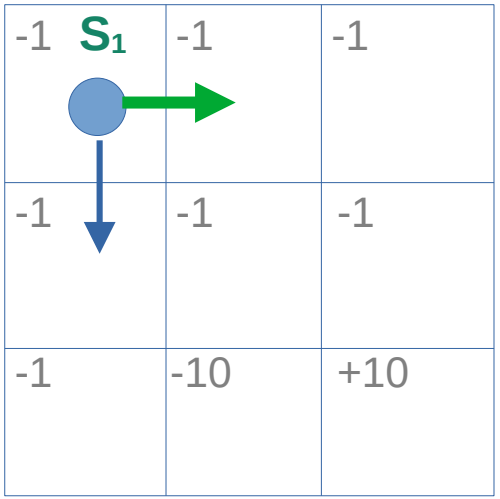
-1	-1	-1
-1	-1	-1
-1	-10	+10

Grid world: 3x3 (observable env)

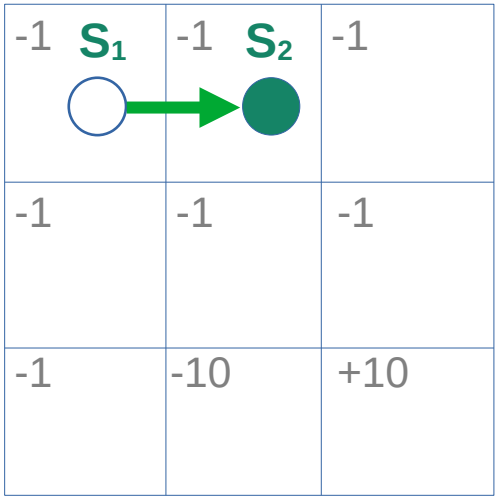
Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	1	2.1	1.3
S_2	0.5	0.75	-0.5	1.5
S_6	-1.2	1.2	0.7	1.7

Start with Random values

Recap: Q-Learning



Agent takes a decision based upon a behavior policy; in this case random



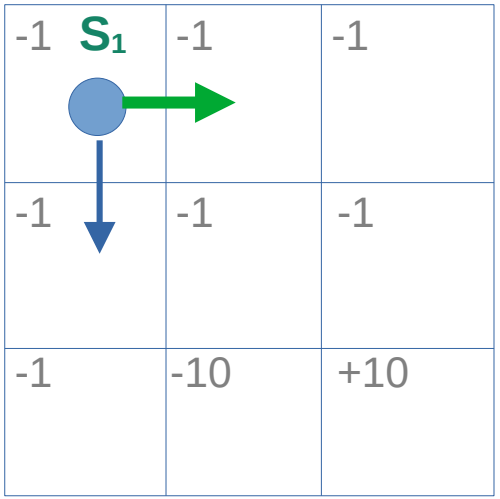
Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	**1**	2.1	1.3
S_2	0.5	0.75	-0.5	**1.5**
S_6	-1.2	1.2	0.7	1.7

Observed:

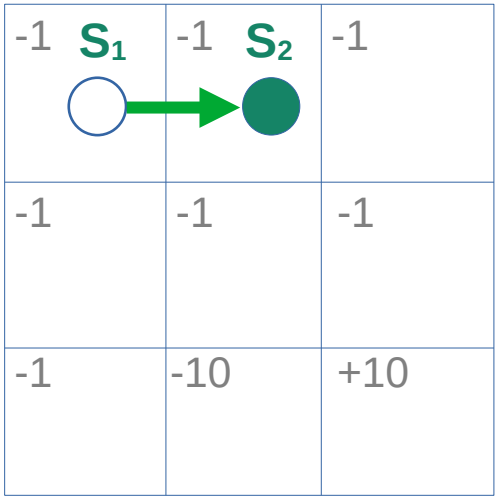
$$Q(S_1, \text{right})_{ob} = R(S_2) + \gamma \max_a Q(S_2, a)$$

$$= -1 + 0.1 * 1.5 = \mathbf{-0.85}$$

Recap: Q-Learning



Agent takes a decision based upon a behavior policy; in this case random



Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	**1**	2.1	1.3
S_2	0.5	0.75	-0.5	1.5
S_6	-1.2	1.2	0.7	1.7

Observed:

$$Q(S_1, \text{right})_{ob} = R(S_2) + \gamma \max_a Q(S_2, a)$$

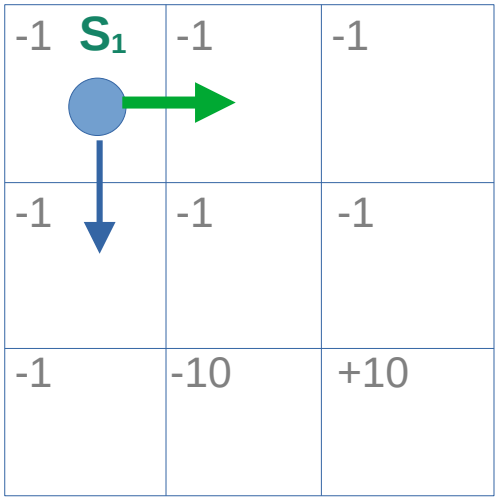
= -1 + 0.1 * 1.5 = **-0.85**

Temporal difference (TD): Observed - Expected

$$\text{TD Error} = Q(S_1, \text{right})_{ob} - Q(S_1, \text{right})_{ex}$$

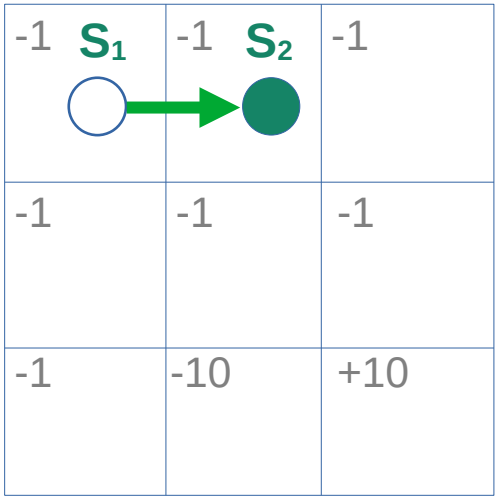
= -0.85 - 1 = **-1.85**

Recap: Q-Learning



Agent takes a decision based upon a behavior policy; in this case random

Step 1.



Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	**0.815**	2.1	1.3
S_2	0.5	0.75	-0.5	**1.5**
S_6	-1.2	1.2	0.7	1.7

Observed:

$$Q(S_1, \text{right})_{ob} = R(S_2) + \gamma \max_a Q(S_2, a)$$

= -1 + 0.1 * 1.5 = **-0.85**

Temporal difference (TD): Observed - Expected

$$\text{TD Error} = Q(S_1, \text{right})_{ob} - Q(S_1, \text{right})_{ex}$$

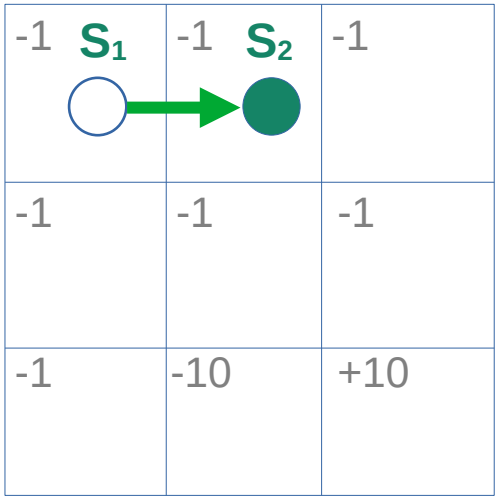
= -0.85 - 1 = **-1.85**

Update Rule:

$$Q(S_1, \text{right}) = Q(S_1, \text{right}) + \alpha \times \text{TD Error}$$

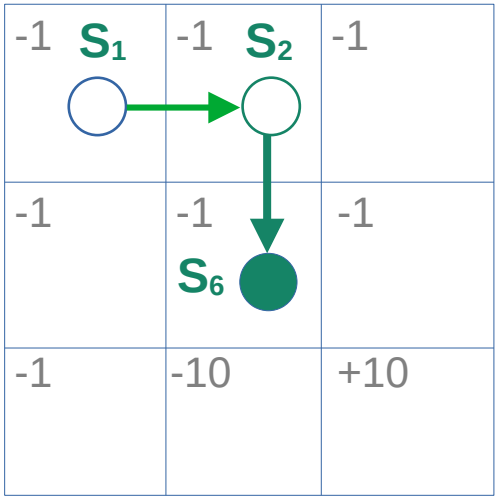
= 1 + 0.1 *(-1.85) = **0.815**

Recap: Q-Learning



Agent takes a decision based upon a behavior policy; in this case random

Step 2.



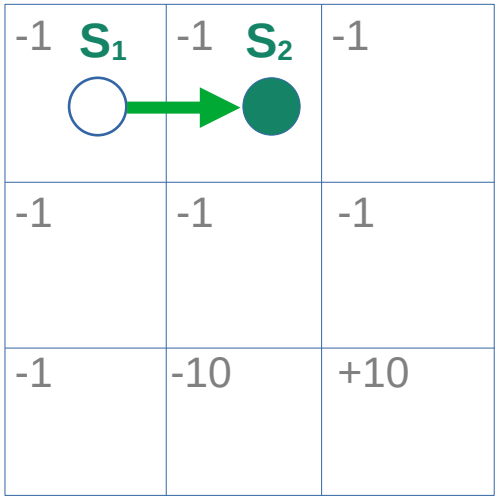
Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	0.815	2.1	1.3
S_2	0.5	0.75	-0.5	1.5
S_6	-1.2	1.2	0.7	**1.7**

Observed:

$$Q(S_2, \text{down})_{ob} = R(S_6) + \gamma \max_a Q(S_6, a)$$

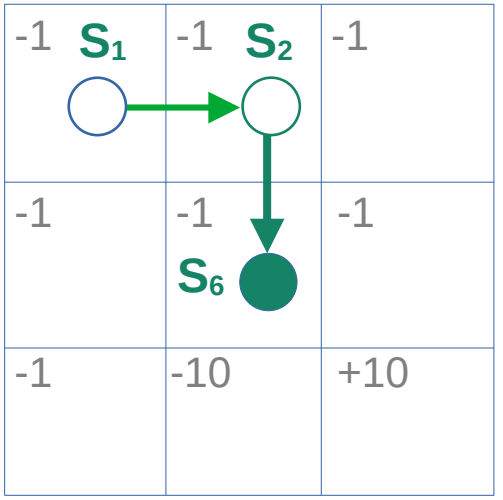
$$= -1 + 0.1 * 1.7 = \mathbf{-0.83}$$

Recap: Q-Learning



Agent takes a decision based upon a behavior policy; in this case random

Step 2.



Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	0.815	2.1	1.3
S_2	0.5	0.75	-0.5	1.5
S_6	-1.2	1.2	0.7	**1.7**

Observed:

$$Q(S_2, \text{down})_{ob} = R(S_6) + \gamma \max_a Q(S_6, a)$$

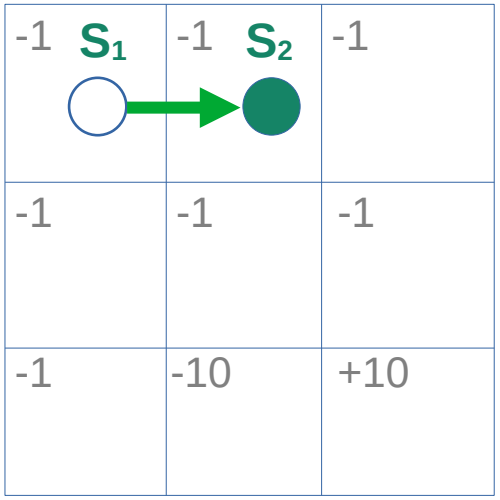
= -1 + 0.1 * 1.7 = **-0.83**

Temporal difference (TD): Observed - Expected

$$\text{TD Error}_{t=2} = Q(S_2, \text{down})_{ob} - Q(S_2, \text{down})_{ex}$$

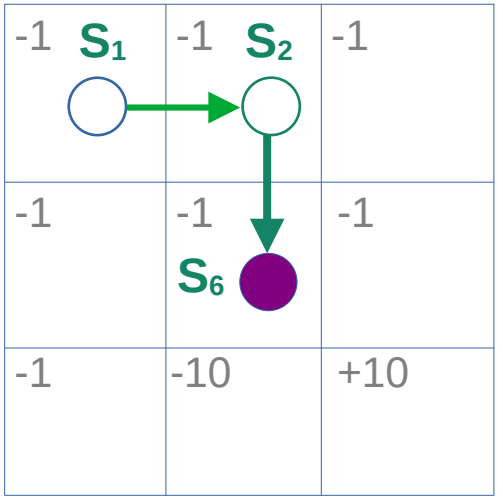
= -0.83 – 1.5 = **-2.33**

Recap: Q-Learning



Agent takes a decision based upon a behavior policy; in this case random

Step 2.



Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	**0.815**	2.1	1.3
S_2	0.5	0.75	-0.5	**1.267**
S_6	-1.2	1.2	0.7	1.7

Observed:

$$Q(S_2, \text{down})_{ob} = R(S_6) + \gamma \max_a Q(S_6, a)$$

= -1 + 0.1 * 1.7 = **-0.83**

Temporal difference (TD): Observed - Expected

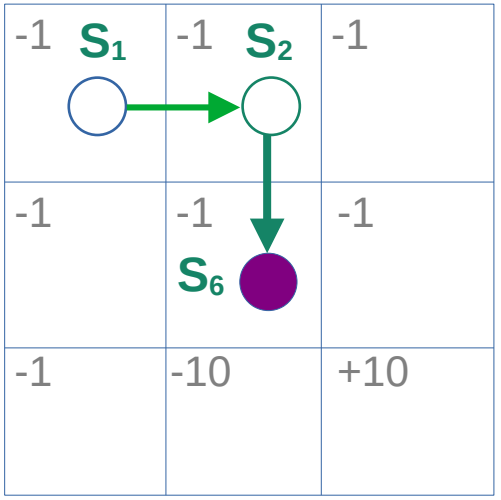
$$\text{TD Error}_{t=2} = Q(S_2, \text{down})_{ob} - Q(S_2, \text{down})_{ex}$$

= -0.83 – 1.5 = **-2.33**

Update Rule:

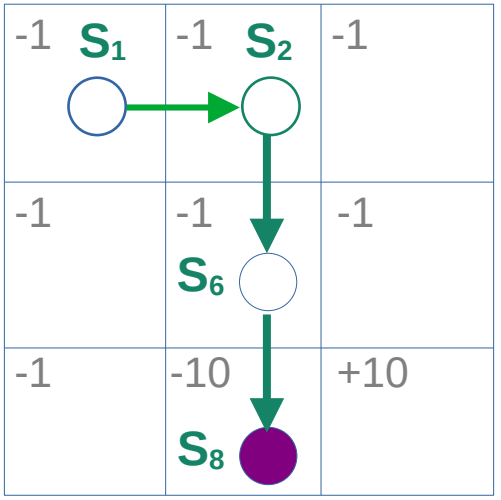
$$Q(S_2, \text{down}) = Q(S_2, \text{down}) + \alpha \times \text{TD Error}_{t=2}$$

= 1.5 + 0.1 *(-2.33) = **1.267**



Step 3...n

“One episode”



Target policy: The policy the agent is trying to learn

Behavior policy: The policy the agent uses to learn the target policy

Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	1	2.1	1.3
S_2	0.5	0.75	-0.5	1.5
S_6	-1.2	1.2	0.7	1.7

After iteration of an episode

Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	0.815	2.1	1.3
S_2	0.5	0.75	-0.5	1.267
S_6	-1.2	1.2	0.7	2.54

What if there is stochasticity?

$$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

The expectation value that s' is taken from P (the policy)

Q-Learning: Optimal Policy

state s → $\pi_\theta(\mathbf{a}|\mathbf{s})$ → V-number

Tells us how good to be in state s

state **s** → $\pi_\theta(\mathbf{a}|\mathbf{s})$ → **{Q_a}**

Q-values goodness values of each action **a**

A universal mapping from the state to a "goodness value" q_a for each action

---

1 $$Q^*(s,a) = \mathbb{E}_{s' \sim P}\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

2 Q-value iteration algorithm

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma \cdot \max_{a'} Q_k(s',a')\right]$$

3 the optimal policy $\pi^\star(s)$ for all (s,a)

$$\pi^\star(s) = \operatorname*{argmax}_a Q^\star(s,a)$$

Q-Learning

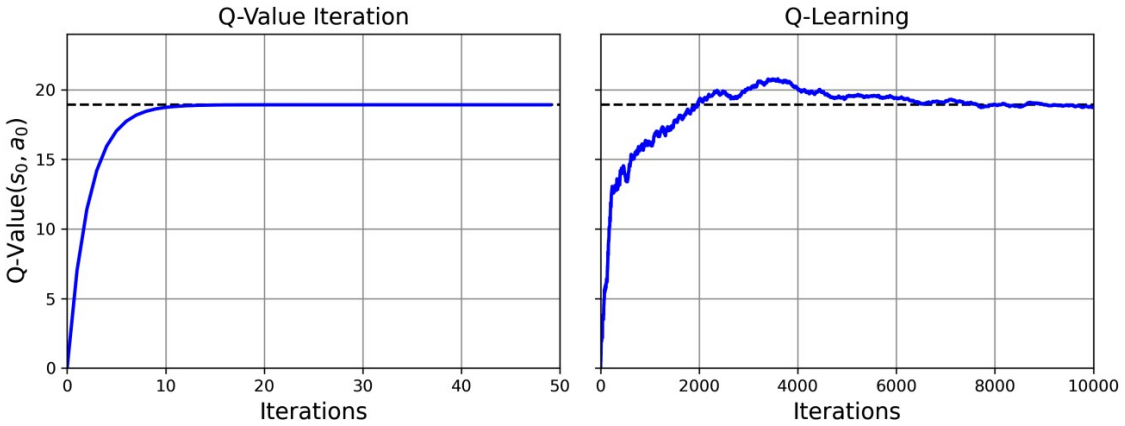
Q-learning algorithm is the Q-value iteration when transition probabilities and the rewards are initially unknown

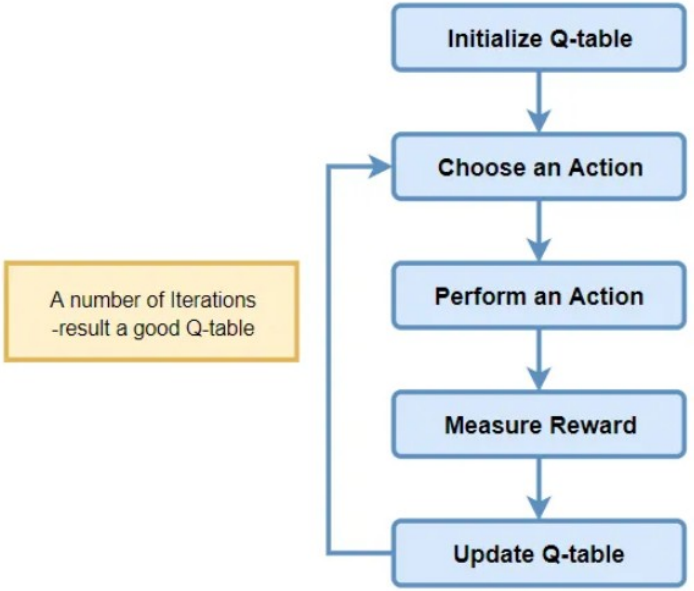
Q-learning: watch an agent play; improve estimates of Q-values.

With accurate Q-value estimates; the optimal policy is action with highest Q-value (i.e., the greedy policy).

- Algorithm tracks state-action pairs (s, a)
- Maintains running average of rewards (r)
- Considers rewards upon leaving state s with action a
- Includes sum of discounted future rewards
- Estimates sum using maximum Q-value for next state s′
- Assumes target policy acts optimally from then on

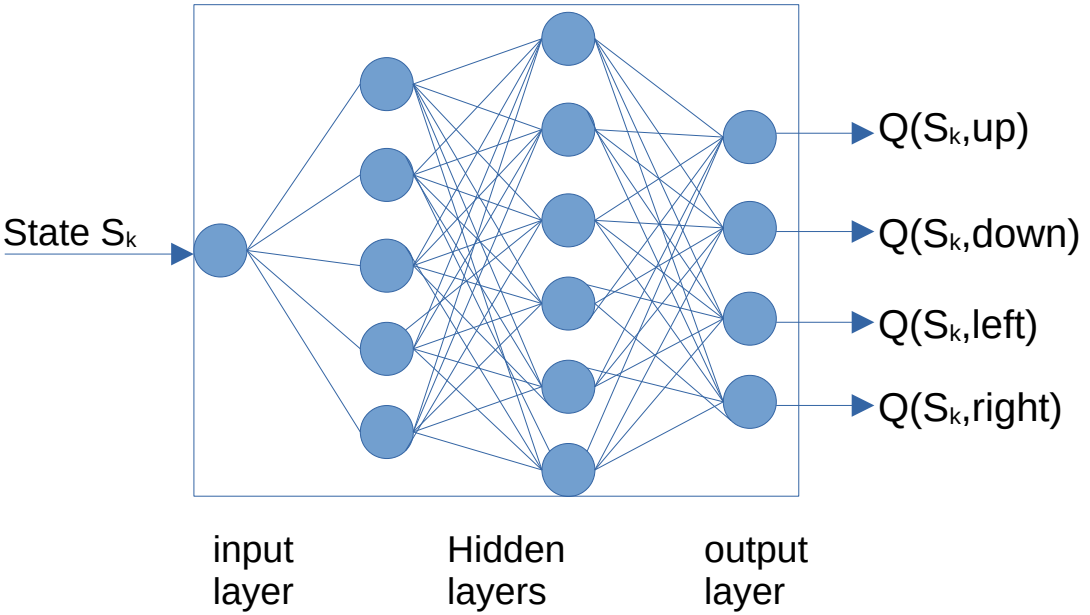
$$Q(s,a) \underset{\alpha}{\leftarrow} r + \gamma \cdot \max_{a'} Q(s',a')$$

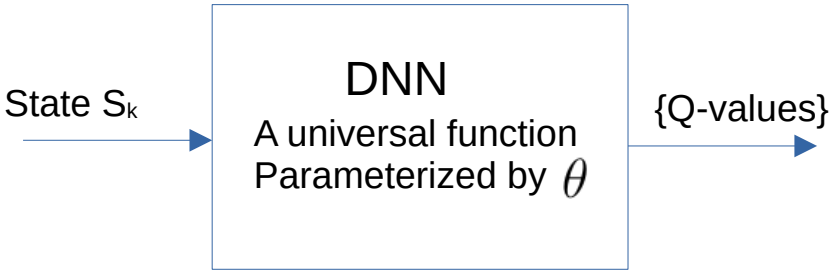




Deep Q-Network

Q	a_1 left	a_2 right	a_3 up	a_4 down
S_1	-0.5	0.815	2.1	1.3
S_2	0.5	1.267	-0.5	1.5
S_6	-1.2	1.2	0.7	2.54



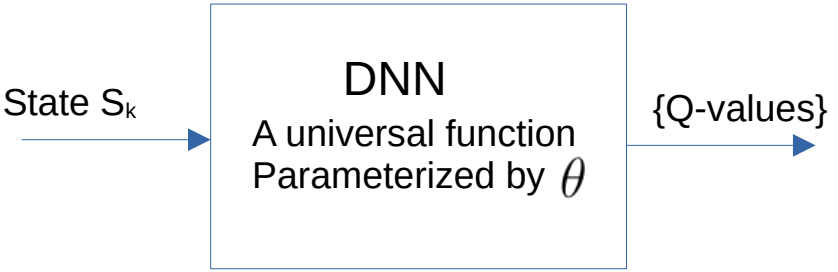


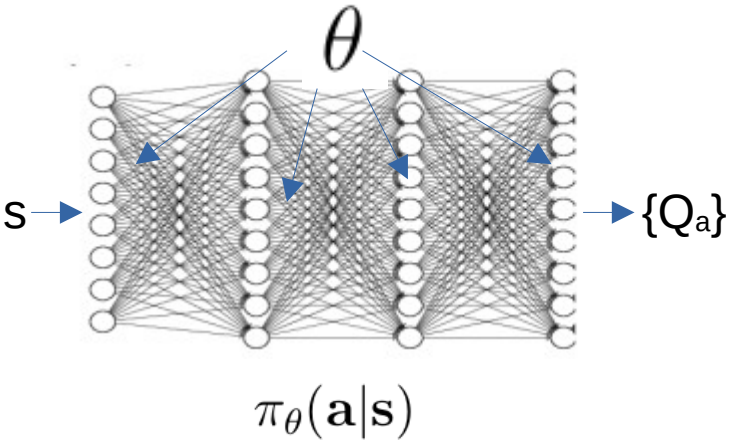
Q-learning: Learns values in the table

Deep Q learning: Learns the parameters of the Neural Network

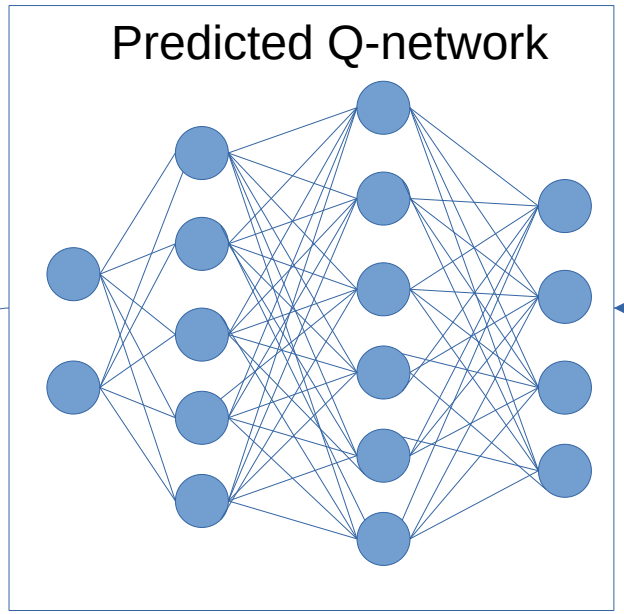
Deep Q-Network

- DQN: General Idea
- How to collect Data
- Training Process





- Input to Network: The state s
  - Gridworld: the position in the grid
  - Lunar lander: (e.g., the lander's position, velocity, orientation).
- Output: Q-values for each possible action in that state.
- Use of r and a:
  - a: Identifies which action's Q-value to adjust in the loss calculation.
  - r: Contributes to the target Q-value, which helps inform the agent of the immediate reward it received by taking action aaa in state s.



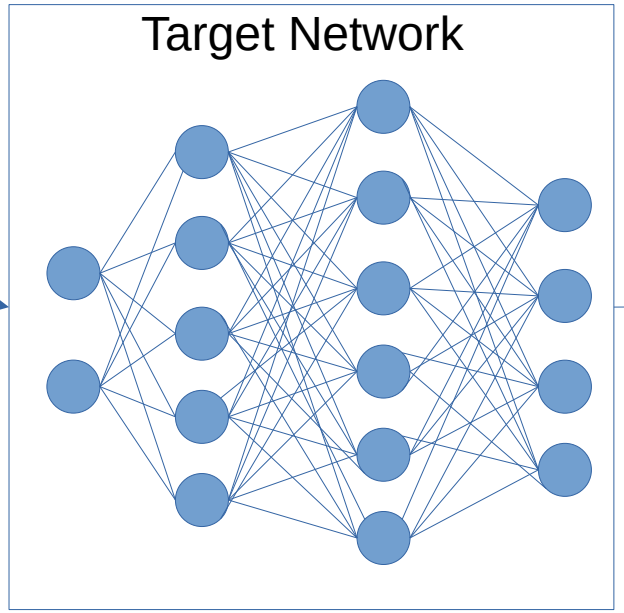
Randomly initialized

Backpropagation

Update
after k-epochs

Loss
$L(\theta, \theta')$

TD update
$\theta = \theta - \alpha \nabla_\theta L(\theta)$



$$L(\theta) = Q^*(s,a) - Q_\theta(s,a)$$

target predicted

$$L(\theta) = \frac{1}{K}\sum_{i=1}^{K}(r_i + \gamma \max_{a'} Q_{\theta'}(s_i', a') - Q_\theta(s_i, a_i))^2$$

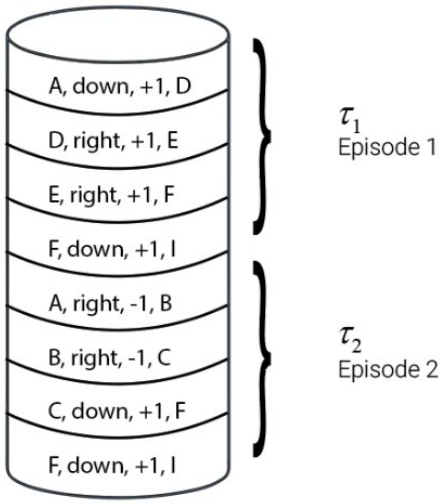
Compute with θ′

Compute with θ

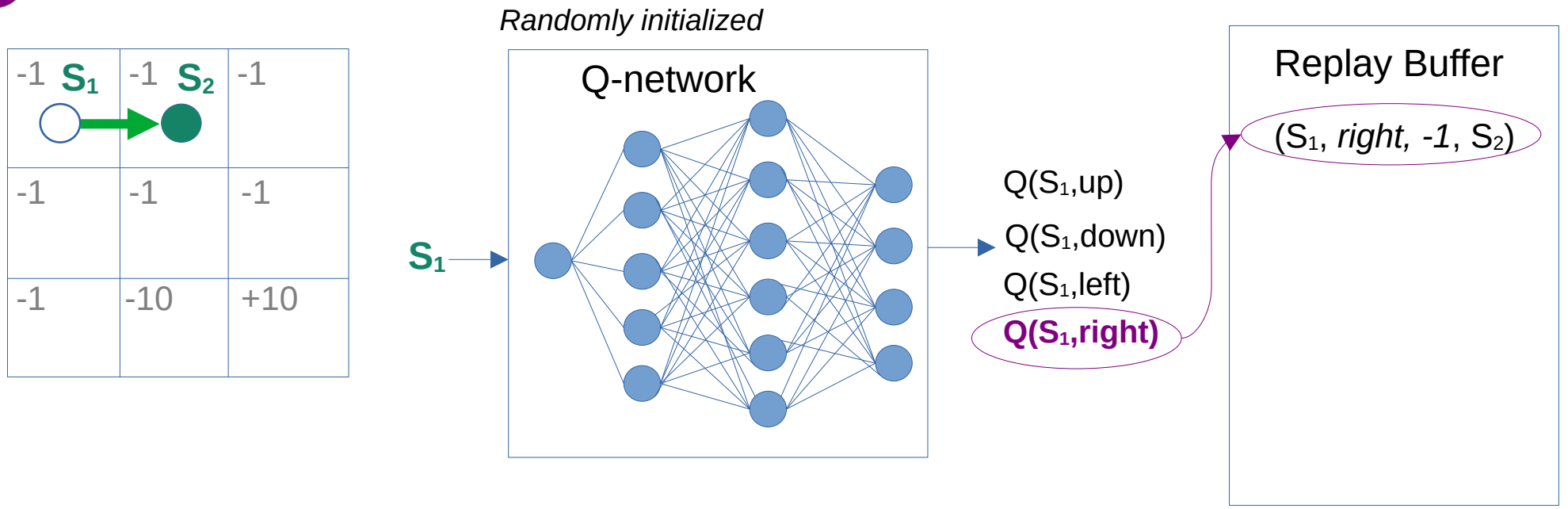
$$\text{MSE} = \frac{1}{K}\sum_{i=1}^{K}(y_i - \hat{y}_i)^2$$

How do I run an episode?
The Replay Buffer

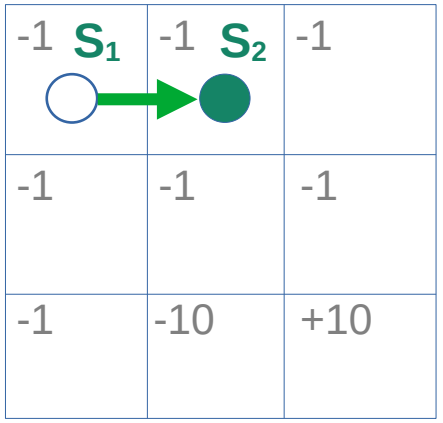
Replay Buffer



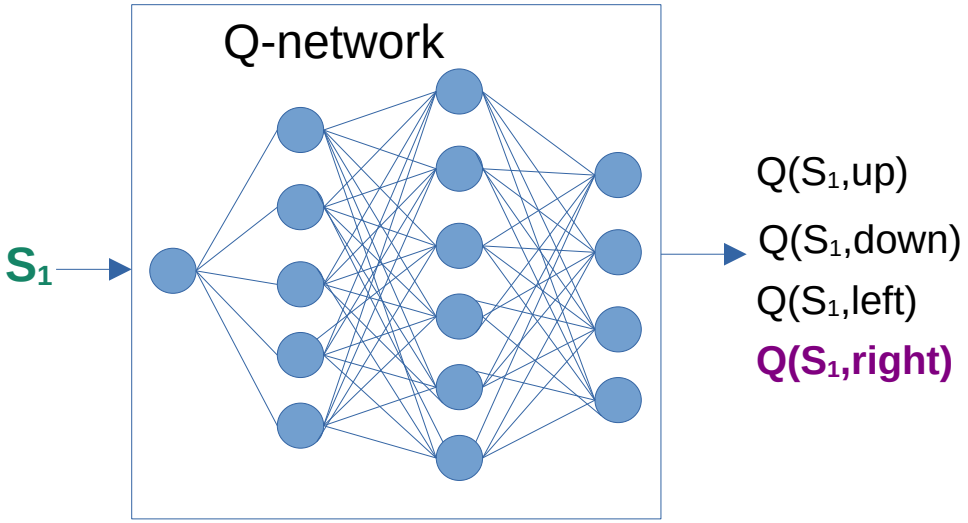
1 *Recap of DQN*



1 *Recap of DQN*

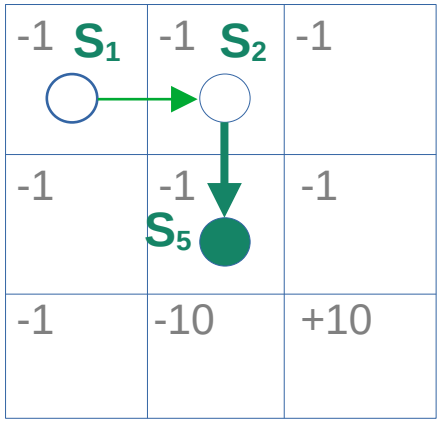


Randomly initialized

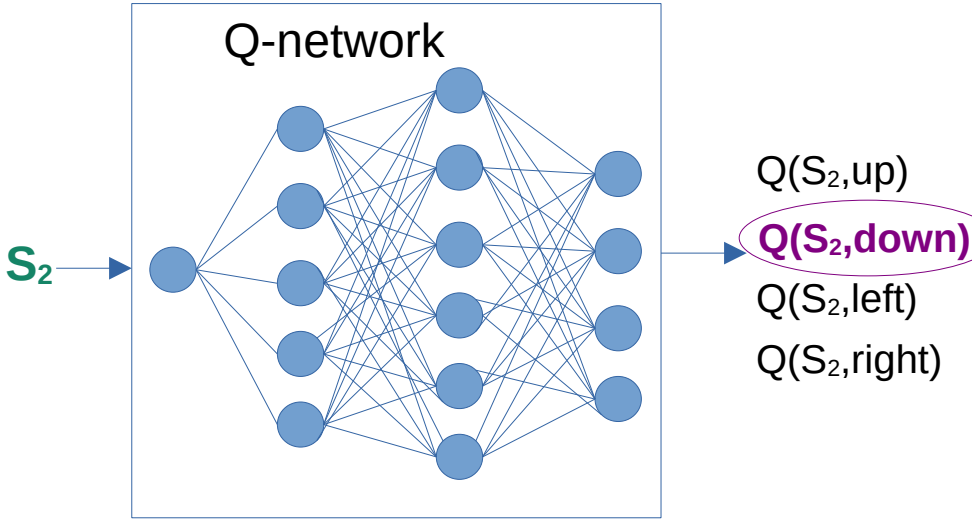


Replay Buffer

(S_1, *right*, *-1*, S_2)



Randomly initialized

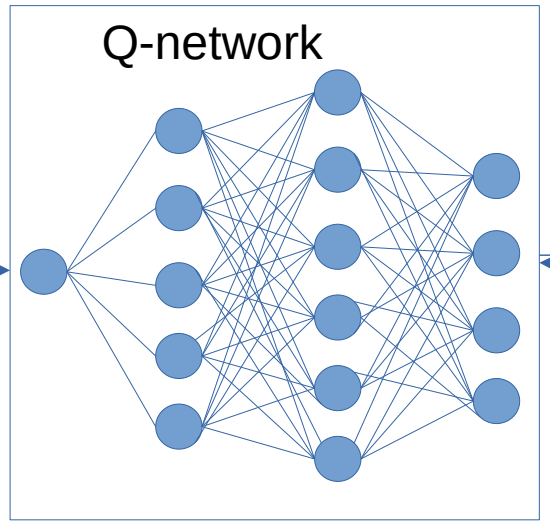


Replay Buffer

(S_1, *right*, *-1*, S_2)

(S_2, *right*, *-1*, S_5)

Training the DQN

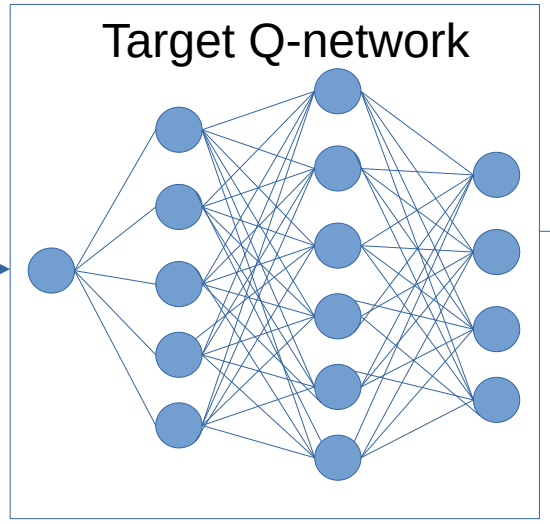


$Q(S_1,up)$ -1.63
$Q(S_1,down)$ 0.72
$Q(S_1,left)$ 1.36
$Q(S_1,right)$ -0.67

Replay Buffer

(S_1, *right*, *-1*, S_2)

Next State: Pass to target network



$Qt(S_2,up)$ -1.36
$Qt(S_2,down)$ 1.42
$Qt(S_2,left)$ -0.66
$Qt(S_2,right)$ 1.02

$$\text{Loss} = (Q(S_1, \text{right}) - 0.4058)^2$$

$$= (-0.67 - 1.42)^2$$

$$\theta = \theta - \alpha \nabla_\theta L(\theta)$$

backpropagate

Do this for some batches; Q-network learns

After a few batches: update Target network Qt

$$L(\theta) = \frac{1}{K} \sum_{i=1}^{K} (r_i + \gamma \max_{a'} Q_{\theta'}(s_i', a') - Q_\theta(s_i, a_i))^2$$

Q-Learning

full fitted Q-iteration algorithm:

1. collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)\}$ using some policy

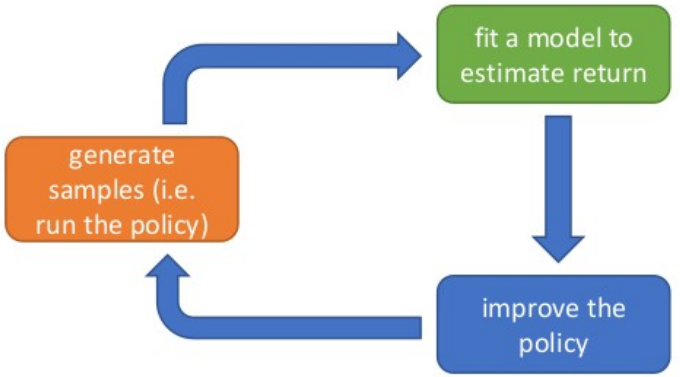
$K\times$

2. set $\mathbf{y}_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'_i} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. set $\phi \leftarrow \arg\min_\phi \frac{1}{2} \sum_i \|Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i\|^2$

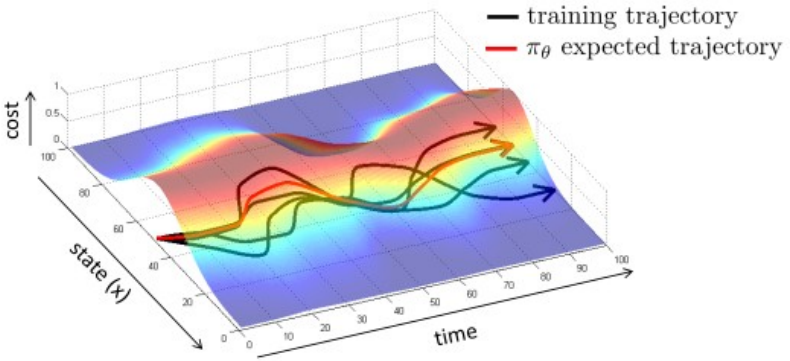
online Q iteration algorithm:

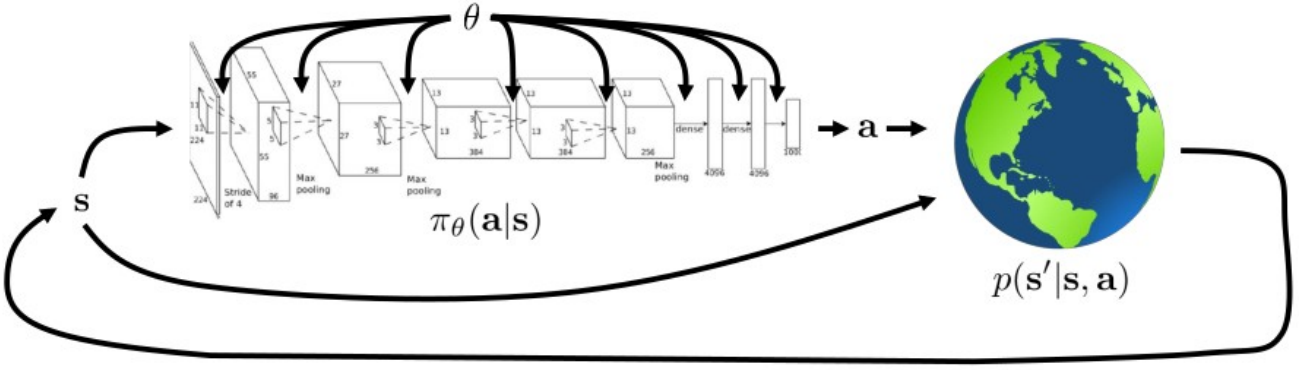
1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$
2. $\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$
3. $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$

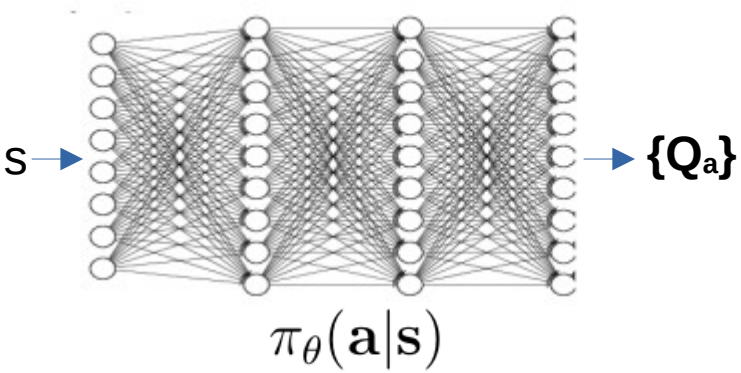
$Q_\phi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$



$\mathbf{a} = \arg\max_{\mathbf{a}} Q_\phi(\mathbf{s}, \mathbf{a})$







5

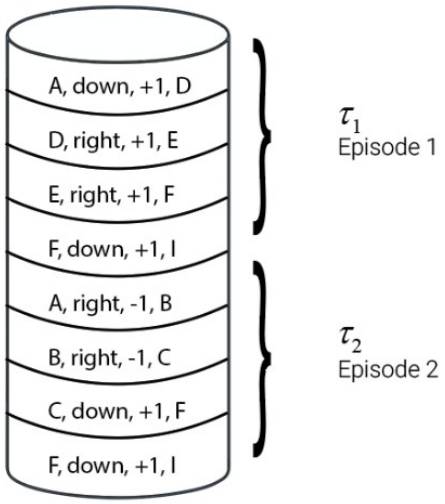
Implementing the DQN

The Replay buffer and Loss

Loss function: Diffference between approximate Q

$$\text{MSE} = \frac{1}{K}\sum_{i=1}^{K}(y_i - \hat{y}_i)^2$$

Replay Buffer



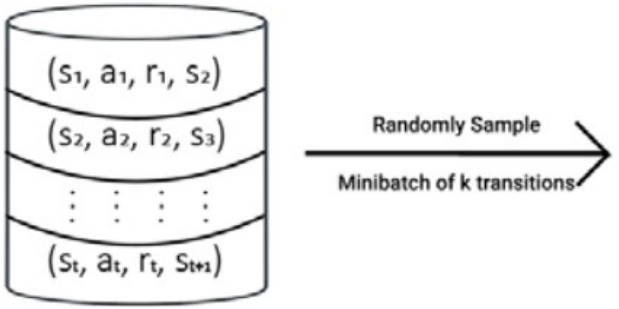
$$L(\theta) = Q^*(s,a) - Q_\theta(s,a)$$

target predicted

$$Q^*(s,a) = \underset{s' \sim P}{E}\left[r + \gamma \max_{a'} Q^*(s',a')\right]$$

$$Q^*(s,a) = r + \gamma \max_{a'} Q^*(s',a')$$

$$L(\theta) = r + \gamma \max_{a'} Q(s',a') - Q_\theta(s,a)$$



$$L(\theta) = \frac{1}{K}\sum_{i=1}^{K}(r_i + \gamma \max_{a'} Q_\theta(s_i', a') - Q_\theta(s_i, a_i))^2$$

Compute with gradient

$$\theta = \theta - \alpha \nabla_\theta L(\theta)$$

Loss Function details

$$L(\theta) = \frac{1}{K}\sum_{i=1}^{K}(r_i + \gamma \max_{a'} Q_\theta(s_i', a') - Q_\theta(s_i, a_i))^2$$

Compute with θ

Improved (for main and target networks of DQN):

$$L(\theta) = \frac{1}{K}\sum_{i=1}^{K}(r_i + \gamma \max_{a'} Q_{\theta'}(s_i', a') - Q_\theta(s_i, a_i))^2$$

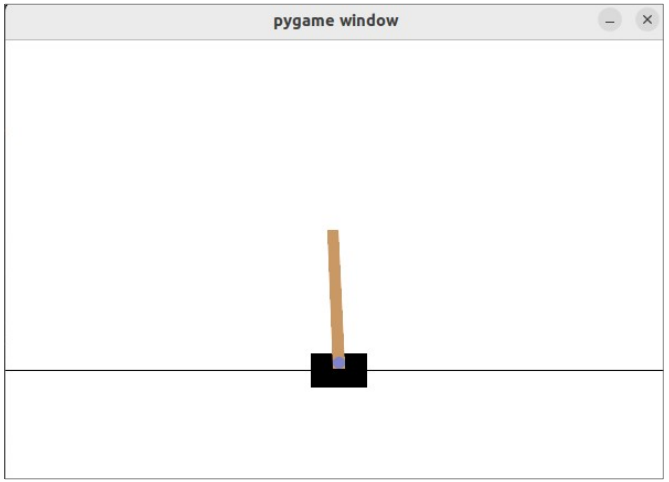
Compute with θ′

Compute with θ

DQN algorithm

1. Initialize the main network parameter θ with random values
2. Initialize the target network parameter θ' by copying the main network parameter θ
3. Initialize the replay buffer $\mathcal{D}$
4. For N number of episodes, perform *step 5*
5. For each step in the episode, that is, for $t = 0, \ldots, T\text{-}1$:
    1. Observe the state s and select an action using the epsilon-greedy policy, that is, with probability epsilon, select random action a and with probability 1-epsilon, select the action $a = \arg\max_a Q_\theta(s, a)$
    2. Perform the selected action and move to the next state s' and obtain the reward r
    3. Store the transition information in the replay buffer $\mathcal{D}$
    4. Randomly sample a minibatch of K transitions from the replay buffer $\mathcal{D}$
    5. Compute the target value, that is, $y_i = r_i + \gamma \max_{a'} Q_{\theta'}(s_i', a')$
    6. Compute the loss, $L(\theta) = \frac{1}{K}\sum_{i=1}^{K}(y_i - Q_\theta(s_i, a_i))^2$
    7. Compute the gradients of the loss and update the main network parameter θ using gradient descent: $\theta = \theta - \alpha\nabla_\theta L(\theta)$
    8. Freeze the target network parameter θ' for several time steps and then update it by just copying the main network parameter θ

Cart pole




```
def visualize_environment(env_name):

    env = gym.make(env_name, render_mode='human')
    env.reset()
    for _ in range(1000):
        env.render()
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)

        if terminated:
            print(terminated)
            break
    env.close()

visualize_environment('CartPole-v1')
```

- Goal: keep the pole standing on the cart by moving the cart left or right.
- Reward: +1 is given for each time step the pole is standing.
- Terminate: if pole is >15 degrees from the vertical, or the cart moves beyond 2.4 units from the center, the game is over

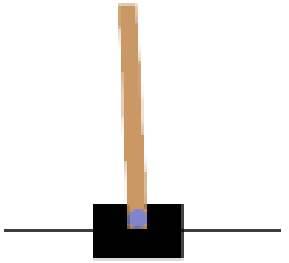
Each time step: `next_state, reward, terminated, truncated, info`

State from step(): [0.1495801 0.36743385 -0.21270445 -0.91803914]

Gives the cosine angle from the vertical

DQN for cart-pole

```
import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
import numpy as np
import random
from collections import deque
```



```
class DQN(nn.Module):
    def __init__(self, input_size, action_size):
        super(DQN, self).__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.fc2 = nn.Linear(64, 128)
        self.fc3 = nn.Linear(128, 64)
        self.fc4 = nn.Linear(64, action_size)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.relu(self.fc3(x))
        return self.fc4(x)
```

A very simple linear network will work.

DQN for cart-pole

The Agent Class

```
class Agent():
   def __init__(self, env_string, batch_size=128, render_mode=None):
       self.memory = deque(maxlen=100000)
       self.env = gym.make(env_string, render_mode=render_mode)
       self.input_size = self.env.observation_space.shape[0]
       self.action_size = self.env.action_space.n
       self.batch_size = batch_size
       self.gamma = 0.5
       self.epsilon = 0.8
       self.epsilon_min = 0.01
       self.epsilon_decay = 0.9995
       #self.device = torch.device("mps"
           if torch.backends.mps.is_available() else "cpu")
       self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

       self.model = DQN(self.input_size, self.action_size).to(self.device)
       self.optimizer = optim.Adam(self.model.parameters(), lr=0.0005)
       self.scheduler = lr_scheduler.ReduceLROnPlateau(self.optimizer,
                              mode='min', factor=0.5, patience=5, min_lr=1e-6)
       self.loss_history = []
```

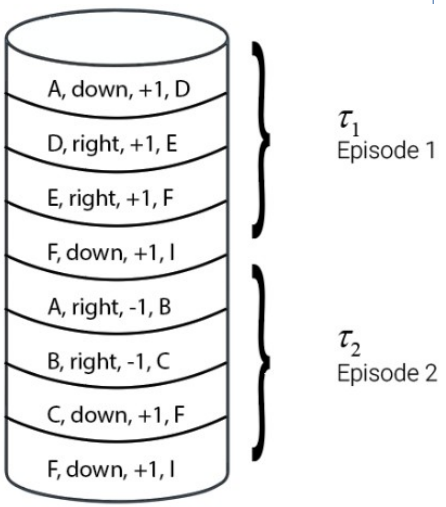
The Agent Class

```
def preprocess_state(self, state):
    if isinstance(state, tuple):
        state = state[0]
    state = np.array(state, dtype=np.float32)
    if state.ndim == 1:
        state = np.expand_dims(state, axis=0)
    return state

def remember(self, state, action, reward, next_state, done):
    self.memory.append((state, action, reward, next_state, done))


def choose_action(self, state, epsilon):
    if np.random.rand() <= epsilon:
        return self.env.action_space.sample()
    else:
        state = torch.FloatTensor(state).to(self.device)
        with torch.no_grad():
            action_values = self.model(state)
        return torch.argmax(action_values).item()
```

The Agent Class

```
def replay(self):
    if len(self.memory) < self.batch_size:
        return 0

    minibatch = random.sample(self.memory, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*minibatch)
    states = np.vstack(states)
    next_states = np.vstack(next_states)
    actions = np.array(actions)
    rewards = np.array(rewards)
    dones = np.array(dones)
    states = torch.FloatTensor(states).to(self.device)
    next_states = torch.FloatTensor(next_states).to(self.device)
    actions = torch.LongTensor(actions).to(self.device)
    rewards = torch.FloatTensor(rewards).to(self.device)
    dones = torch.FloatTensor(dones).to(self.device)
    current_q_values = self.model(states).gather(1, actions.unsqueeze(1)).squeeze(1)
    next_q_values = self.model(next_states).max(1)[0]
    expected_q_values = rewards + self.gamma * next_q_values * (1 - dones)
    loss = nn.MSELoss()(current_q_values, expected_q_values.detach())
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
    return loss.item()
```

- This is the network playing back the episodes with minibatches.



5 *Implementing the DQN*

```
def train(self, epochs=300, threshold=195):
   scores = deque(maxlen=100)
   avg_scores = []
   for epoch in range(epochs):
      state = self.env.reset()
      state = self.preprocess_state(state)
      done = False
      i = 0
      epoch_loss = 0
      num_batches = 0
       while not done:
          action = self.choose_action(state, self.epsilon)
          next_state, reward, terminated, truncated, info = self.env.step(action)
          done = terminated or truncated
          next_state = self.preprocess_state(next_state)
          self.remember(state, action, reward, next_state, done)
          state = next_state
          self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)
          batch_loss = self.replay()
          epoch_loss += batch_loss
          num_batches += 1
          I += 1

      avg_epoch_loss = epoch_loss / num_batches if num_batches > 0 else 0
      self.loss_history.append(avg_epoch_loss)
      if (epoch + 1) % 10 == 0:
         print(f'Epoch {epoch + 1}, Avg Loss: {avg_epoch_loss}')
      scores.append(i)
      mean_score = np.mean(scores)
      avg_scores.append(mean_score)

      if mean_score >= threshold and len(scores) == 100:
              print(f'Ran {epoch} episodes. Solved after {epoch - 100} trials ✔')
              torch.save(self.model.state_dict(), 'dqn_cartpole.pth')
              return avg_scores
      print(f'Did not solve after {epochs} episodes')
      torch.save(self.model.state_dict(), 'dqn_cartpole.pth')
      return avg_scores
```

Training results